
Components
Data Binding
Directives
Services
Dependency Injection
Routing
Observables
RxJS
HttpClient
Forms
Lifecycle Hooks
Standalone Architecture
Route Guards
Interceptors
Signals
Angular
Interview Handbook
by Interview Happy

PREFACE

ABOUT THE BOOK



- **Angular Interview Handbook** is a clear, practical, and interview-focused guide designed to help you learn, revise, and remember Angular concepts step by step.
- This book explains important Angular topics in simple language with practical examples, real-world understanding, and interview-ready explanations.
- Whether you are a beginner, a .NET/Java developer moving into frontend development, or a candidate preparing for Angular interviews, this handbook will help you build a strong foundation and revise quickly before interviews.

Learn
Concepts explained in simple language.

Understand
Practical examples with real-world understanding.

Explain
Interview-ready answers you can speak with confidence.

Succeed
Build strong concepts and crack Angular interviews.

ABOUT THE AUTHOR

ABOUT THE AUTHOR



Happy Rawat

Founder, Interview Happy

Mentoring software professionals for technical interviews

15+ Years of Experience: Happy Rawat has over 15 years of experience in software development and has worked as a full-stack developer in IT and product-based companies.

Mentor & Creator: Through Interview Happy, he mentors software professionals and helps them prepare for technical interviews with simple explanations, practical examples, and interview-focused learning.

Mission: His mission is to help candidates become confident, skilled, and interview ready.

Impact: Thousands of software professionals have improved their concepts, cracked interviews, and accelerated their careers with his guidance.

Website
interviewhappy.com

Email
happy@interviewhappy.com

Angular Roadmap

Zero → Interview Ready

Foundation (Beginner)

- Angular Fundamentals
- TS Overview
- Angular Environment Setup
- Angular Project Files
- Components
- Application Startup Flow
- Decorators – Basics
- Data Binding
- Directives Basics & Structural Directives
- Attribute Directives
- Pipes

Real World (Intermediate)

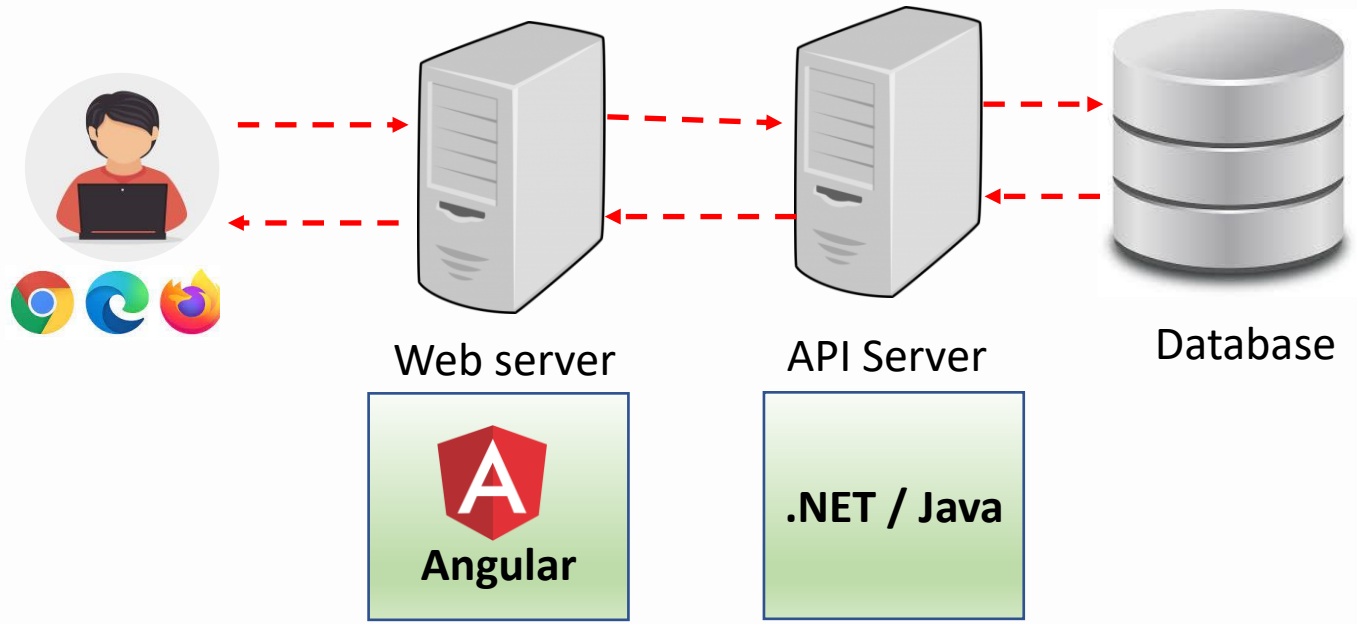
- Advance Decorators
- Components Communication
- Services & DI
- Routing Basics
- Asynchronous programming
- Observables
- HttpClient
- RxJS Operators
- Main Lifecycle Hooks
- Angular Forms - Template Driven
- Angular Forms - Reactive Forms

Interview Ready (Expert)

- JWT Authentication
- Route Guards
- Interceptor
- Signals
- Angular 20+ Versions
- Standalone Architecture
- ViewChild & ViewChildren
- ngAfterViewInit() lifecycle hook
- Content Projection & ng-content
- ContentChild & ContentChildren
- ngAfterContentInit() lifecycle hook

Angular Fundamentals

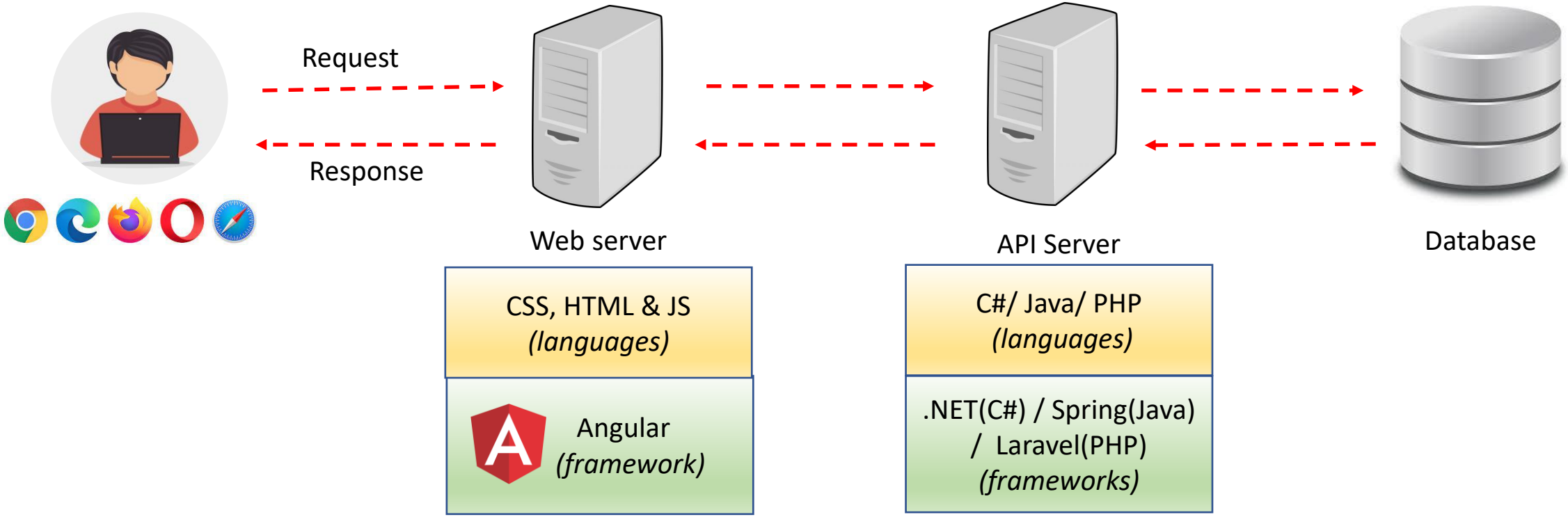
- 👉 Introduction to Angular
- 👉 Components in Angular
- 👉 Single Page Application
- 👉 Angular Main Features



Q. What is Angular? V. IMP.



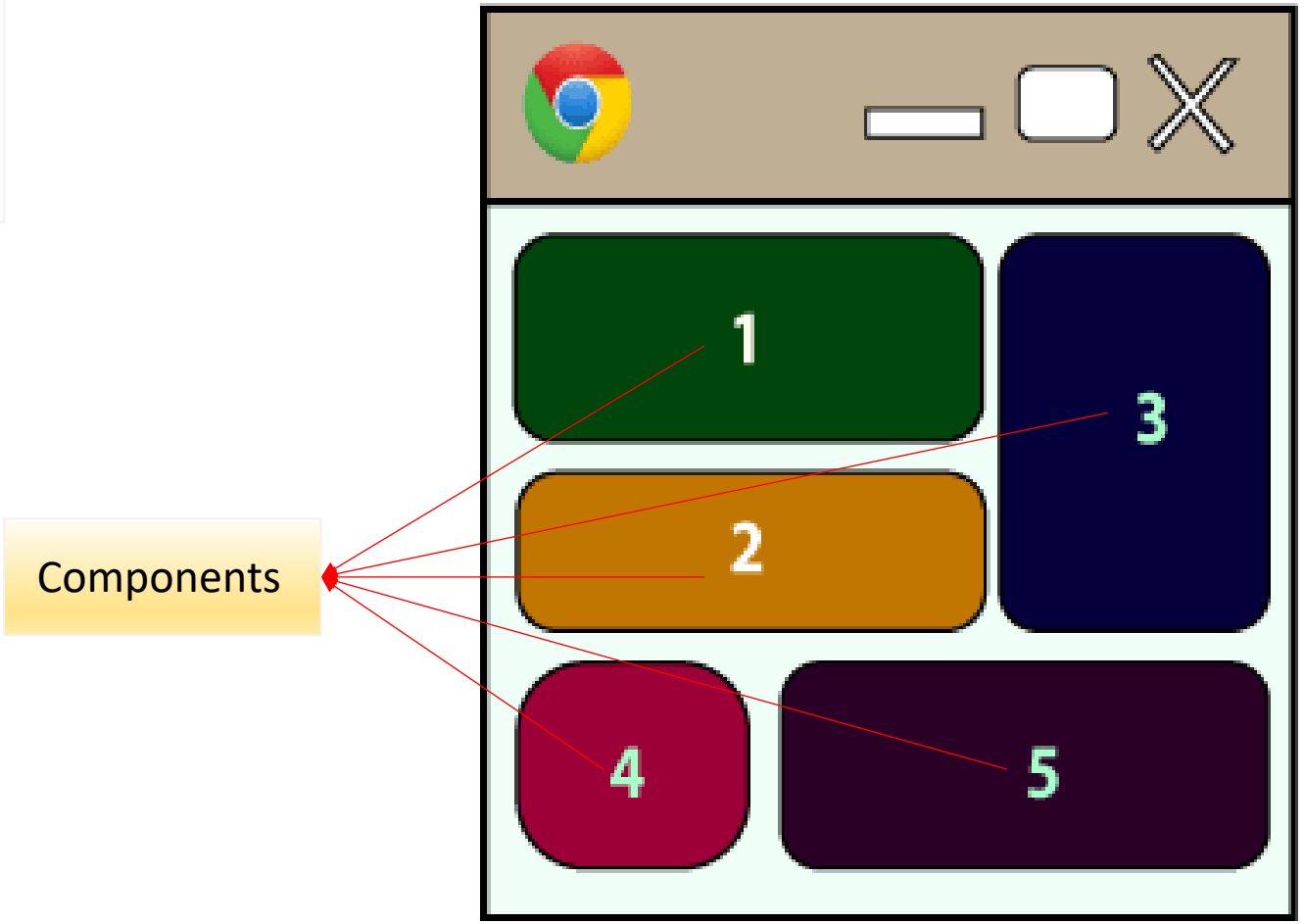
- Angular is a **component-based** front-end framework used to build dynamic **single-page web applications (SPAs)** using TypeScript.



Q. What are Components in Angular?



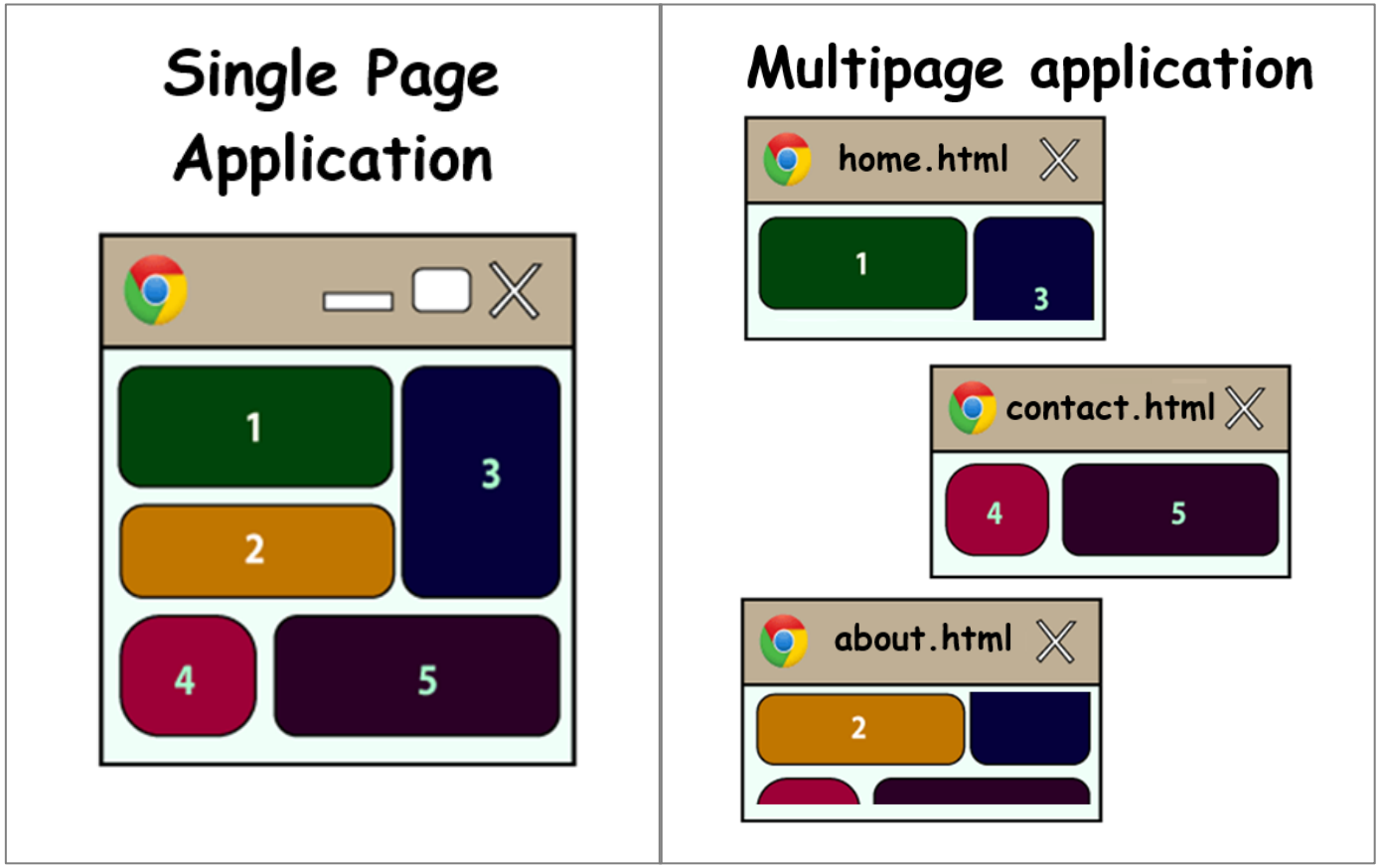
- A component is a **reusable building block** used to create user interfaces in single-page applications.



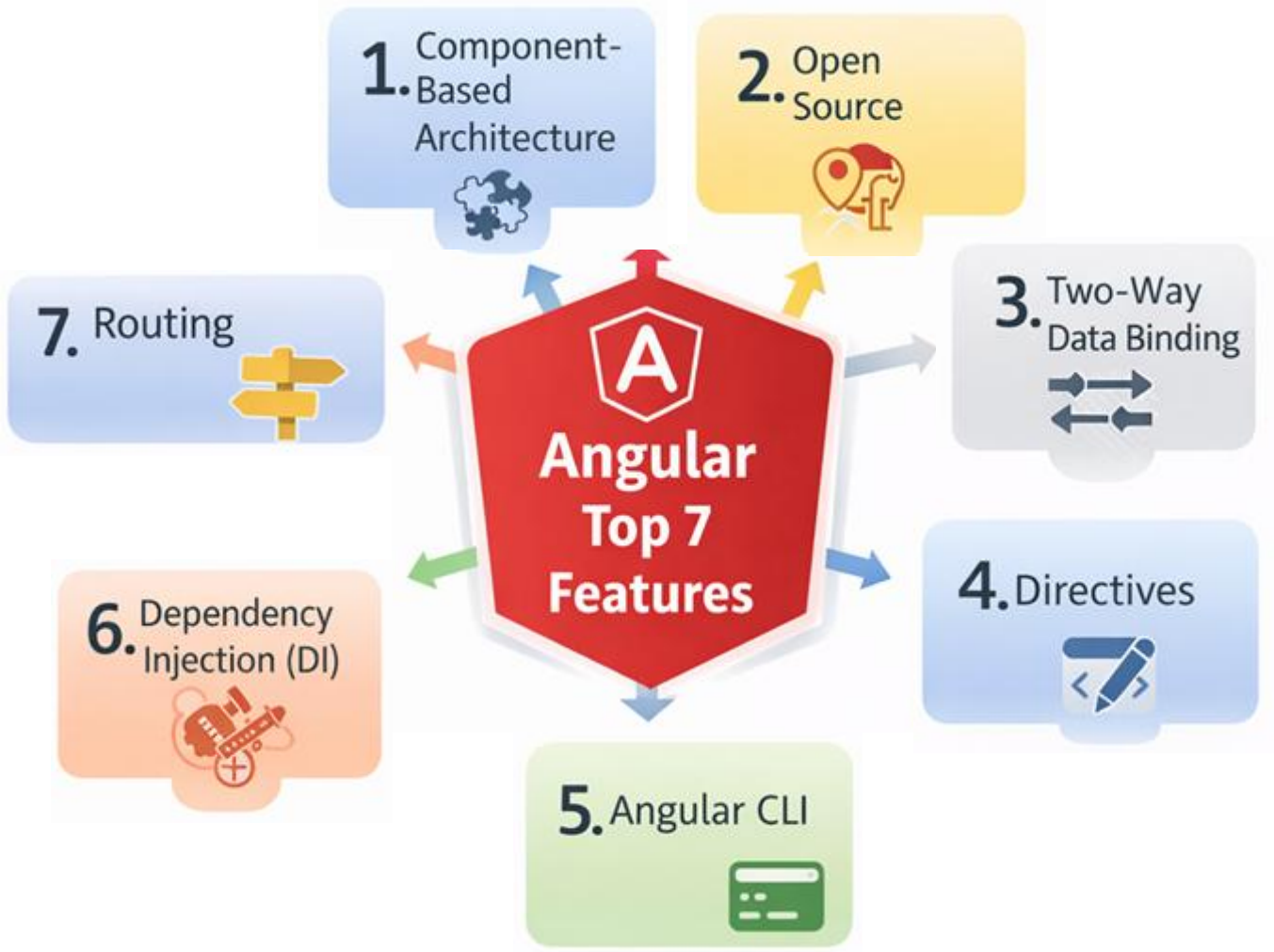
Q. What is a Single Page Application?



- A Single Page Application (SPA) is a web application that **loads only one HTML page** and updates the content dynamically without reloading the entire page.

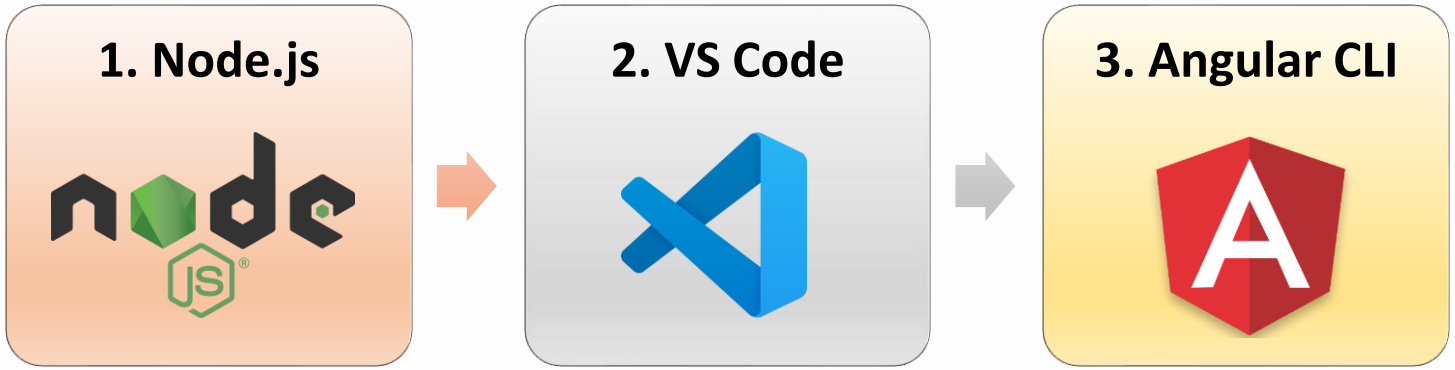


Q. What are the Top 7 Features of Angular?



Angular Environment Setup

- 👉 Angular Software Setup
- 👉 Role of NPM
- 👉 Angular CLI
- 👉 First Angular Project
- 👉 Live Reload



Q. What software's are required to run Angular applications?



1. Node.js

- *https://nodejs.org*

2. VS Code

- *https://code.visualstudio.com/*

3. Angular CLI

- *npm install -g @angular/cli*
- *ng version*

Q. What are VS Code extensions required for Angular development?



❖ **Top 3 Extensions for VS Code:**

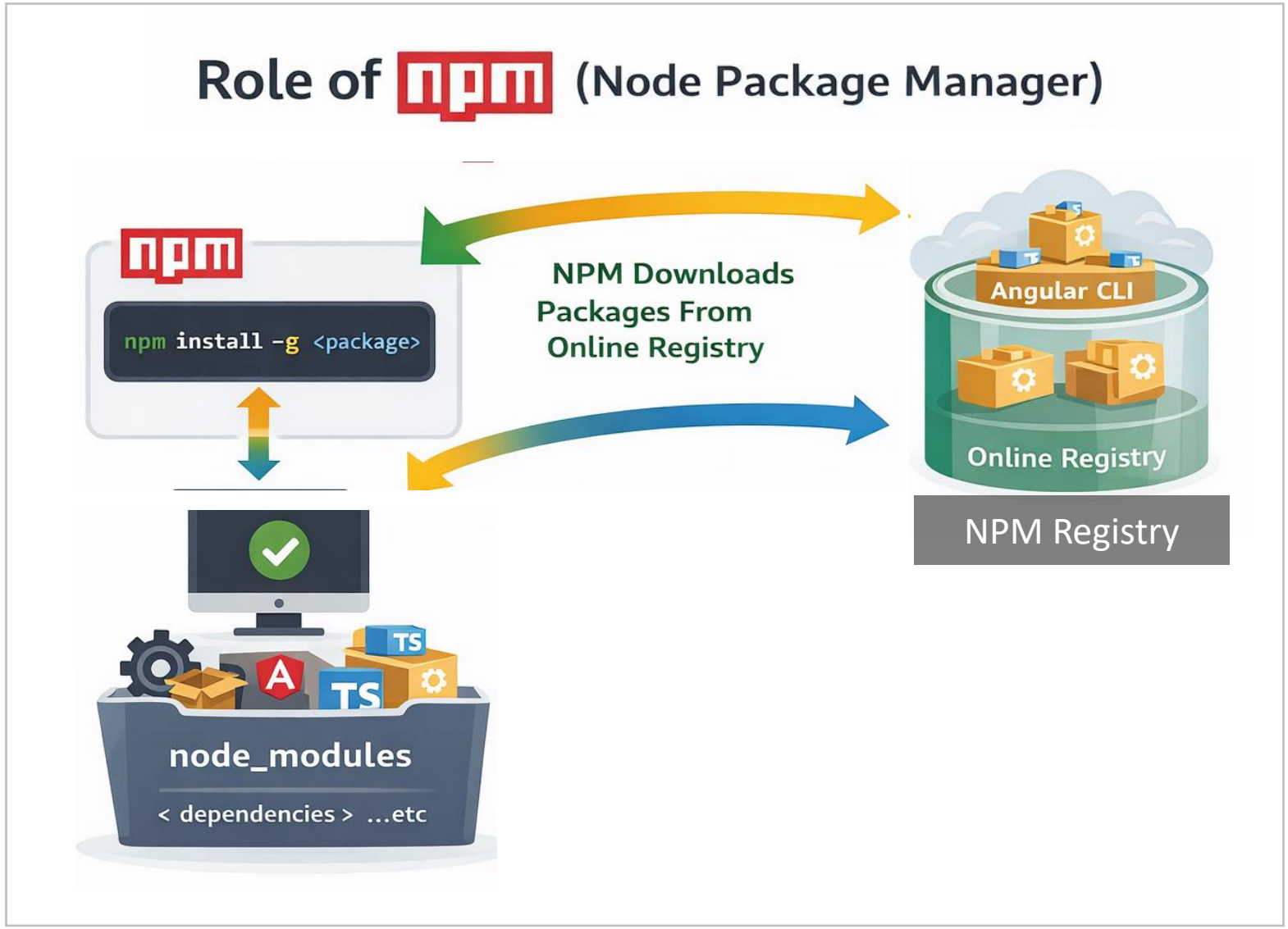
1. **Angular Language Service**
   Auto-complete + error detection in Angular templates
2. **Prettier - Code Formatter**
   Automatically formats your code
3. **ESLint**
   Finds mistakes and enforces best practices



Q. What is the role of NPM in Angular?



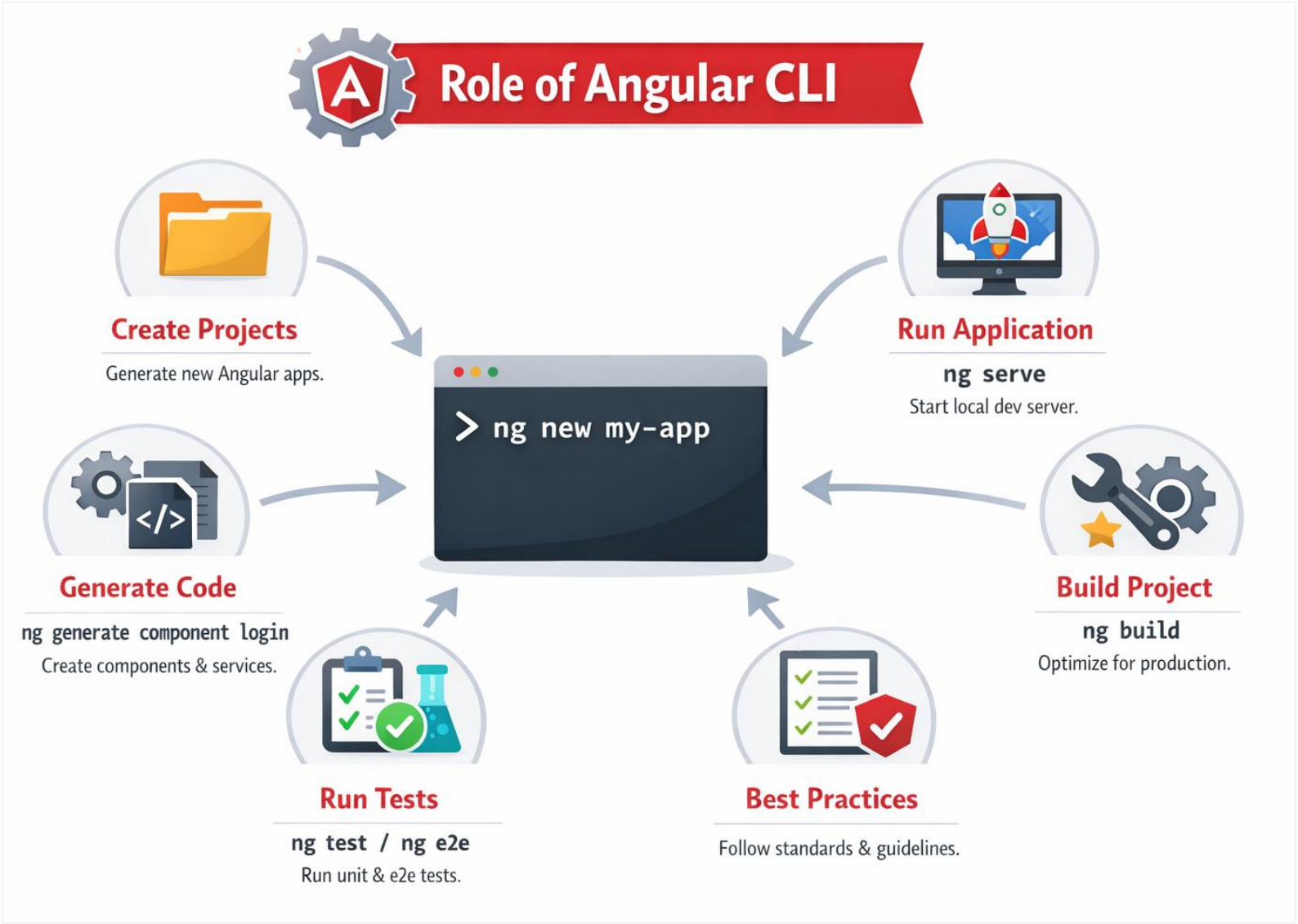
- NPM (Node Package Manager) is a tool that comes with Node.js and is **used to download, install, update, and manage packages (libraries)** from an online repository called the NPM Registry.



Q. What is the role of the Angular CLI?



- Angular CLI is a command-line tool that helps you **create, build, and manage** Angular projects easily.



Q. How you can setup and create your First Angular project?



1. Install Node.js

- *https://nodejs.org*
- Install LTS version

```
node -v
npm -v
```

2. Install VS Code

- *https://code.visualstudio.com/*
- Used to write and manage Angular code

3. Install Angular CLI

- *npm install -g @angular/cli*
- *ng version*

```
npm install -g @angular/cli
```

4. Create a New Angular Project

```
ng new my-first-app
```

- Choose routing: Yes
- Choose stylesheet: CSS

5. Run Your Application

```
cd my-first-app
```

- *cd my-first-app*
- *ng serve*

6. Open in Browser

- Open browser and go to:

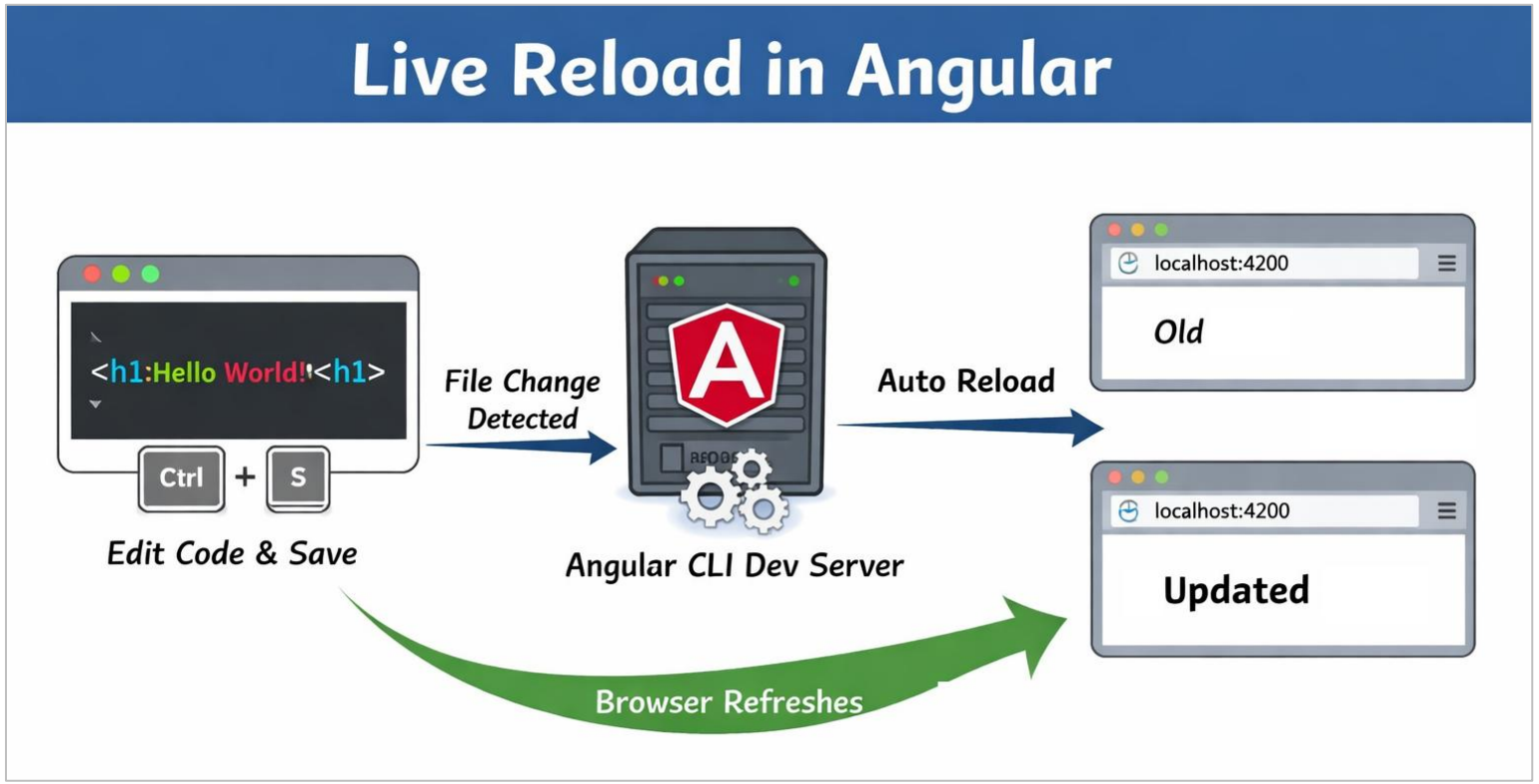
```
http://localhost:4200
```

Your Angular application is now running!

Q. What is Live Reload in Angular?

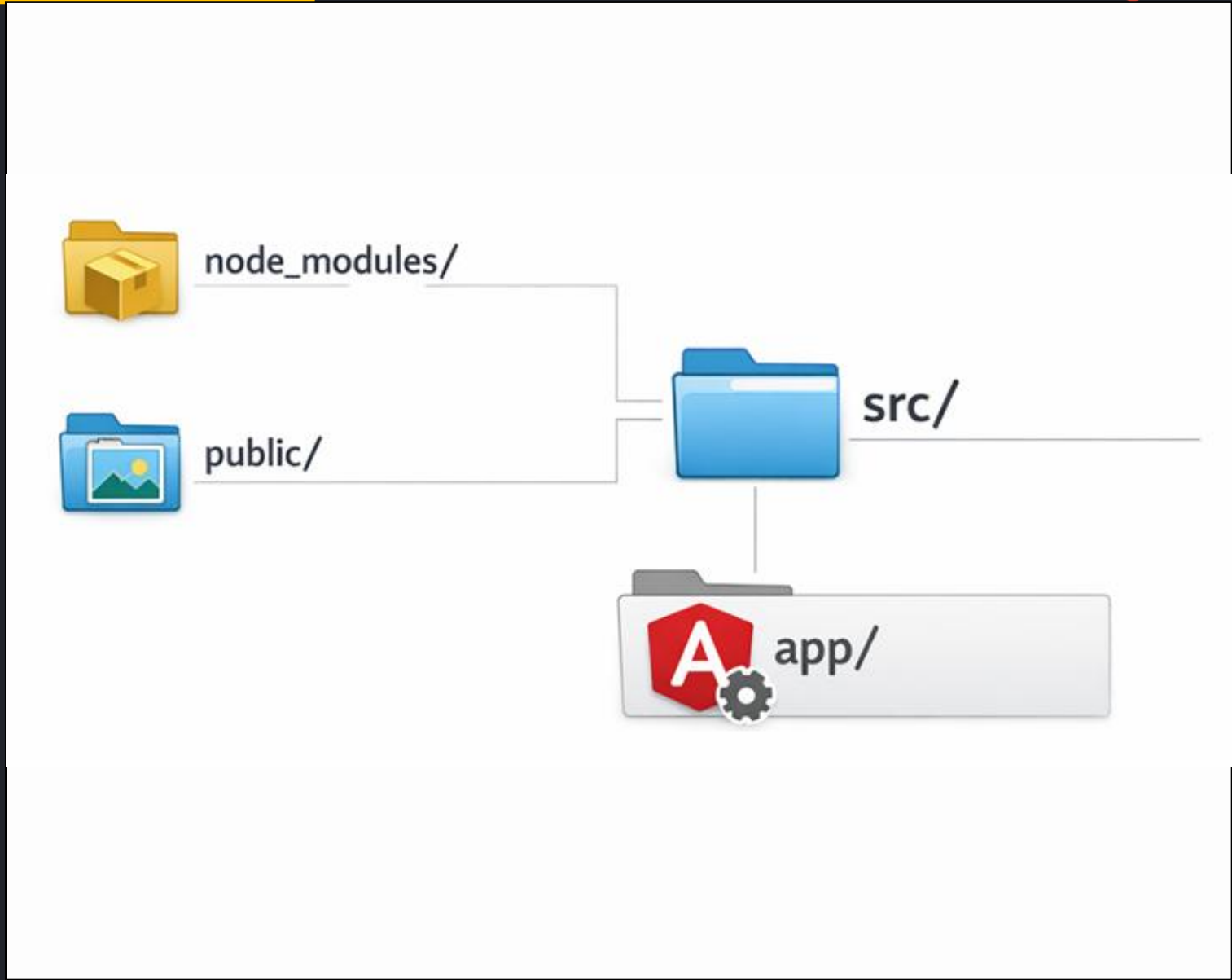


- In Angular, Live Reload is a feature of ng serve that **automatically refreshes the browser whenever you save changes in your code**, HTML, or CSS files.



Angular Project Files

- 👉 package.json
- 👉 node_modules
- 👉 Angular Folder Structure
- 👉 Run Angular Project



Q. What is the role of package.json file & node_modules folder?

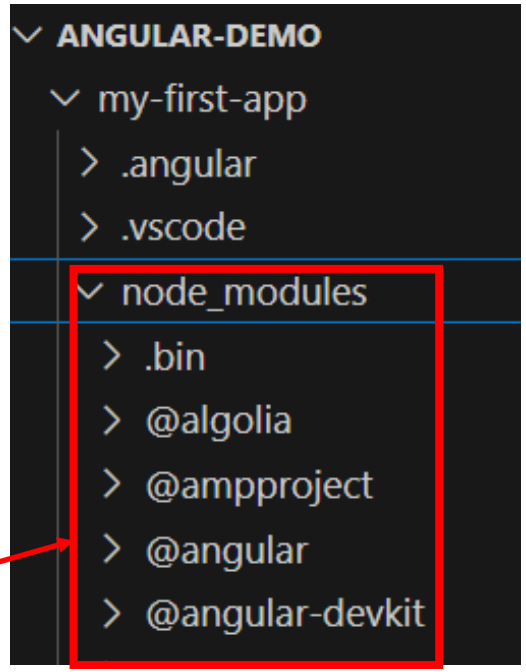


- **package.json:** It keeps track of all the libraries your project needs and the commands to run or build your app.

- **Node_modules**: NPM reads that package.json file and downloads all those libraries from the internet into a folder named node_modules.

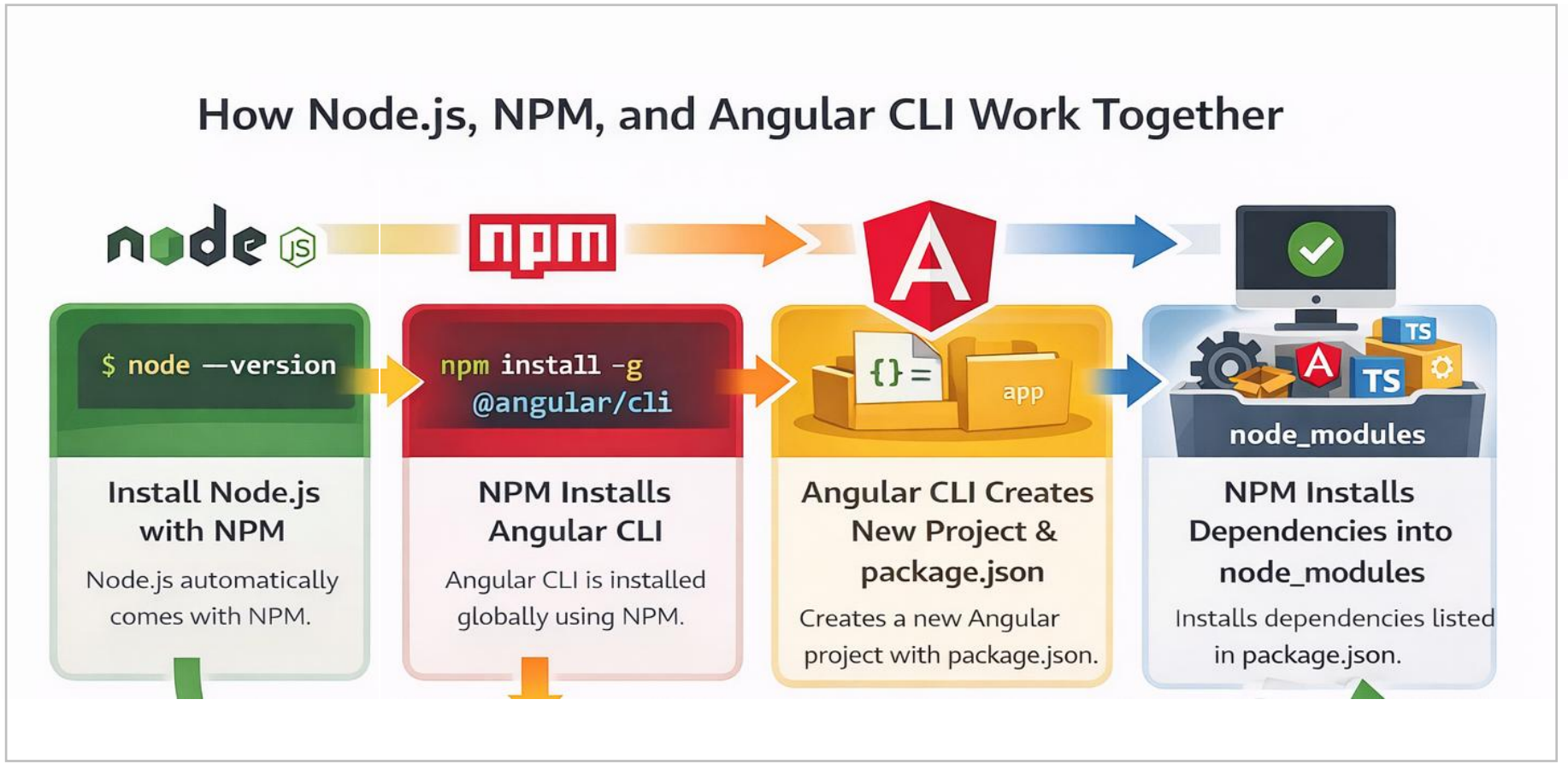
```
// package.json

{
  "name": "my-first-app",
  "version": "0.0.0",
  "scripts": {
    "ng": "ng",
    "start": "ng serve",
    "build": "ng build",
    "watch": "ng build --watch --
configuration development",
    "test": "ng test",
    "serve:ssr:my-first-app": "node dist/my-
first-app/server/server.mjs"
  },
  "private": true,
  "dependencies": {
    "@angular/common": "^20.3.0",
    "@angular/compiler": "^20.3.0",
    "@angular/core": "^20.3.0",
    "@angular/forms": "^20.3.0",
    "@angular/platform-browser": "^20.3.0",
```

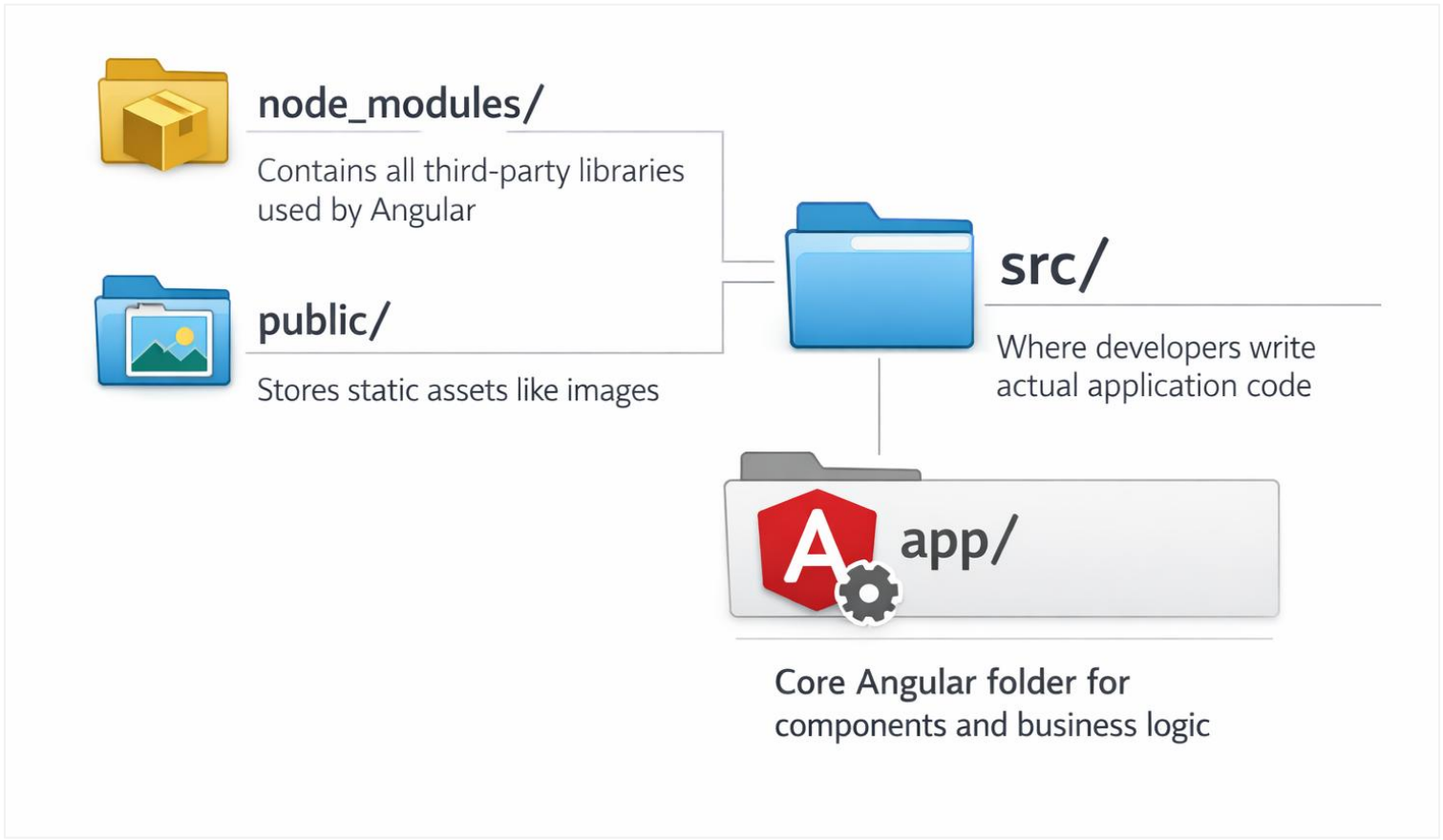


Q. Summarize how Node.js, NPM, Angular CLI, package.json & node_modules work together.





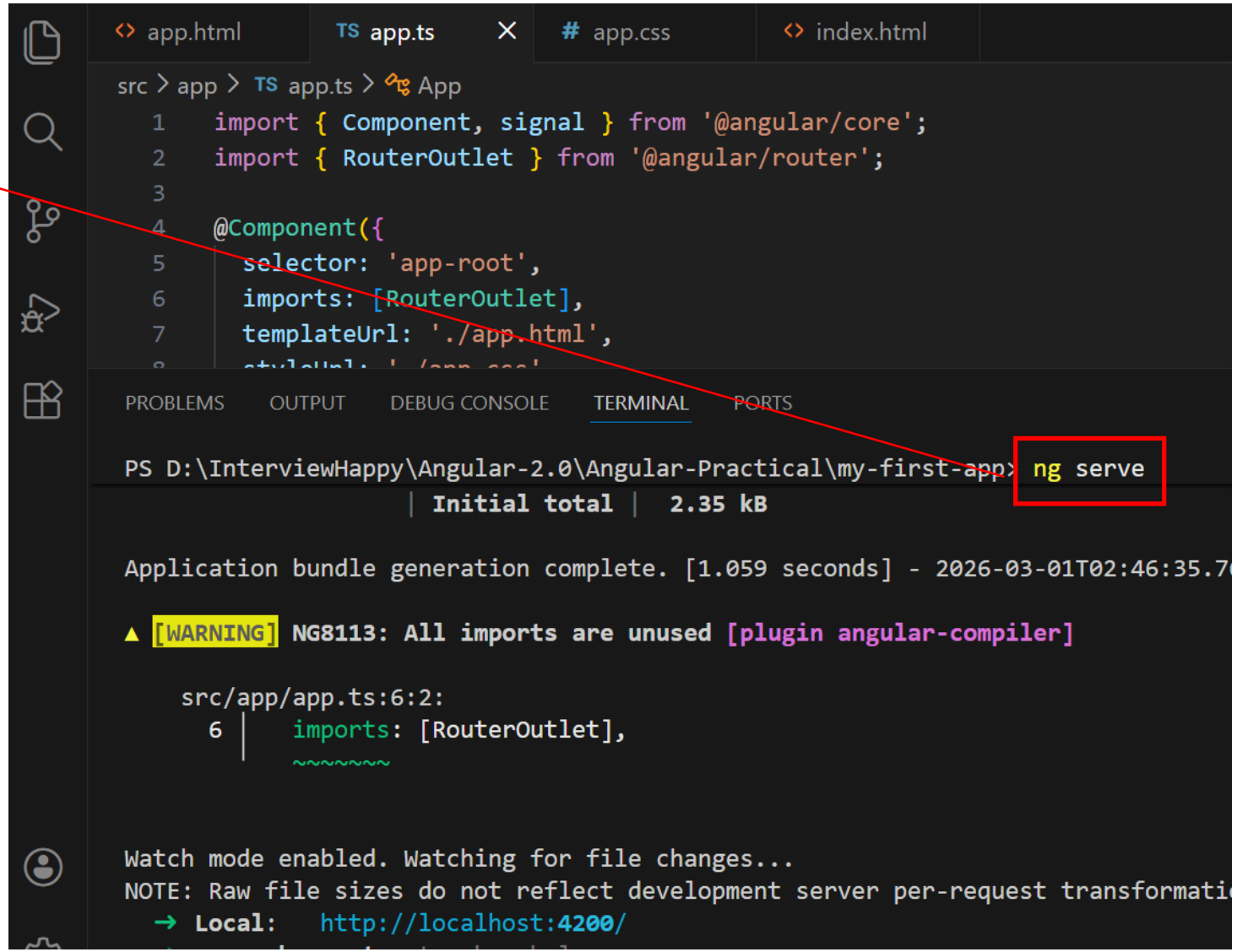
Q. Explain the basic Folder Structure of an Angular project.



Q. What is the purpose of the ng serve command in Angular? How to stop running application?

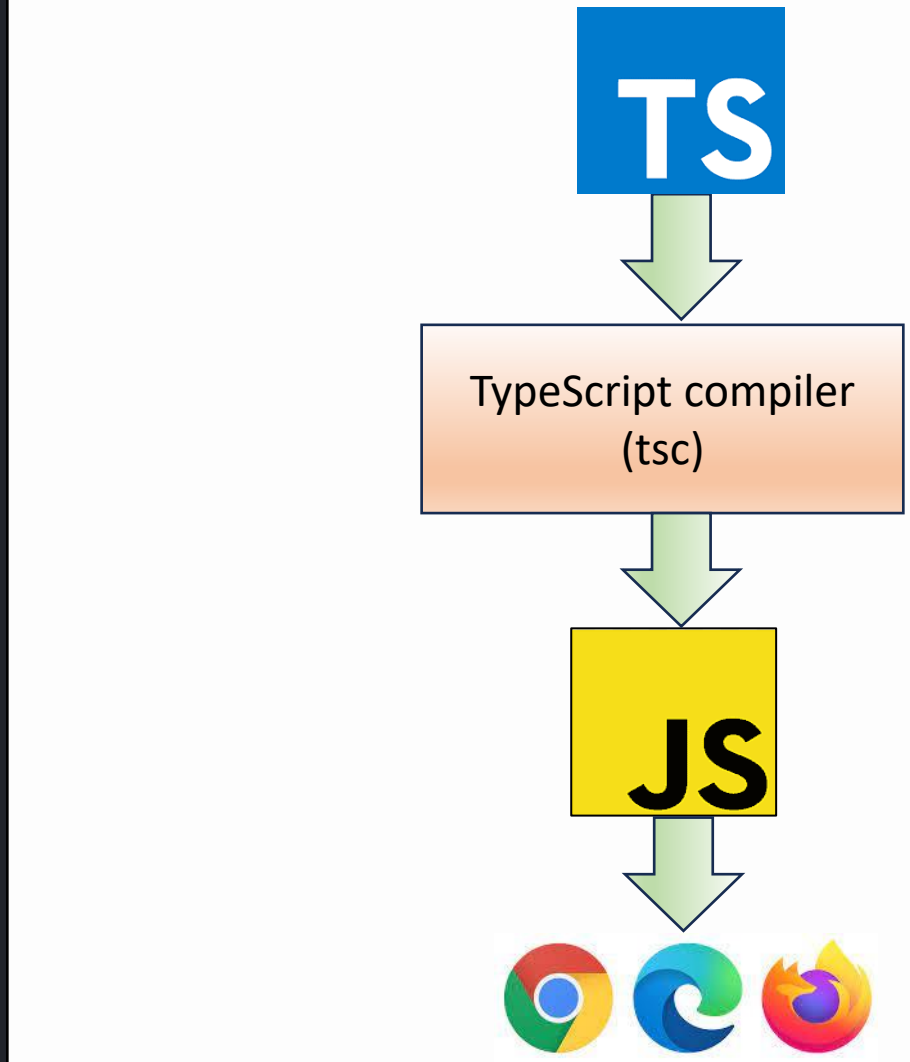


- To run the Angular application: use the **ng serve** command.
- To stop the running Angular application: press **Ctrl + C** in the terminal.



TypeScript Overview

- 👉 Introduction to TS
- 👉 TS Static Typing & Strong Typing



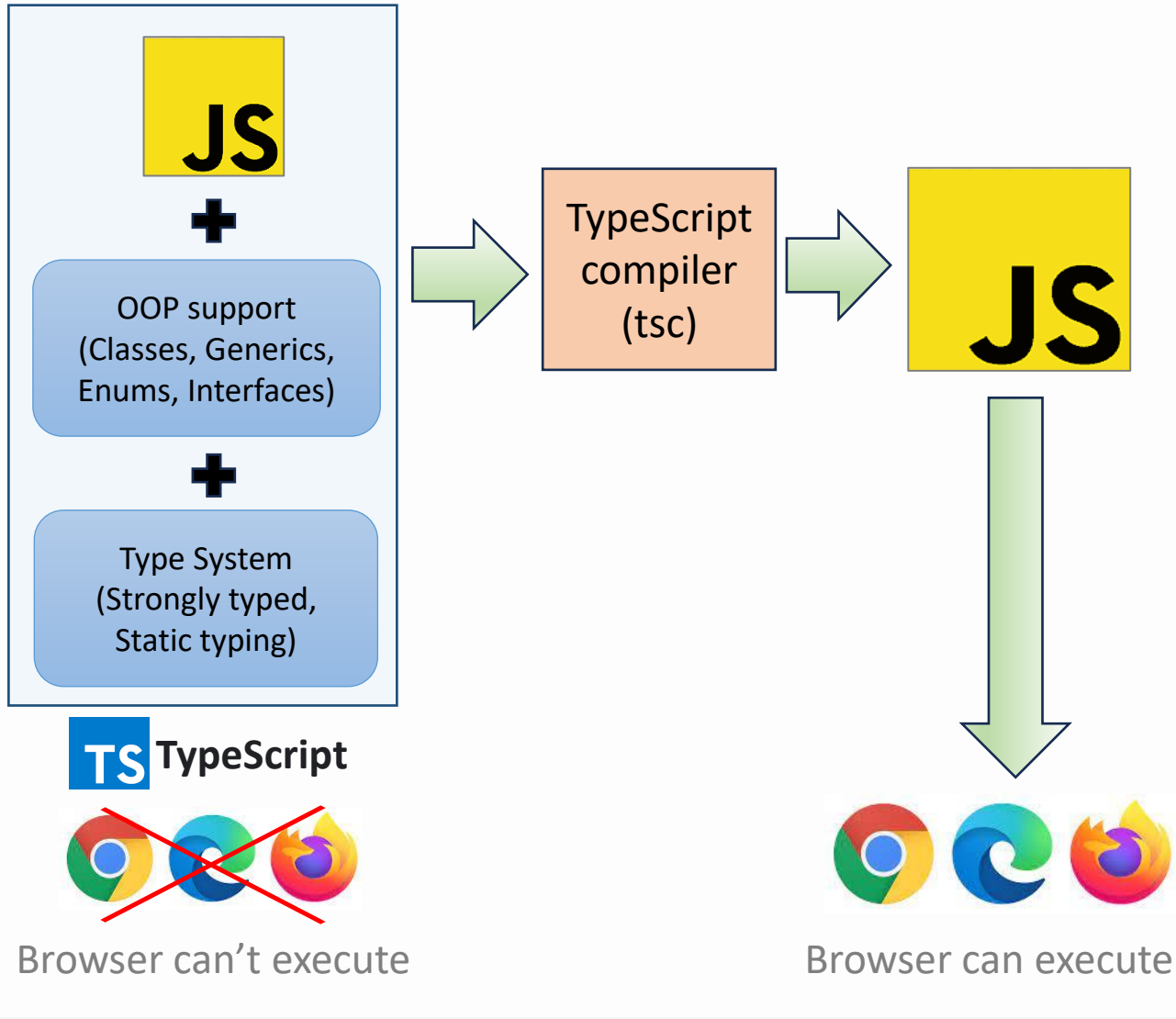
Q. What is TypeScript? How is it different from JavaScript?



- TypeScript is an open-source programming language developed by Microsoft that serves as a **superset of JavaScript**.

- TS advantages over JS:

1. TS is **strongly typed** and supports static typing.
2. TS also adds **object-oriented** features to JS.



Q. How TypeScript is a Strongly Typed language and supports Static Typing?



- TypeScript is **strongly typed** and supports static typing because **it does not allow changing the type** of a variable once it has been defined.

```
// TypeScript is strongly typed

let name: string  = "Happy";

name = 40; // ❌ Compile-time error

// Type 'number' is not assignable to type 'string'
```

- JavaScript is **loosely typed** and supports dynamic typing because **it allows changing the type** of a variable at any time.

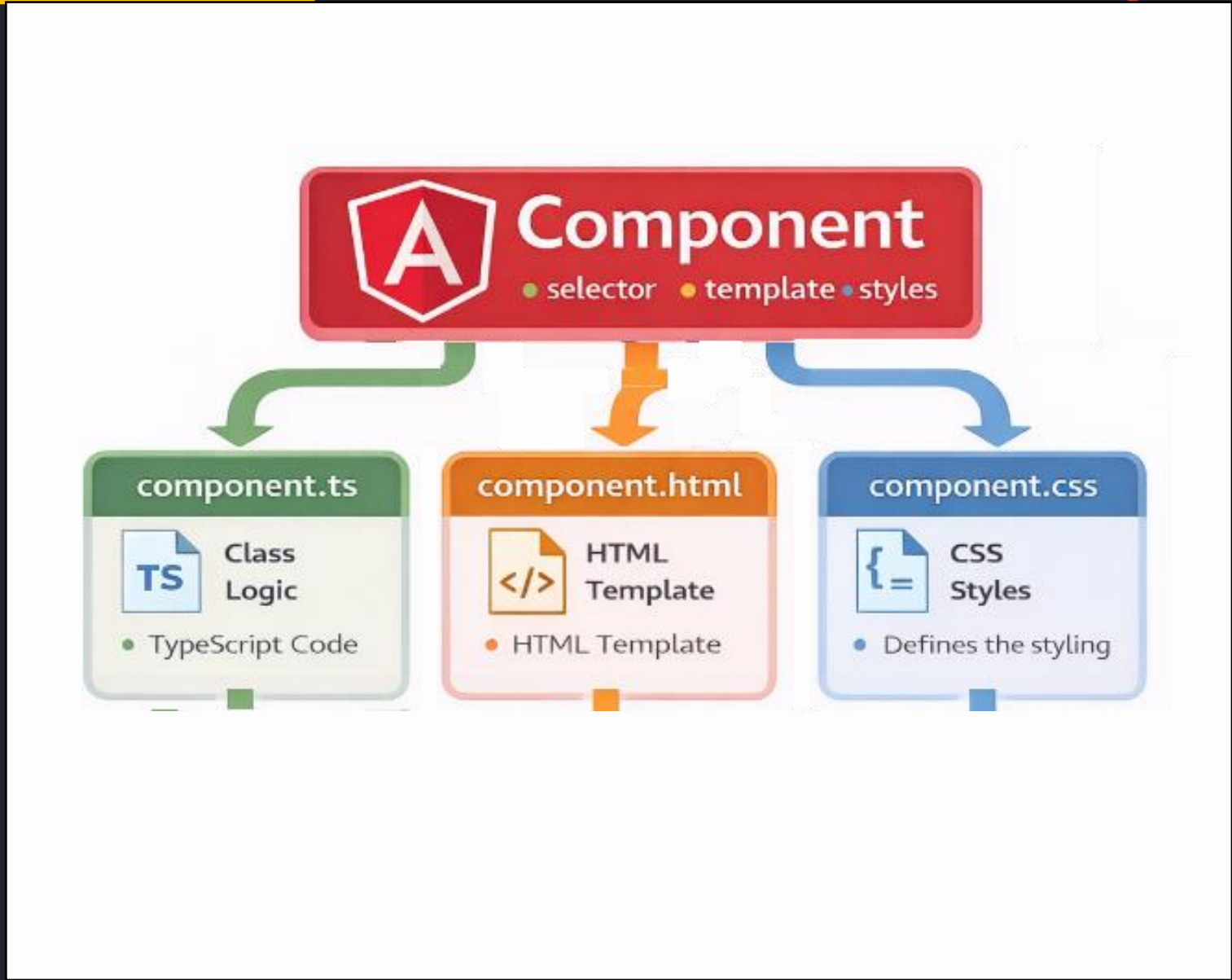
```
// JavaScript is loosely typed

let name = "Happy";

name = 40; // ✅ No error
```

Components

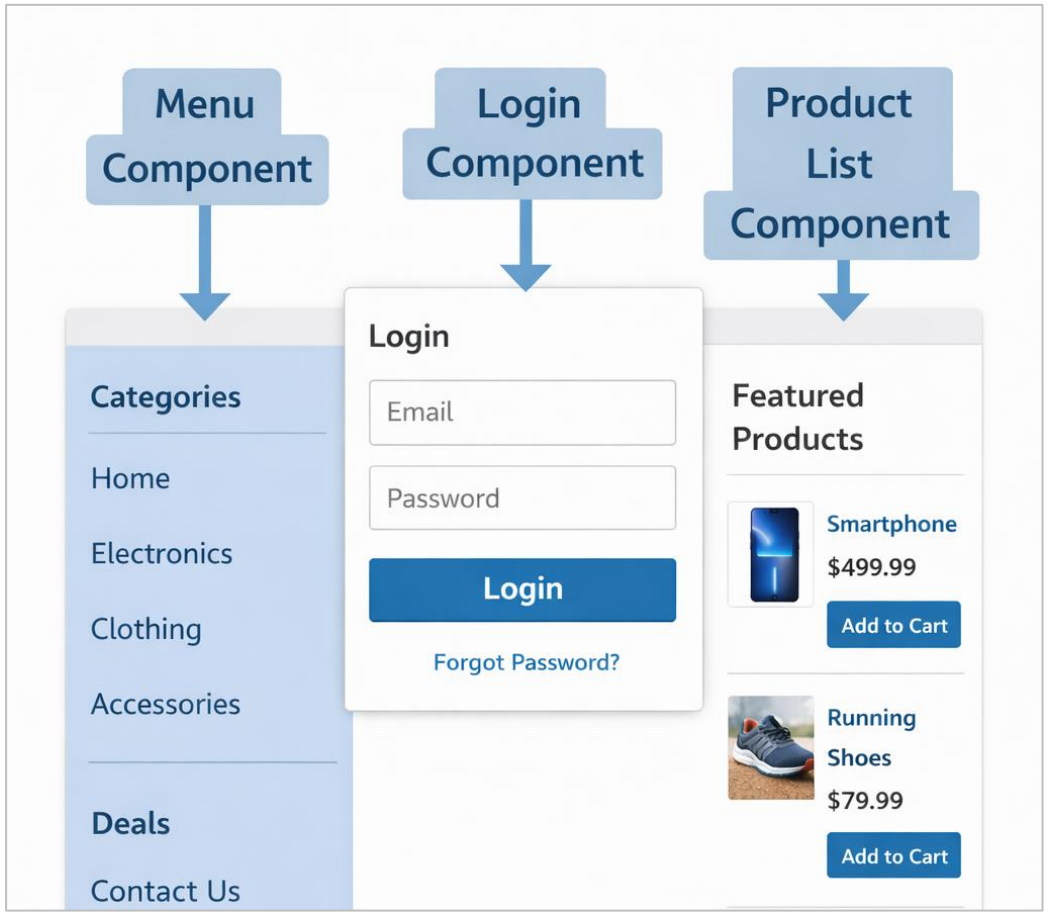
- 👉 App Component
- 👉 Creating First Component
- 👉 Parent-Child Components
- 👉 import & export

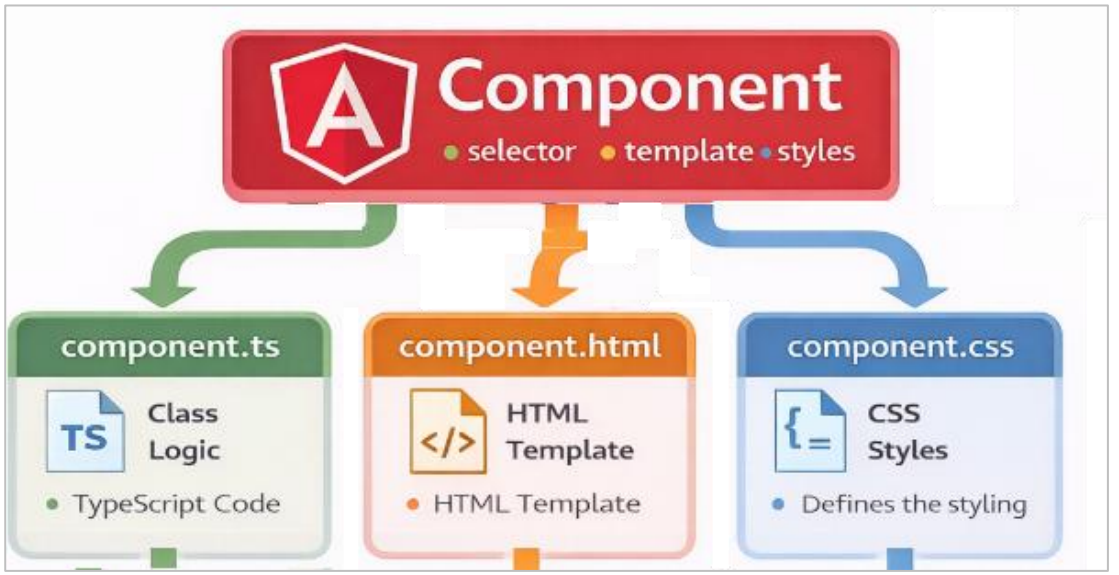


Q. What is App Component in Angular? V. IMP.



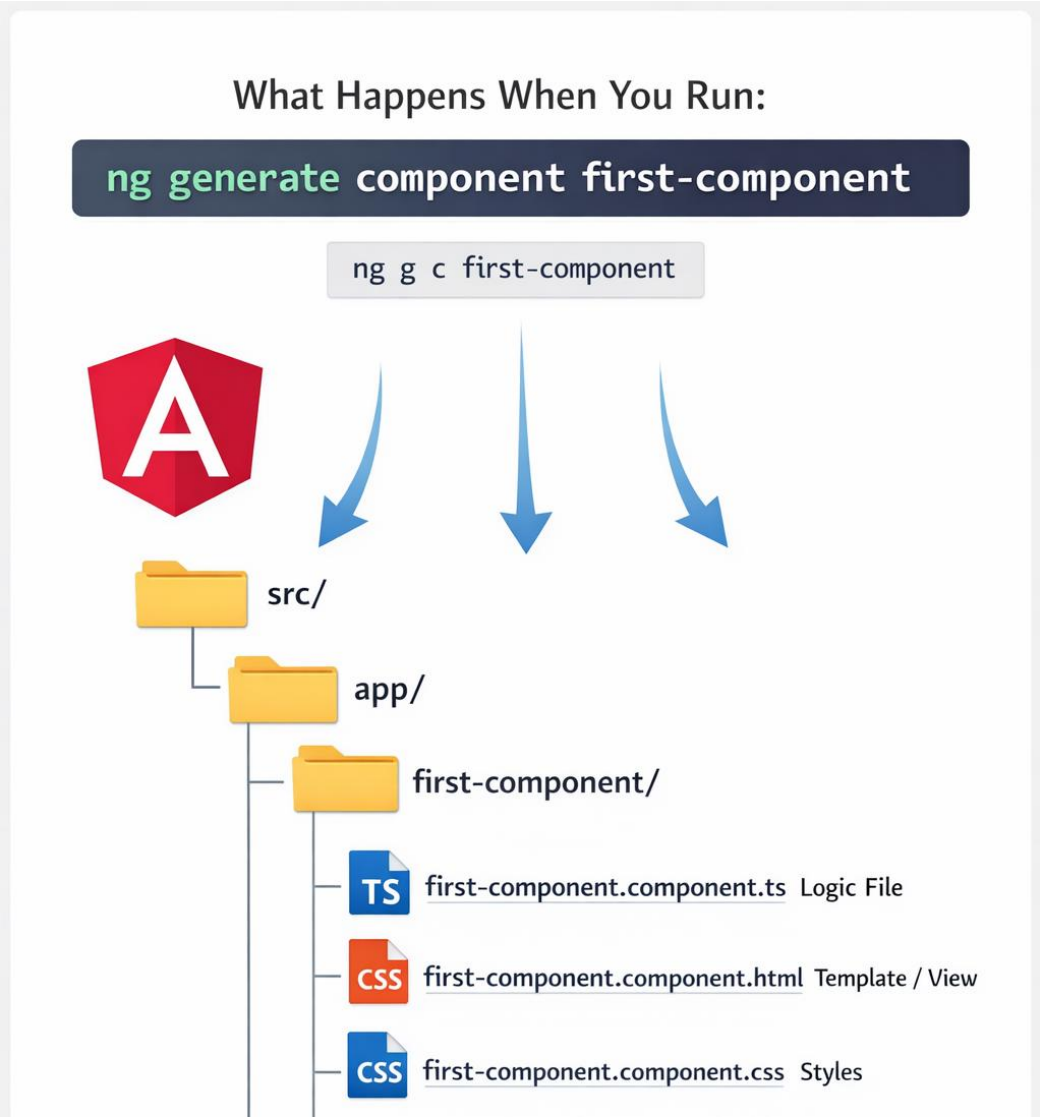
- **App Component** is the main component of your Angular application. It is the first component that loads when the application starts.





Q. How to create your First Component in Angular?

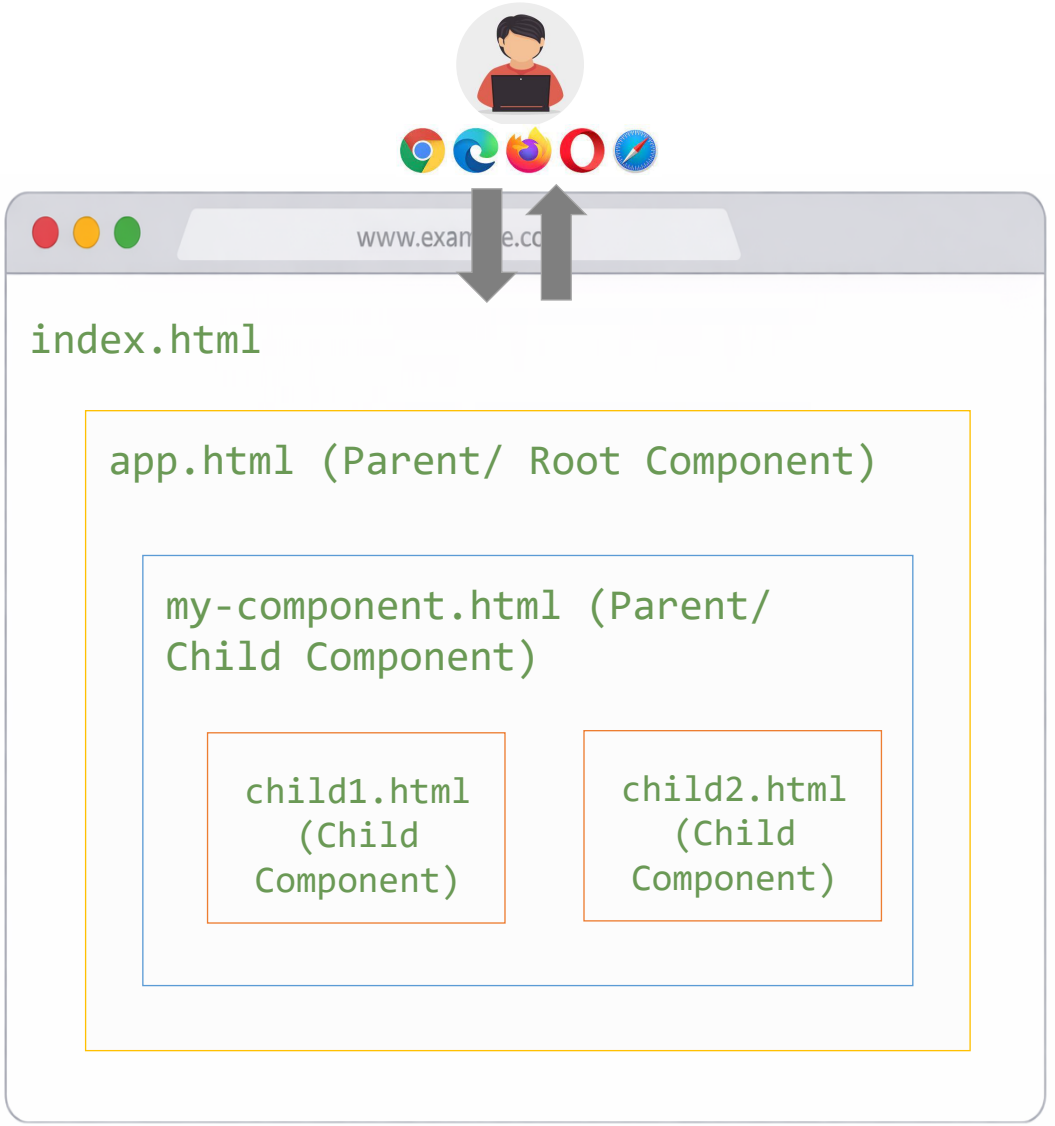




Q. What are Parent-Child Components?



- A Parent - Child relationship means one component (parent) includes another component (child) in its template.
- It is achieved by placing the child component selector inside the parent component's HTML.



Q. What are import and export keywords in component?



- **import keyword:** It is used to bring something (like a class or function) from another file or Angular library into current file.

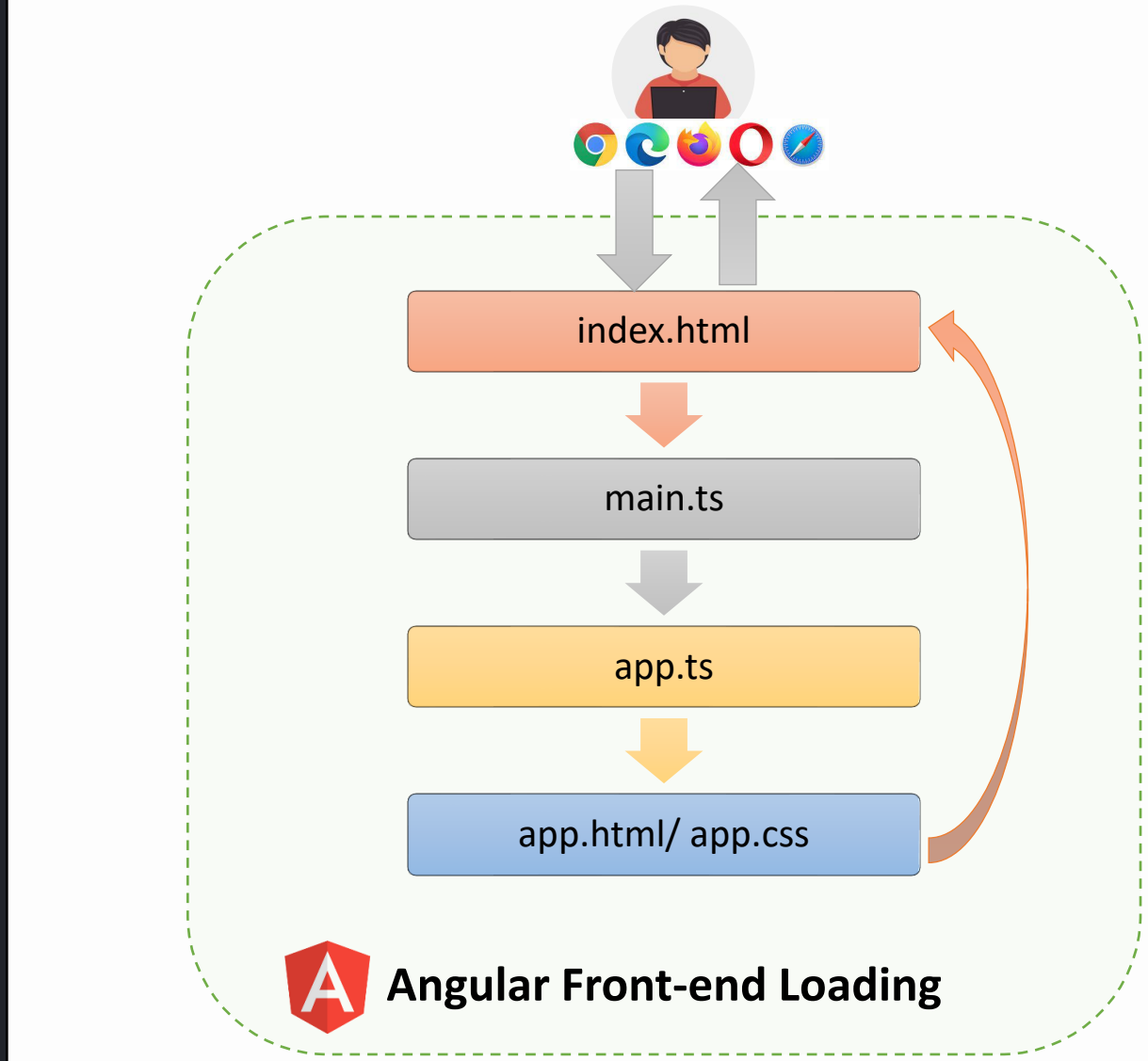
- **export keyword:** It is used to make this class available to other files. If you don't export it, you can't use it in other parts of the app.

```ts
//app.ts
import { Component, signal } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet],
  templateUrl: './app.html',
  styleUrl: './app.css'
})
export class App {
  protected readonly title = signal('my-first-app');
}
```

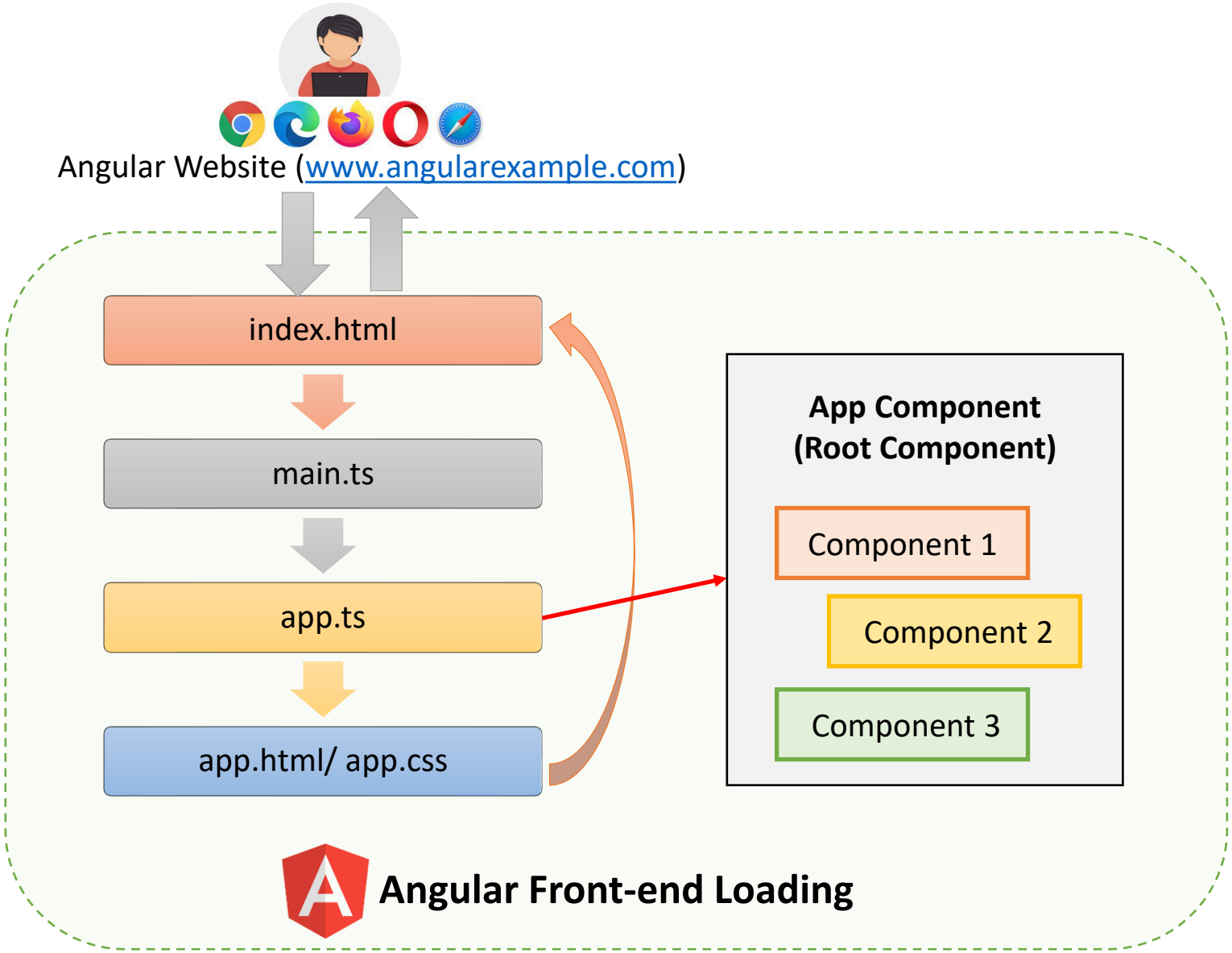
Application Startup Flow

- 👉 Angular Bootstrapping Flow
- 👉 index.html
- 👉 main.ts
- 👉 angular.json



Q. How does an Angular application get loaded and started?





Q. What is the role of index.html file?



- **index.html:** This is the main HTML file of your Angular app. It loads first in the browser and acts as the single page where the whole application runs.

- **<app-root>:** This is just a placeholder tag inside index.html. When the application starts, Angular looks for the selector app-root from the AppComponent, and replaces this tag with the HTML content of AppComponent.

```
<!-- index.html -->
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>MyFirstApp</title>
  <base href="/">
  <meta name="viewport" content="w
  <link rel="icon" type="image/x-i
</head>
<body>
  <app-root></app-root>
</body>
</html>
```

Q. What is the role of main.ts file?



- **main.ts:** This is the entry point of your Angular app. It tells Angular which component to load first and starts the application in the browser.

- **bootstrapApplication():** This method starts the Angular application by loading the root component (AppComponent).

```ts
// main.ts
import { bootstrapApplication } from '@angular/platform-browser';
import { appConfig } from './app/app.config';
import { App } from './app/app';

bootstrapApplication(App, appConfig)
  .catch((err) => console.error(err));
```

Q. What is the role of angular.json file?

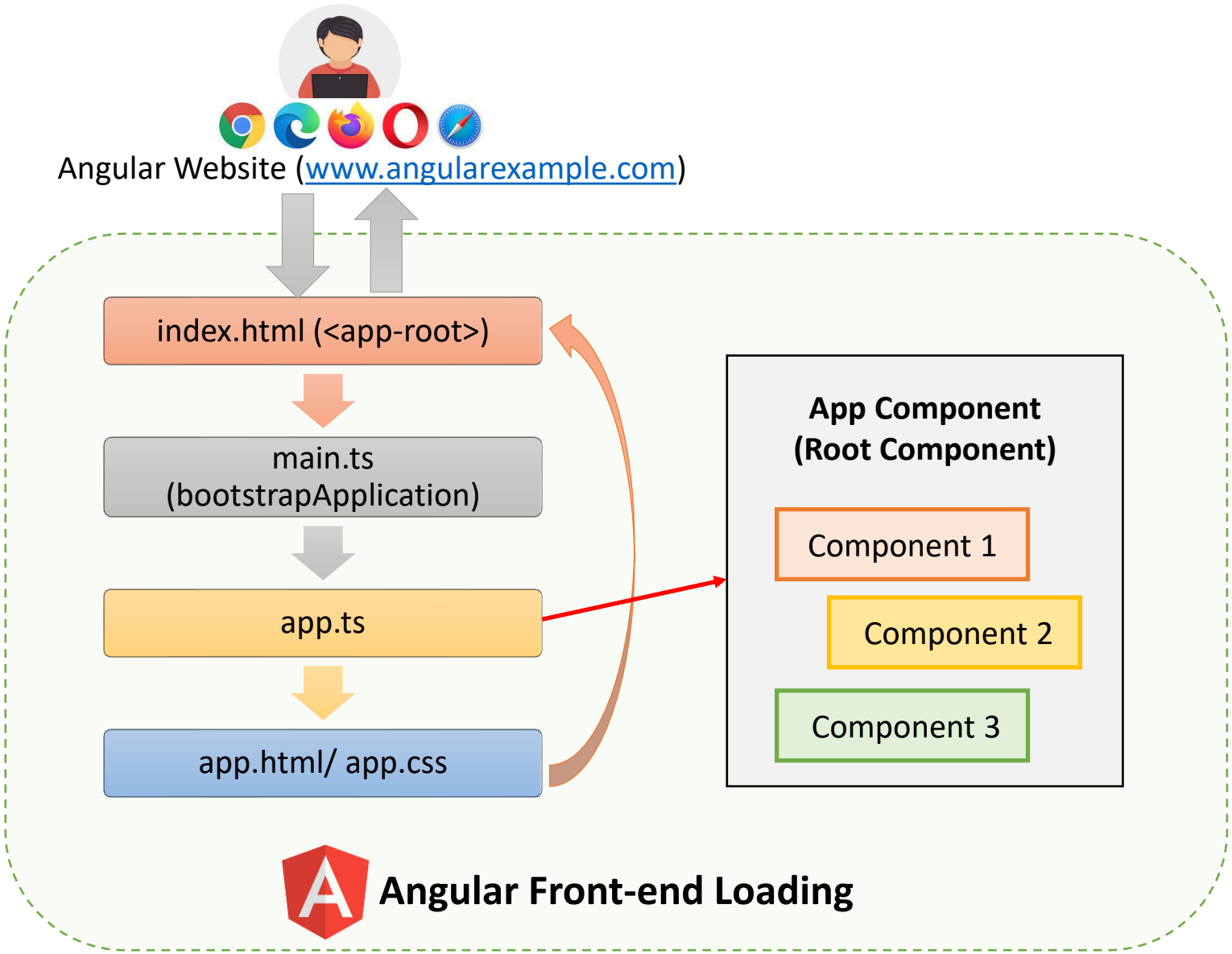


- Angular.json: It is a configuration **setting file** that tells Angular how to build, run, and structure your project.

```
// angular.json
{
  "$schema":
"./node_modules/@angular/cli/lib/config/schema.json"
,
  "version": 1,
  "newProjectRoot": "projects",
  "projects": {
    "my-first-app": {
      "projectType": "application",
      "schematics": {},
      "root": "",
      "sourceRoot": "src",
      "prefix": "app",
      "architect": {
        "build": {
          "builder": "@angular/build:application",
          "options": {
            "browser": "src/main.ts",
            "tsConfig": "tsconfig.app.json",
            "assets": [
              {
                "glob": "**/*",
                "input": "public"
              }
```

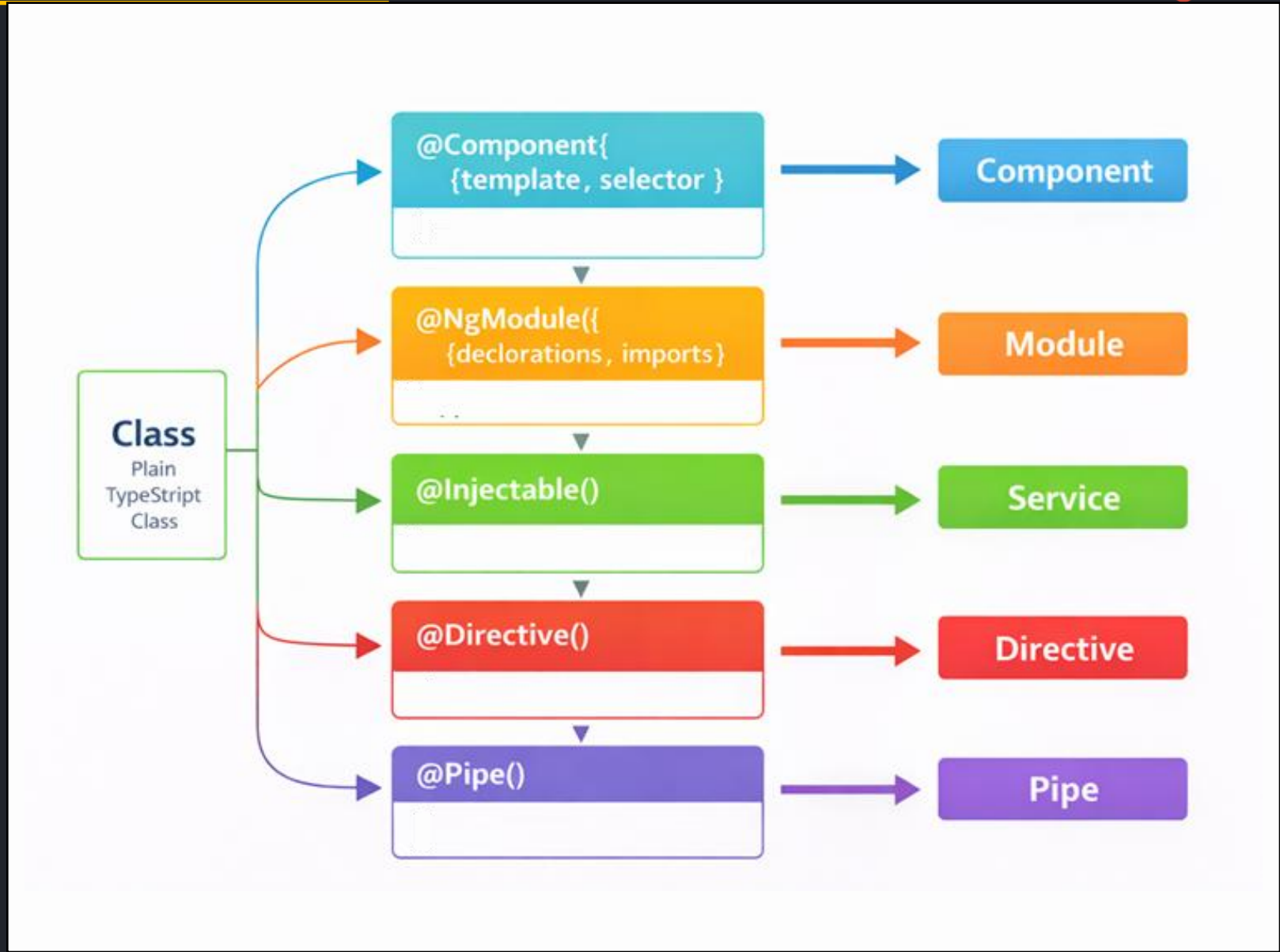
Q. Summarize Angular application loading flow?



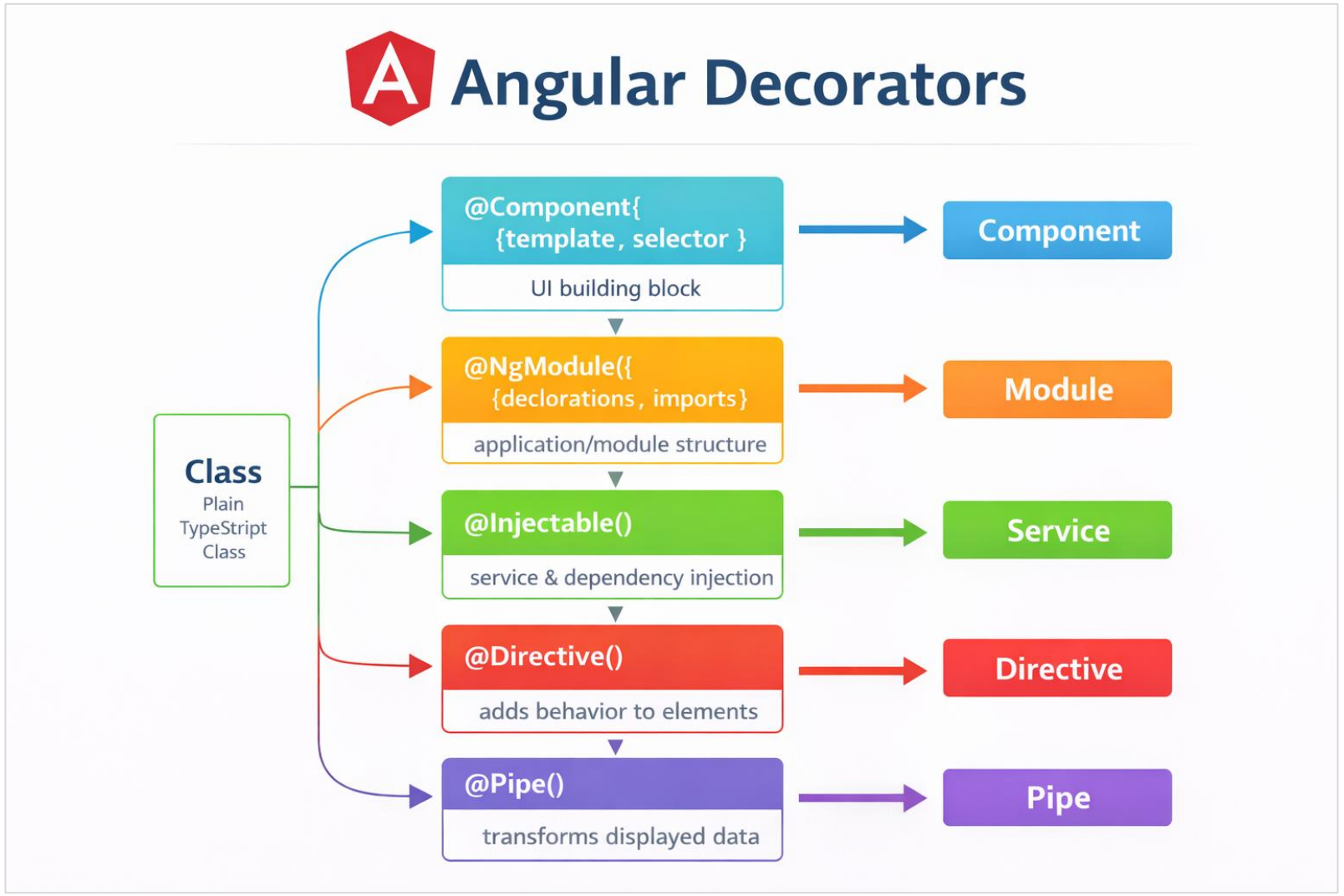


Decorators

- 👉 Decorators Introduction
- 👉 Class Decorators



Q. What are Decorators?



Q. What are Decorators? Why are they used in Angular applications?



- Decorators help Angular understand **what kind of class it is** - such as a component, service, pipe, or module.
- Decorator is a special function that **adds extra information** (called metadata) to a class.

```ts
//app.ts
import { Component, signal } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet],
  templateUrl: './app.html',
  styleUrl: './app.css’
})

export class App {
}
```

Q. What are Class Decorators? Explain @Component decorators?



- **Class decorators** are special functions that are applied on a TypeScript class to tell Angular **what that class represents**.

- **@Component** is a type of class decorator used to **define a Component**.

```
@Component({

  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet],
  templateUrl: './app.html',
  styleUrl: './app.css'

})

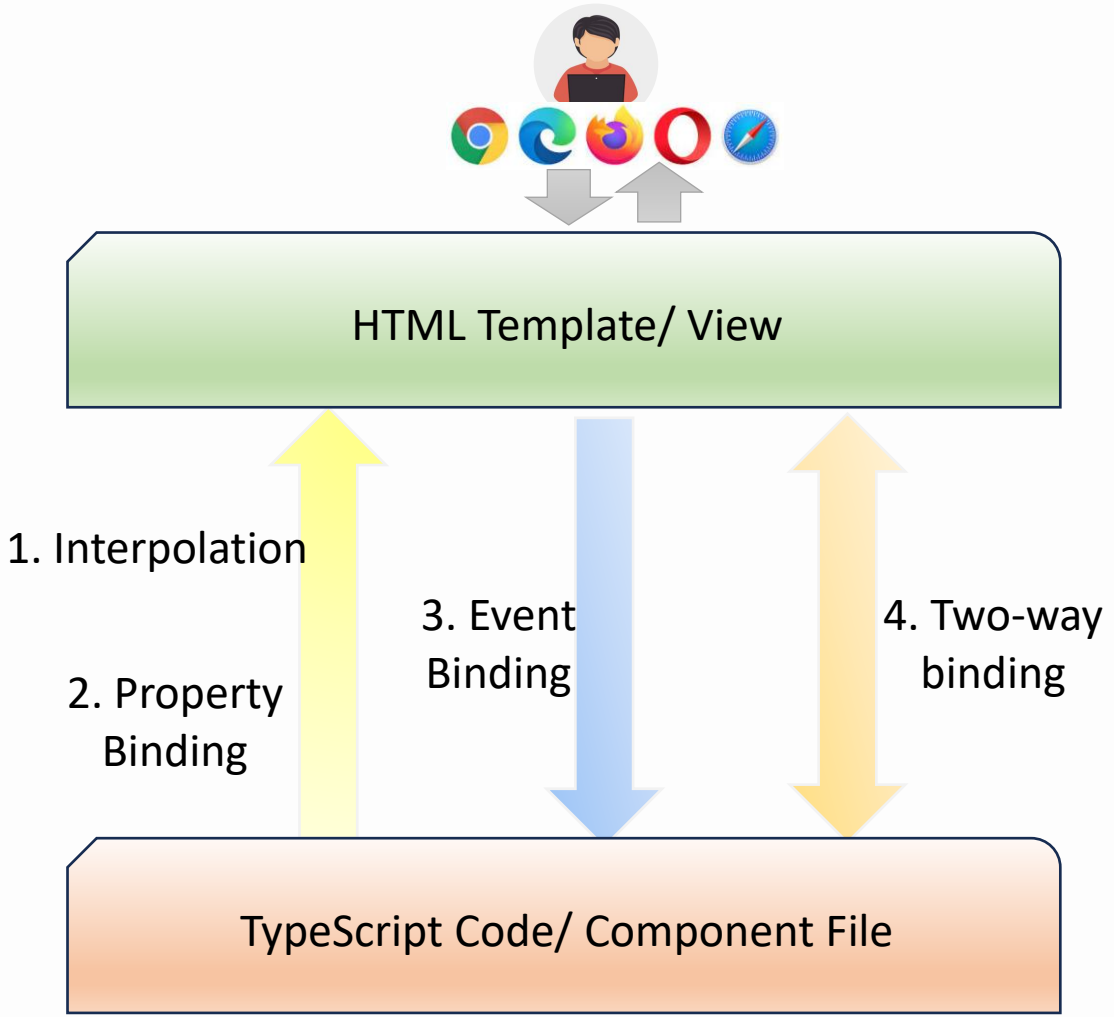
export class App {

 selectedTechnology = 'Angular';

}
```

Data Binding

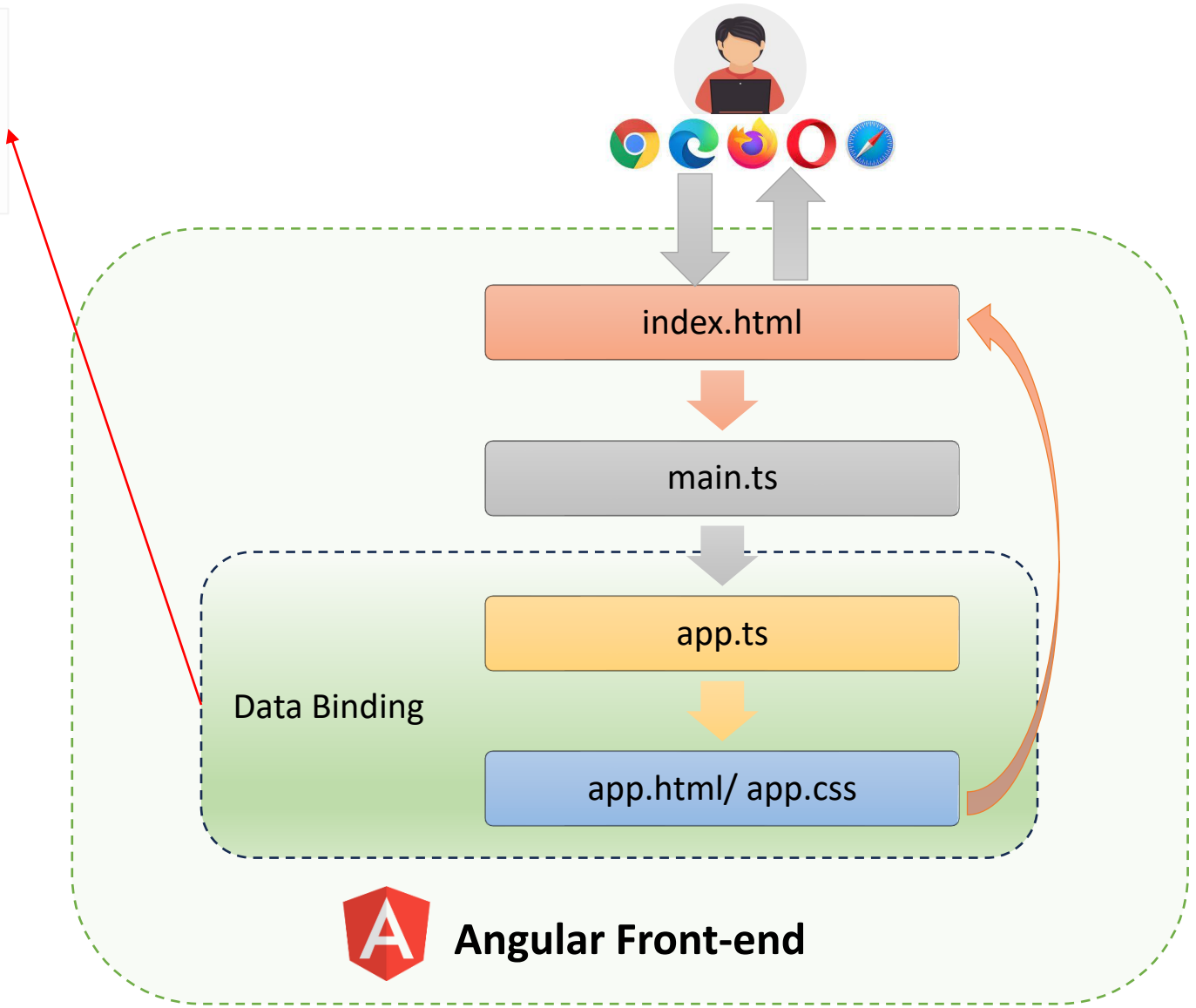
- 👉 Data Binding Introduction
- 👉 Interpolation
- 👉 Property Binding
- 👉 Event Binding
- 👉 Two-way Binding



Q. What is Data Binding in Angular?

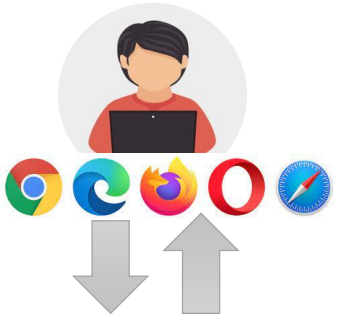


- Data binding is the way to communicate between your typescript code of your component and your html view.



Q. What are the Types of Data Binding?





HTML Template/ View
(app.html/ any component html)

1. Interpolation
{{ data }}

2. Property Binding
[property] = “data”

Output Data

3. Event Binding
(event) = “expression”

Input Data

4. Two-way binding
[(“ ngModel ”)] = “data”

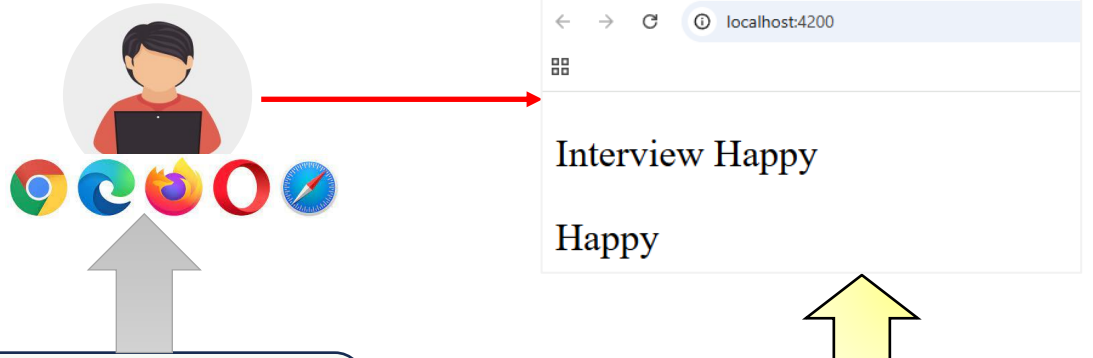
Both ways

TypeScript Code/ Component File
(app.ts/ any component)

Q. What is Interpolation? How to implement it?



- Interpolation is a one-way data binding technique used to transfer **data from a component (TypeScript) to an HTML template (view)**.



HTML Template/ View

```
// interpolation.html
<p>{{ appName }}</p>

<p>{{ user.firstName }} </p>
```

Interpolation
{{ data }}

Output Data

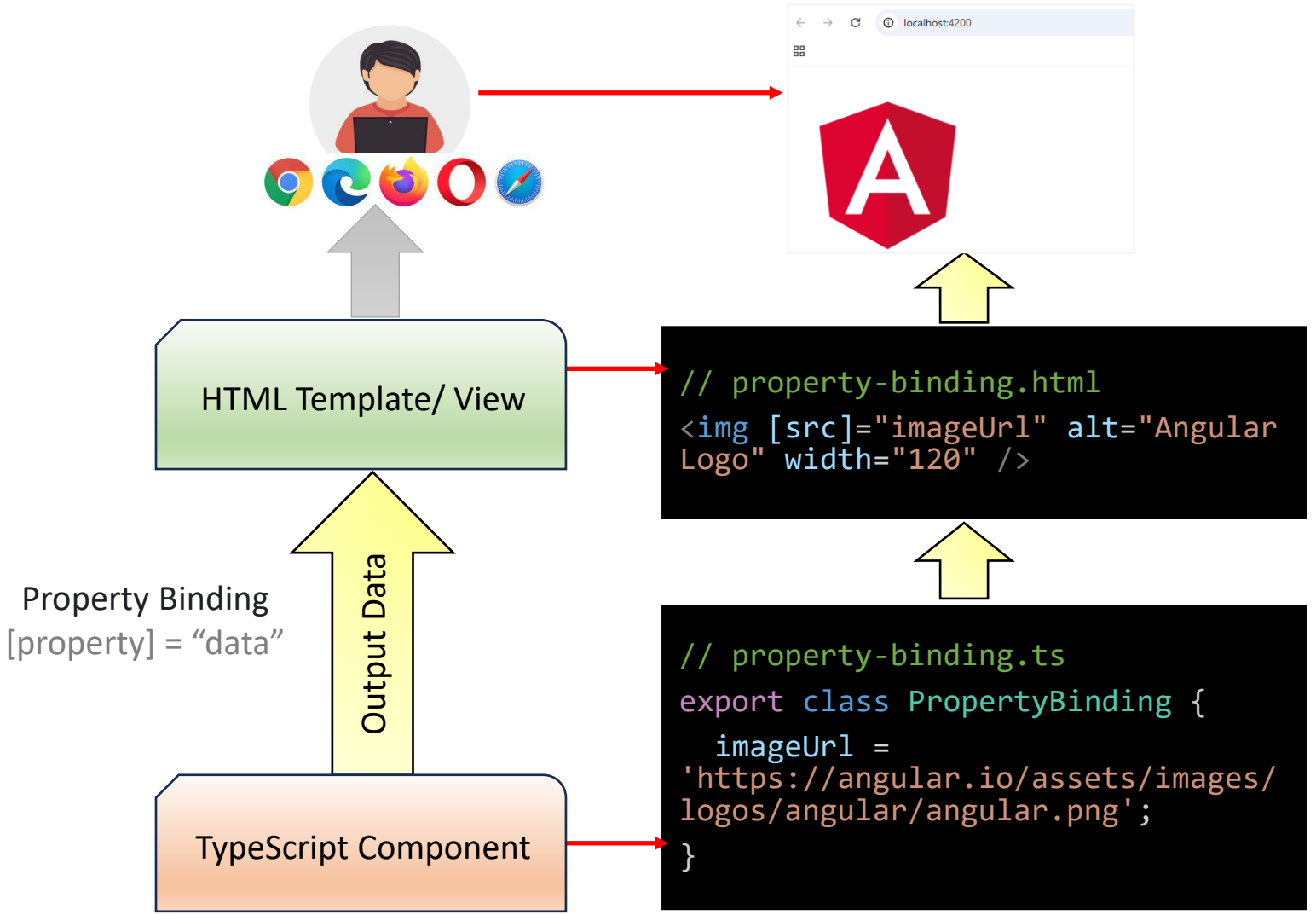
TypeScript Component

```
// interpolation.ts
export class Interpolation {
  appName = 'Interview Happy';
  user = { firstName: 'Happy'};
}
```

Q. What is Property Binding? How to implement it?



- Property Binding in Angular is a one-way data binding from component (TypeScript) to **HTML element property, using square brackets [ ]**.

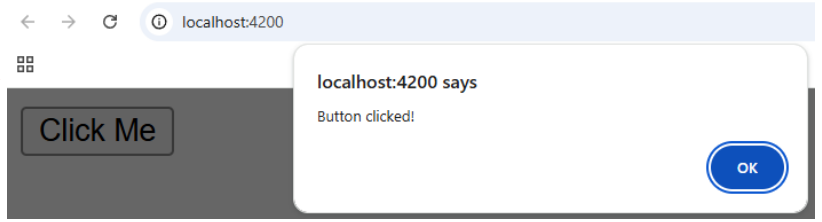


Q. What is Event Binding? How to implement it?



- Event Binding in Angular is a one-way binding from the **HTML view to the component (TypeScript)**.
- It is implemented using round brackets ( ) to listen to events like click, input, keyup, and change.





HTML Template/ View

```
// event-binding.html
<button (click)="showMessage()">Click Me</button>
```

Event Binding
(event) = “expression”

Input Data

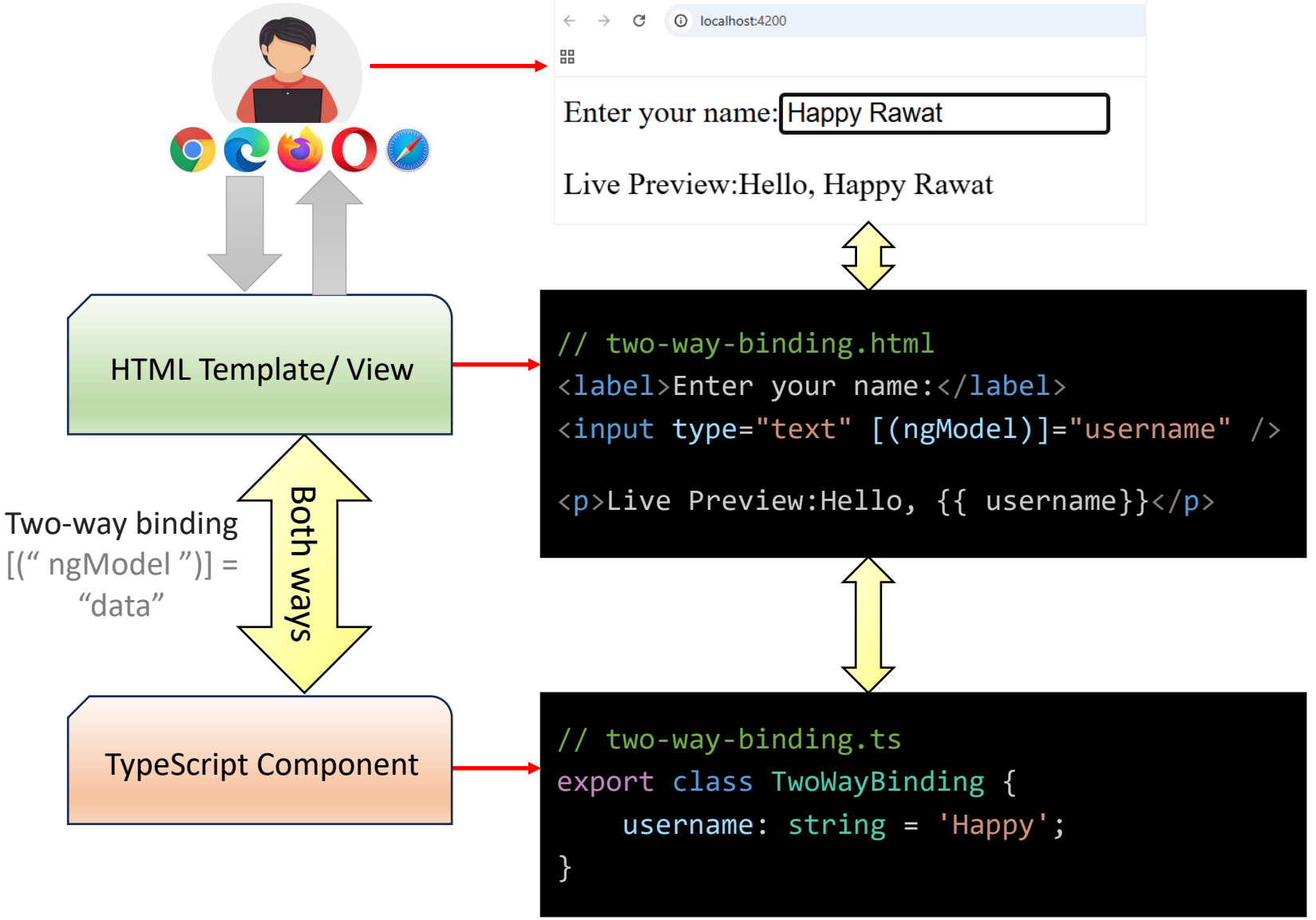
TypeScript Component

```
// event-binding.ts
export class EventBinding {
     showMessage() {
    alert('Button clicked!');
  }
}
```

Q. What is Two-way Data Binding? How to implement it? V. IMP.

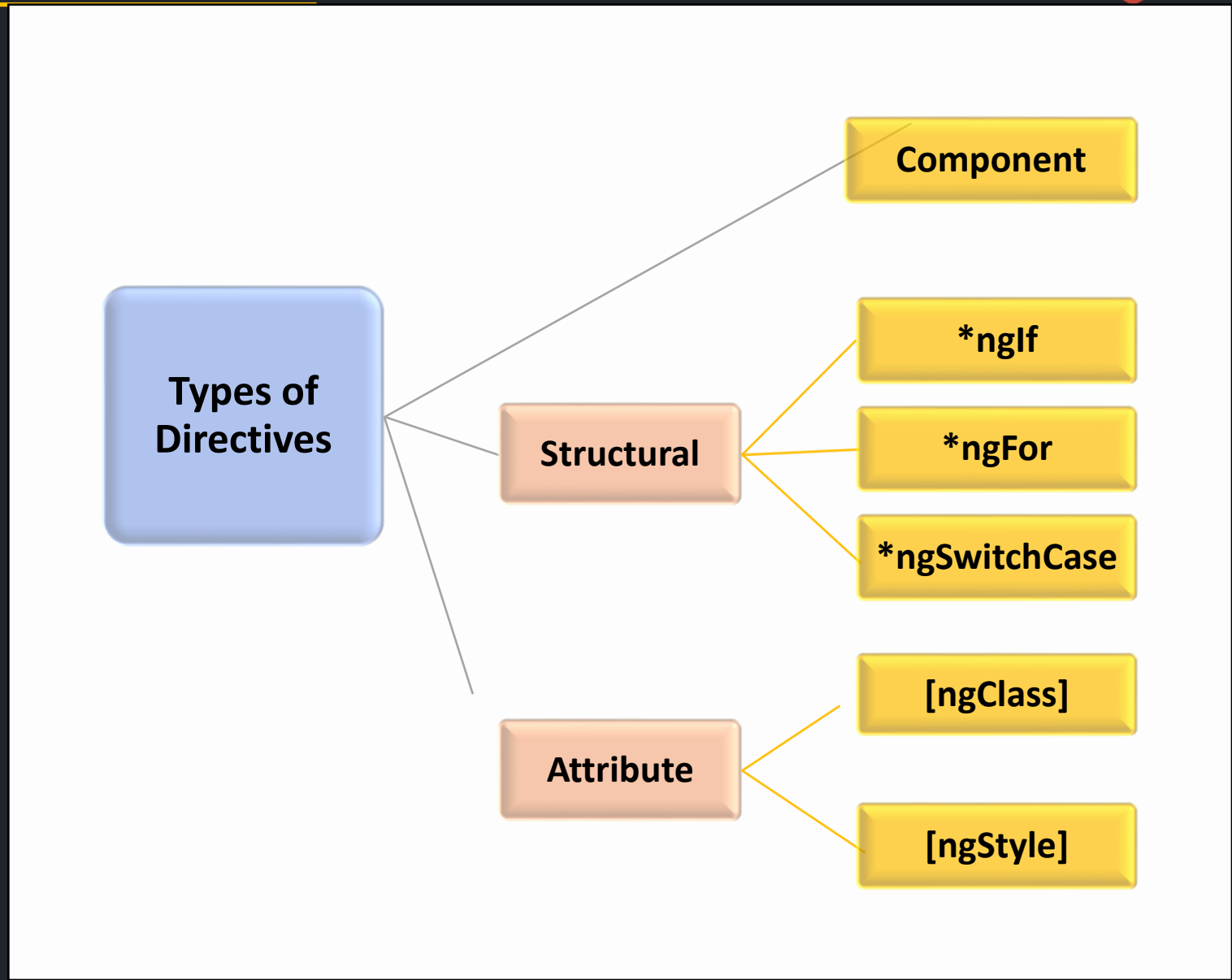


- Two-way data binding in Angular will help users to exchange data from the view to component and then from component to the view at the **same time**(sync automatically).
- Two way binding = Event binding + property binding



Directives

- 👉 Directives Introduction
- 👉 Structural Directives
- 👉 *ngIf
- 👉 *ngFor



Q. What are Directives?



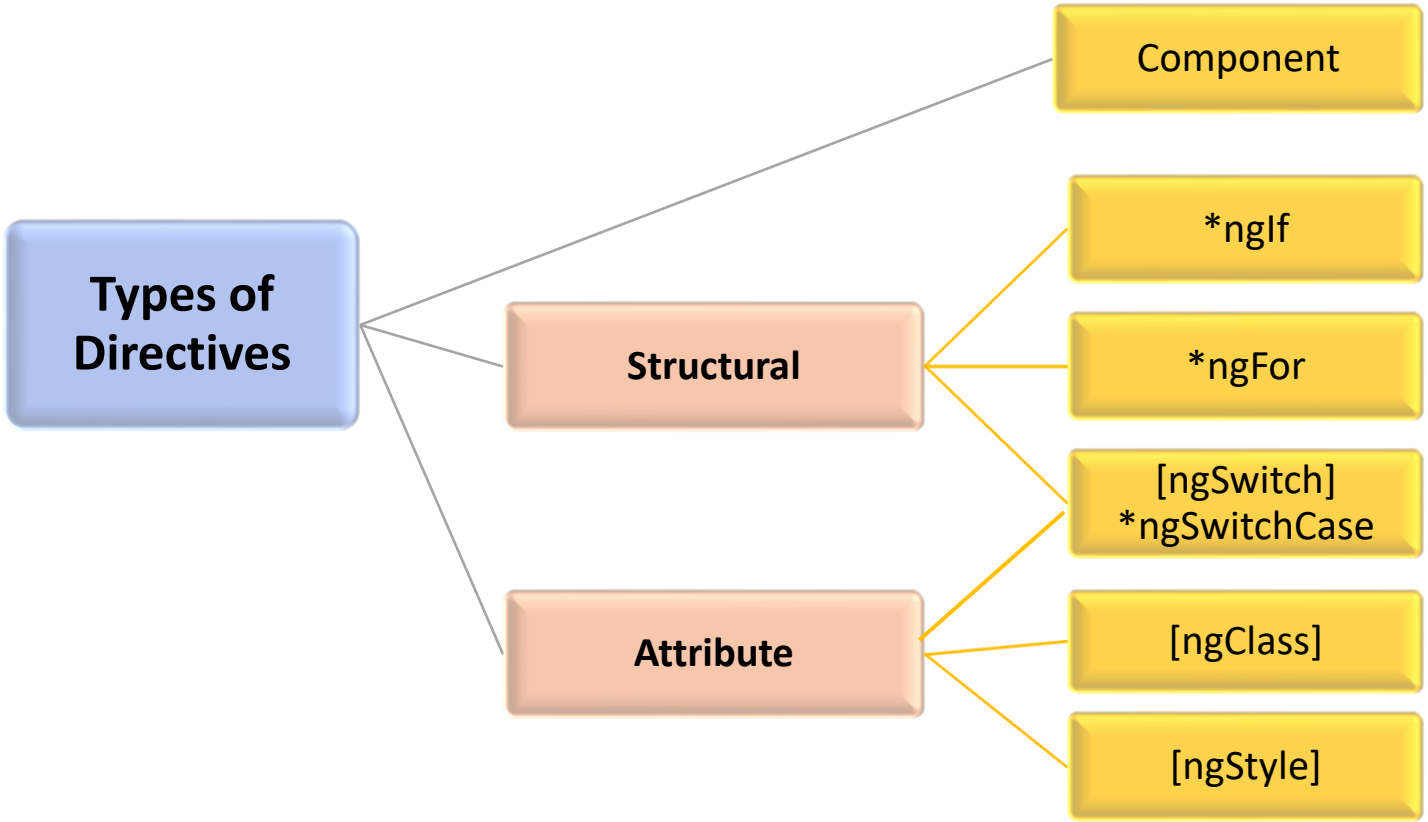
- Directives are special Angular classes used to modify the **behavior, appearance, or structure of HTML** elements in the DOM.



Q. What are the Types of Directives? What is Structural directive?



- A structural directive is used to change the structure of the DOM **by adding or removing elements**.



Q. What is *ngIf directive? How to implement it?

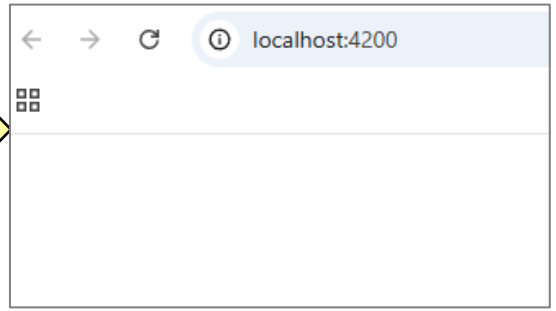


- *ngIf is an Angular Structural Directive used to **add or remove an element** from the DOM based on a boolean condition.

```
// ng-if.html – Add the element
<p *ngIf="true">Interview Happy</p>
```




```
// ng-if.html – Remove the element
<p *ngIf="false">Interview Happy</p>
```



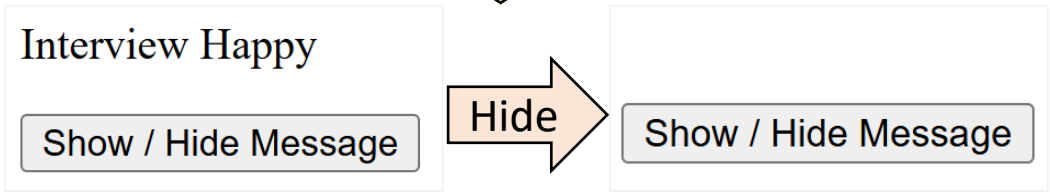
Q. How to use *ngIf directive in real applications?



```
// ng-if.html
<p *ngIf="showMessage">Interview Happy</p>

<!-- Button to toggle -->
<button (click)="toggleMessage()">Show / Hide
Message</button>
```

```
// ng-if.ts
export class NgIf {
    showMessage = true;
    toggleMessage() {
      this.showMessage = !this.showMessage;
    }
}
```



Q. What is the @if block in Angular?

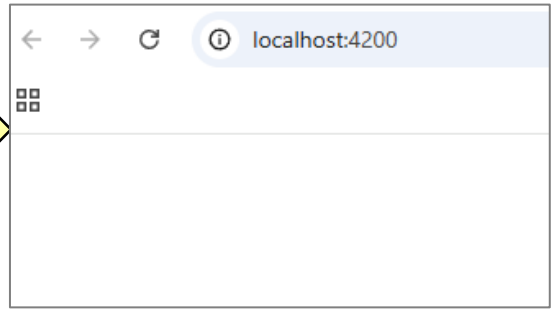


- @if in Angular is the built-in control flow syntax used to **conditionally render HTML** based on a condition.
- @if is the **modern alternative to *ngIf** and supports @else and @else if as well.

```
<!-- <p *ngIf="true">Interview
Happy</p> -->
@if (true) {
  <p>Interview Happy</p>
}
```




```
<!-- <p *ngIf="true">Interview
Happy</p> -->
@if (false) {
  <p>Interview Happy</p>
}
```



Q. What is *ngFor directive? How to implement it?



- *ngFor is an Angular Structural Directive used to **loop through** a list or array and create one HTML element for each item in Array.

```
// ng-for.html
<ul>
<li *ngFor="let language of languages">{{language}}</li>
</ul>
```

```
// ng-for.ts
export class NgFor {

  languages = ['Java', '.NET', 'Angular', 'React', 'Node'];

}
```



Q. What is the @for block in Angular?



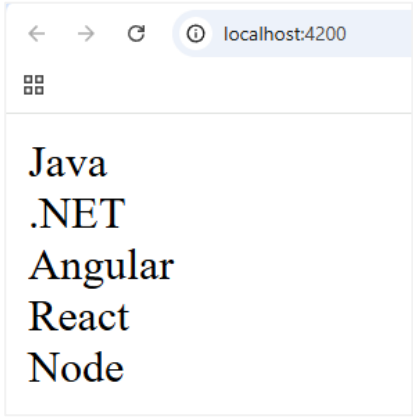
- The @for block in Angular is the built-in control flow syntax used to **iterate over a collection** and render HTML for each item.
- @for is the modern alternative to *ngFor.
- track in the @for block specifies how Angular should uniquely identify each item in the collection, which Angular then uses to update only the necessary DOM elements instead of recreating the entire list.

```html
// .html
<ul>
  <!-- <li *ngFor="let language of languages">{{language}}</li> -->
  @for (language of languages; track language) {
    <li>{{ language }}</li>
  }
</ul>
```

```ts
// .ts
export class App {

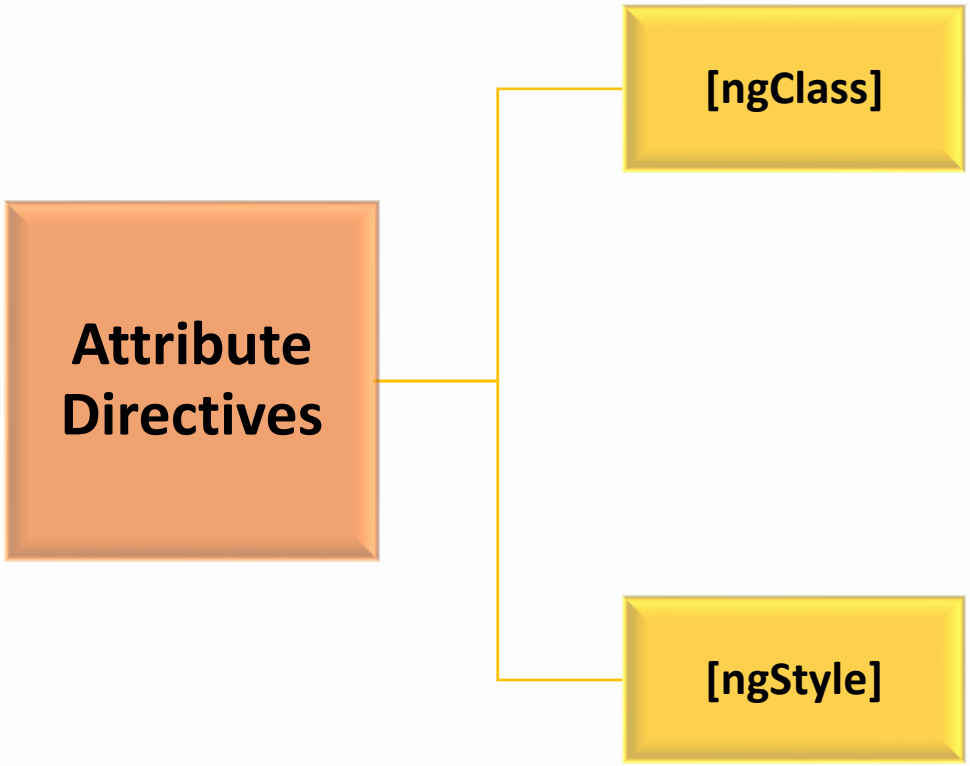
  languages = ['Java', '.NET', 'Angular', 'React', 'Node'];

}
```



Attribute Directives

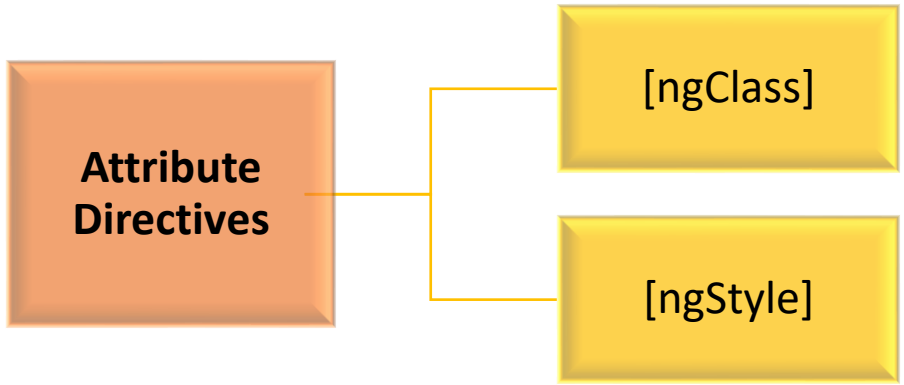
- 👉 Attribute Directives
- 👉 ngClass
- 👉 ngStyle
- 👉 ngSwitch
- 👉 Component Directive



Q. What is Attribute directive? What are the types of Attribute directive?



- Attribute directives modify how an **HTML element looks** or behaves, but they don't add or remove elements.



Q. What is [ngClass] directive? How to implement it.

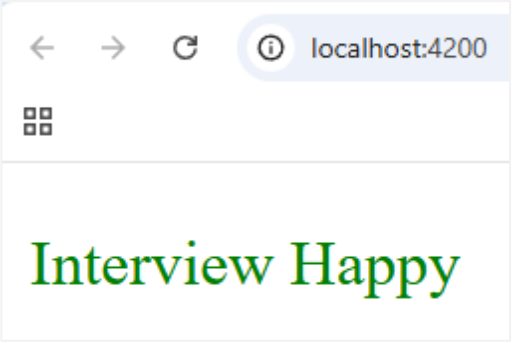


- [ngClass] is an Angular attribute directive that allows you to **add or remove CSS classes dynamically** on an HTML element based on component data.
- [ngClass] = Property Binding[] + class attribute

```
// .html
<p [ngClass]="isSuccess">Interview Happy</p>

<style>
  .success {
    color: green;
  }
  .error {
    color: red;
  }
</style>
```

```
// .ts
export class MyComponent {
  isSuccess = 'success';
}
```



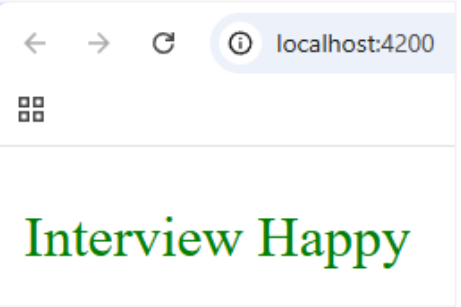
Q. What is [ngStyle] directive? How to implement it.



- [ngStyle] is an Angular attribute directive that allows you to apply **inline CSS styles** dynamically on an HTML element based on component data.

```html
// .html
<p [ngStyle]="{ color: isSuccess
=== 'success' ? 'green' : 'red'
}">Interview Happy</p>
```

```ts
// .ts
export class MyComponent {
  isSuccess = 'success';
}
```



Q. What is ngSwitch directive? How to implement it.



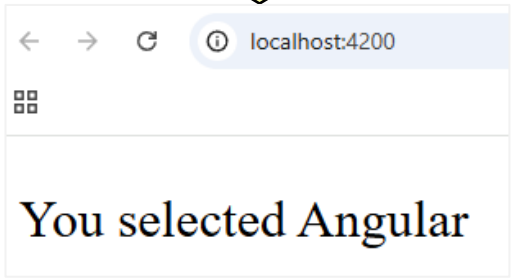
- ngSwitch is an Angular Structural Directive used to display **one element among many possible elements**, based on a matching expression.

```
// ng-switch.html
<div [ngSwitch]="selectedTechnology">
  <p *ngSwitchCase="'Java'">You selected Java</p>
  <p *ngSwitchCase="'.NET'">You selected .NET</p>
  <p *ngSwitchCase="'Angular'">You selected Angular</p>
  <p *ngSwitchDefault>Unknown Technology</p>
</div>
```

```
// ng-witch.ts
export class NgSwitch {

  selectedTechnology = ‘Angular’;

}
```



Q. What is the @switch block in Angular?



- @switch block in Angular is the built-in control flow syntax used to **render different HTML based on the value of an expression.**
- It works with:
  - @case to match specific values
  - @default when no case matches

```
// .html
@switch (selectedTechnology) {
  @case ('Java') {
    <p>You selected Java</p>
  }
  @case ('.NET') {
    <p>You selected .NET</p>
  }
  @case ('Angular') {
    <p>You selected Angular</p>
  }
  @default {
    <p>Unknown Technology</p>
  }
}
```

```
// .ts
export class NgSwitch {

  selectedTechnology = ‘Angular’;

}
```

Q. What is Component Directive?



- A Component Directive is a directive in Angular that has its own HTML template and optional CSS to build UI.
- Every component is a component directive.

```
// component-directive.ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-component-ex',
  standalone: true,
  imports: [],
  templateUrl: './component-ex.html',
  styleUrl: './component-ex.css'
})
export class ComponentEx {
  title = 'Interview Happy';
  subtitle = 'Component Directive Example’;
}
```

Angular Pipes

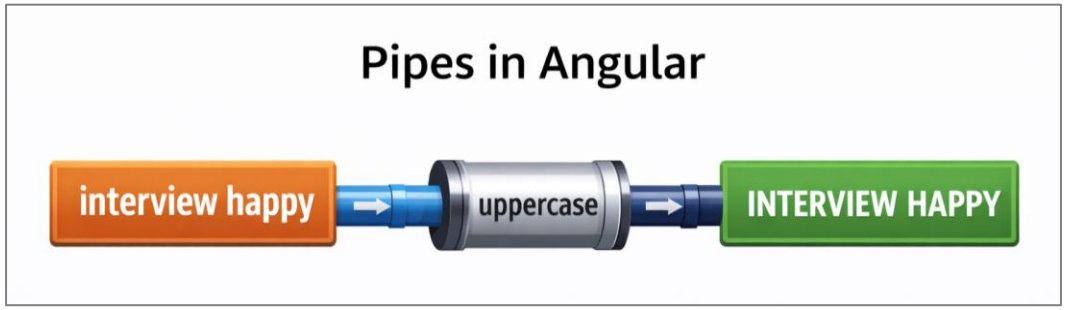
- 👉 Pipes Introduction
- 👉 Text Pipes
- 👉 Number Pipes
- 👉 Slice & JSON Pipe
- 👉 Parameterized Pipes

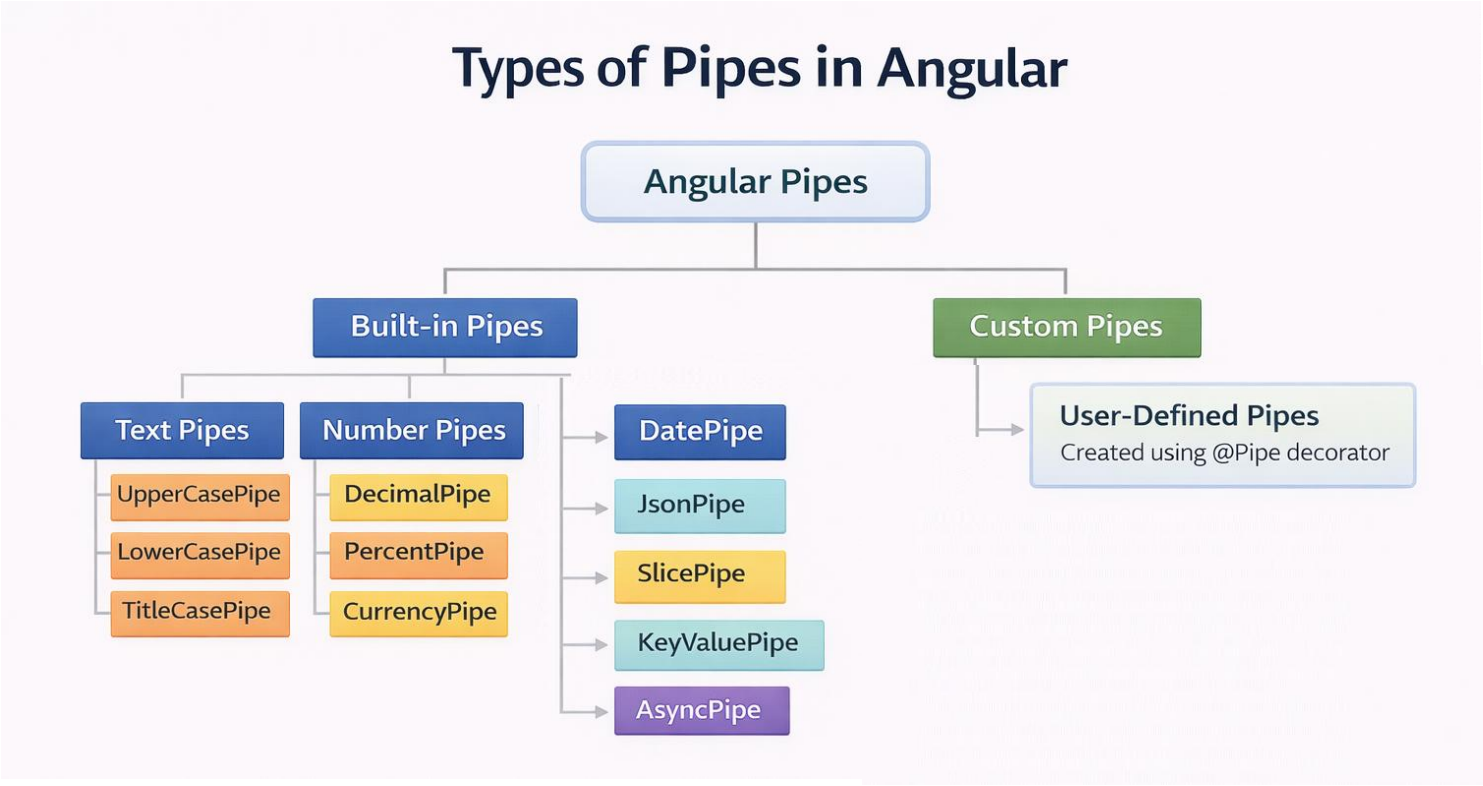


Q. What are Pipes? What are the types of Pipes?



- A Pipe is a feature in Angular that is used to **transform or format data** in the template without changing the original value.





Q. What are Text Pipes - UpperCase, LowerCase and TitleCase?



- Pipes are implemented using the **| (pipe)** operator in Angular templates.

```
//pipes.html

<h3>Pipes Example</h3>

<p>Original: {{ text }}</p>
<p>{{ text | uppercase }}</p>
<p>{{ text | lowercase }}</p>
<p>{{ text | titlecase }}</p>
```



Q. What are Number Pipes - Decimal and Currency Pipe?

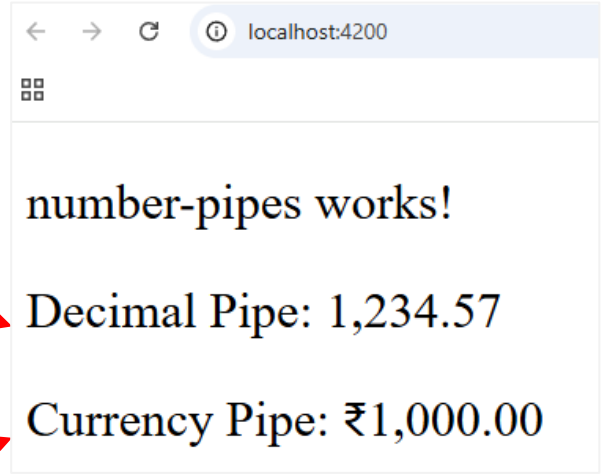


```
//number-pipes.html
<p>number-pipes works!</p>

<p>Decimal Pipe: {{ numberValue | number: '1.2-2' }}</p>
// 1.2-2 explained below

// 1 → Minimum digits before decimal
// 2 → Minimum digits after decimal
// 2 → Maximum digits after decimal


<p>Currency Pipe: {{ currencyValue | currency: 'INR' }}</p>
```




```
//number-pipes.ts
export class NumberPipes {
  numberValue = 1234.567;
  currencyValue = 1000;
}
```

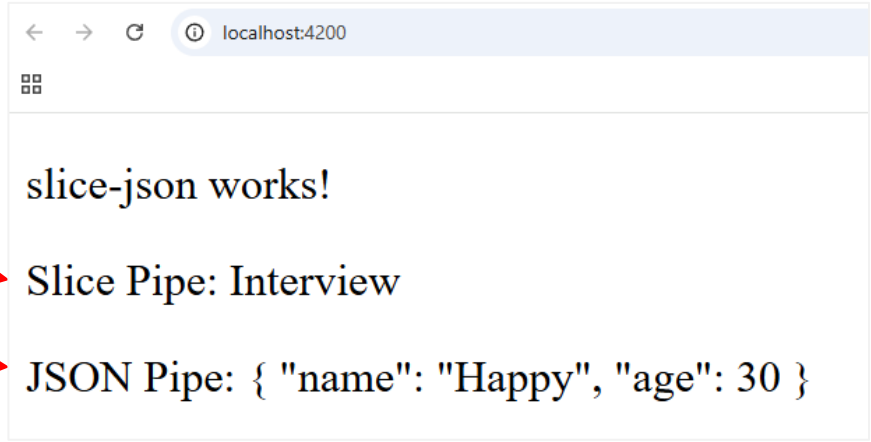
Q. What are Slice and JSON Pipe?



```
<p>slice-json works!</p>

<p>Slice Pipe: {{ text | slice: 0 : 9 }}</p>

<p>JSON Pipe: {{ user | json }}</p>
```




```
//number-pipes.ts

export class SliceJson {
  text = 'InterviewHappy';
  user = { name: 'Happy', age: 30 };
}
```

Q. What are Parameterized Pipes?



- A parameterized pipe is a pipe that takes **additional arguments (parameters)** to customize how the data is transformed.

```
//parameterized-pipes

<p>Decimal Pipe:{{ 1234.567 | number:'1.2-2' }}</p>

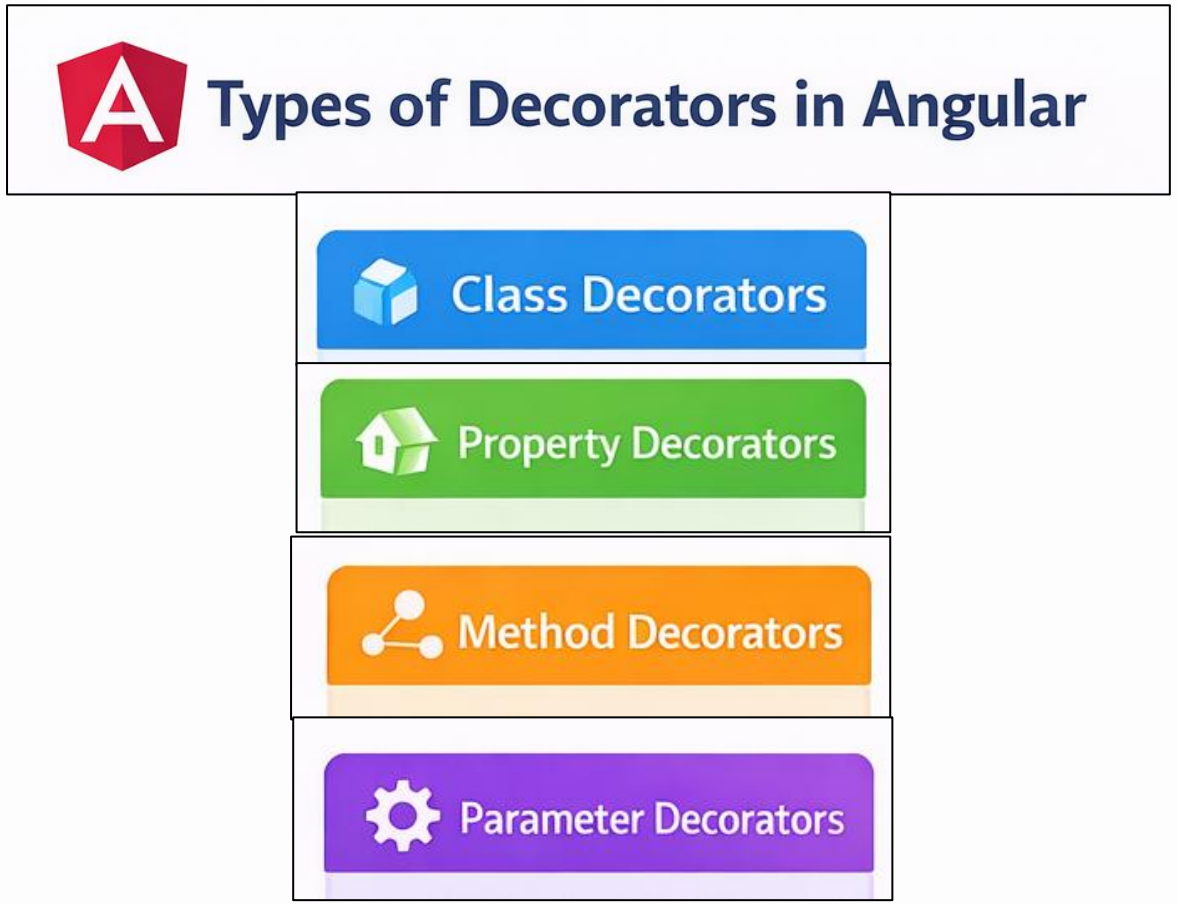
<p>Currency Pipe:{{ 1000 | currency:'INR' }}</p>

<p>Slice Pipe:{{ 'InterviewHappy' | slice:0:9 }}</p>

//non-parameterized-pipes
<p>Uppercase Pipe:{{ 'InterviewHappy’ | uppercase }}</p>
```

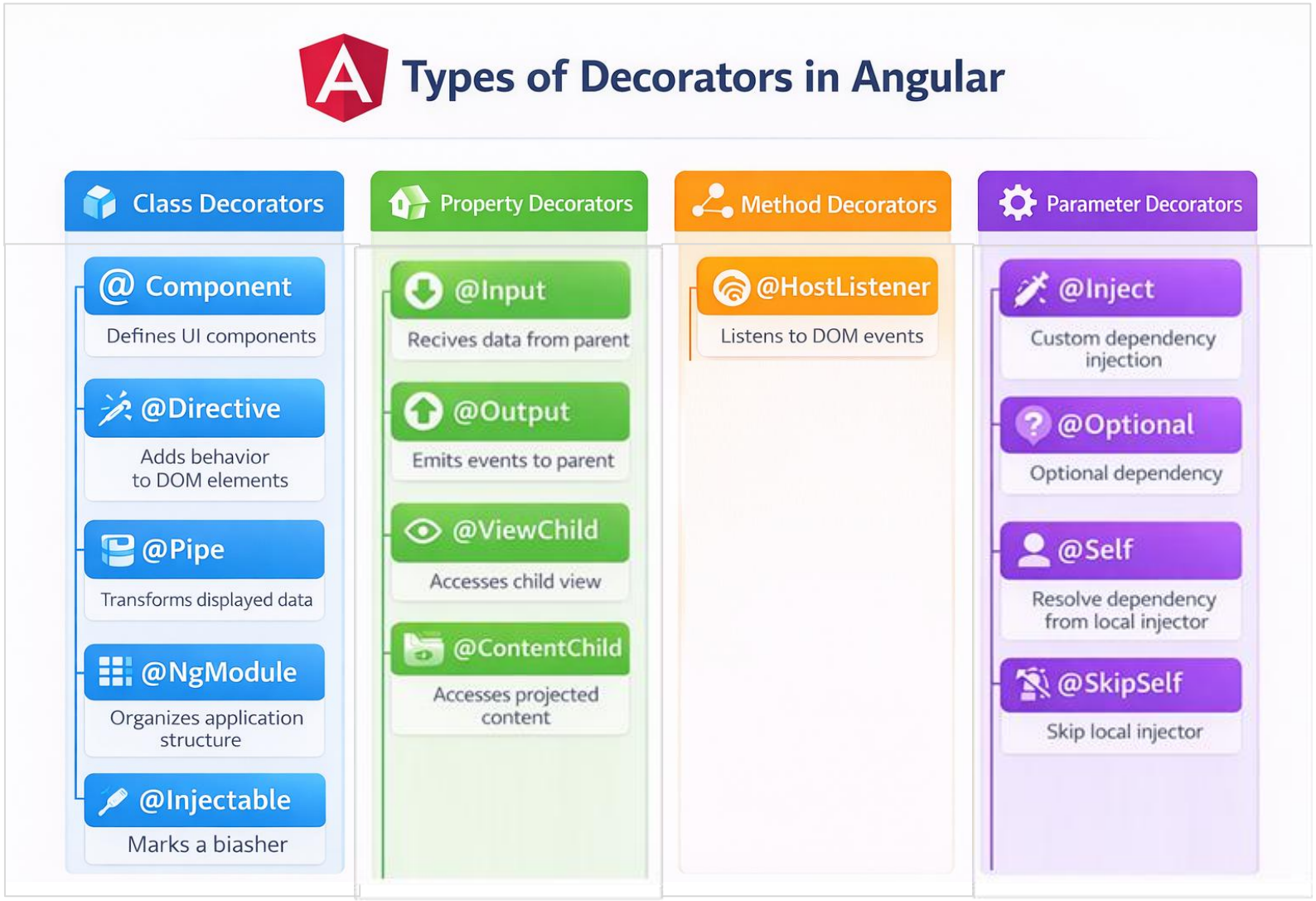
Advance Decorators

- 👉 Types of Decorator
- 👉 Modules
- 👉 Property Decorators



Q. What are the Types of Decorator?





Q. What are Modules? Explain @NgModule Decorator?



- A **module** in Angular is a group of related components or directives or pipes, or services.

- **@NgModule** is a class decorator in Angular that turns a normal TypeScript class into an **Angular Module**.

```
@NgModule({
  declarations: [
    AppComponent,
    HeaderComponent,
    FooterComponent
  ],
  imports: [BrowserModule],
  bootstrap: [AppComponent]
})
export class AppModule {
}
```

Q. What are Property Decorators? What are their types?



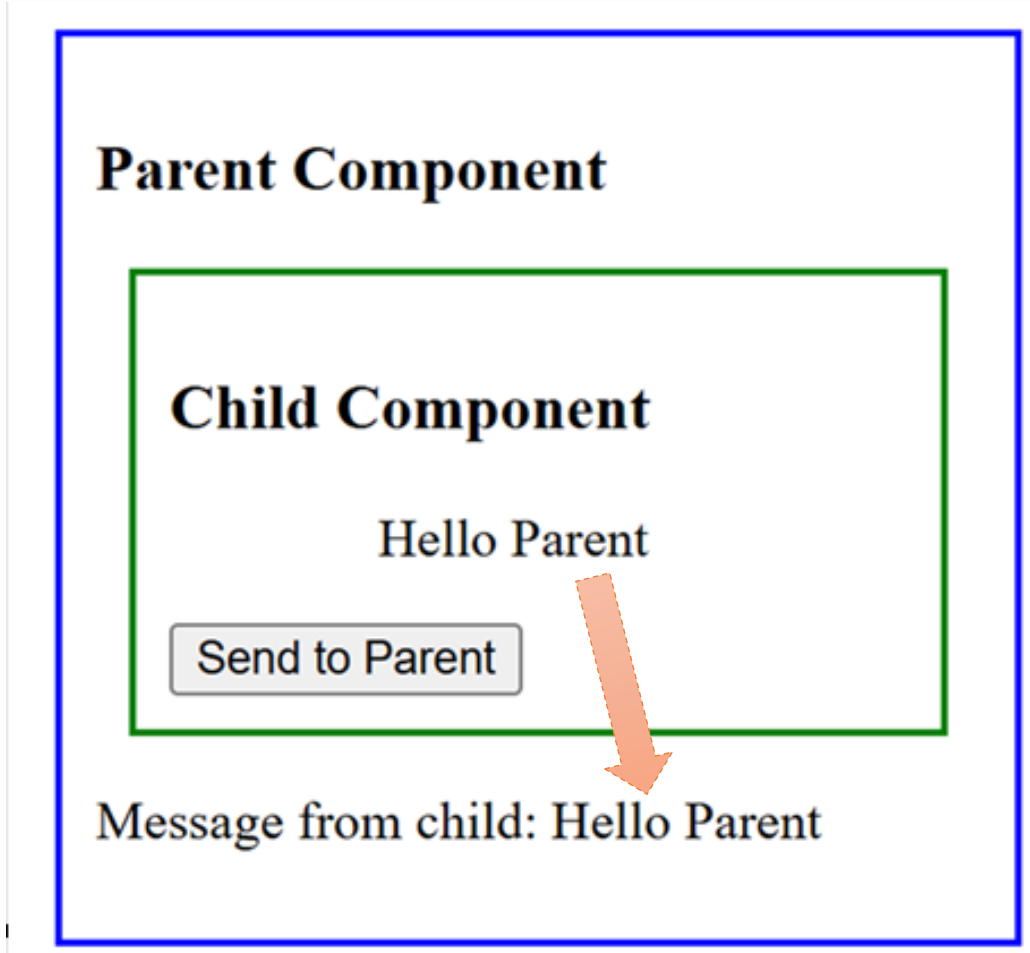
- Property decorators in Angular are decorators used on **class properties to define their behavior** and enable communication or interaction between components.




```
@Component({
  selector: 'app-child',
  imports: [],
  templateUrl: './child.html',
  styleUrl: './child.css',
})
export class Child {
  // Receiving data from parent
  @Input() message!: string;
}
```

Components Communication

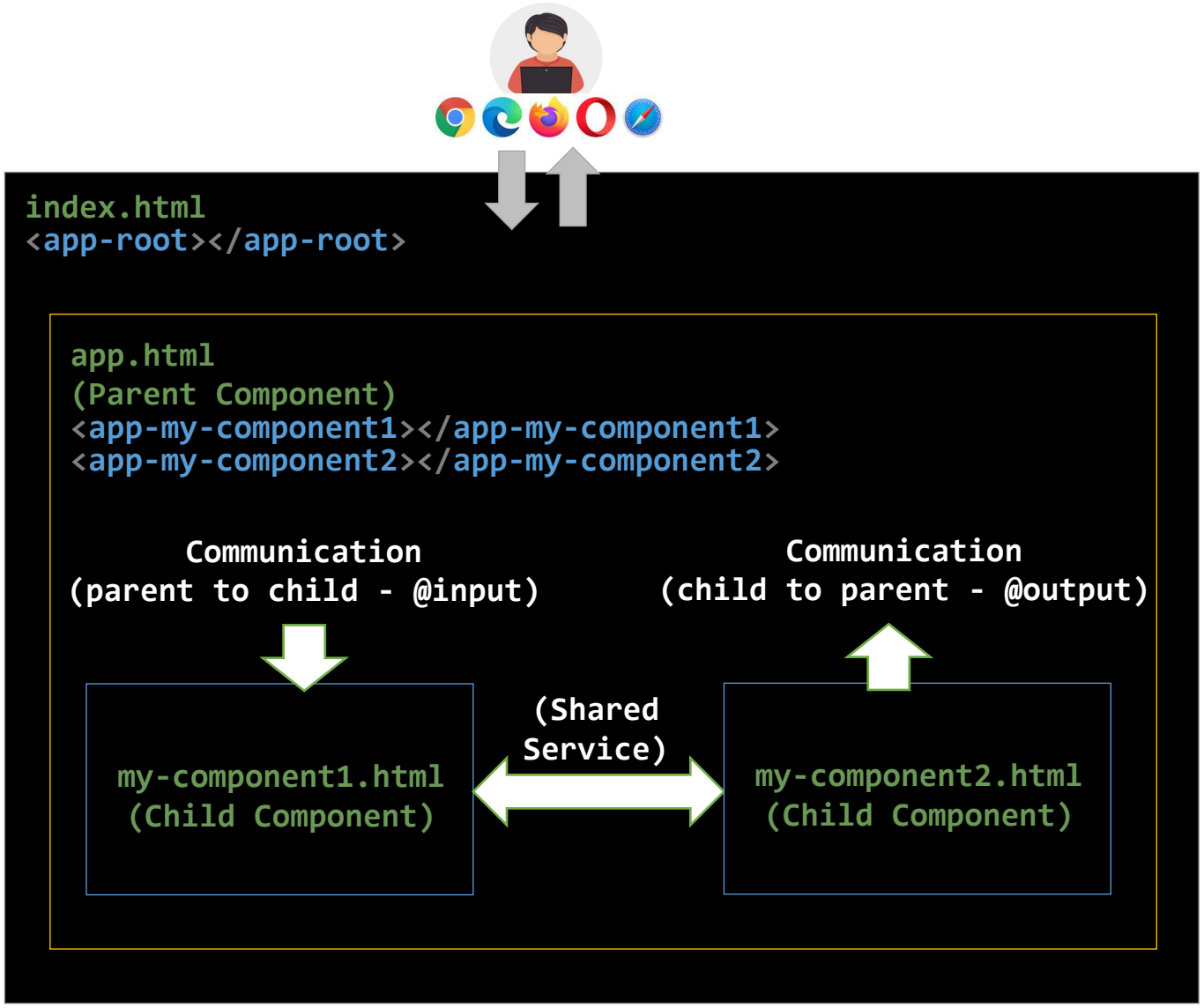
- 👉 Component Hierarchy
- 👉 Input Decorator
- 👉 Output Decorator
- 👉 EventEmitter
- 👉 Child to Parent Communication



Q. Explain Component Hierarchy? How Parent-Child communication works.



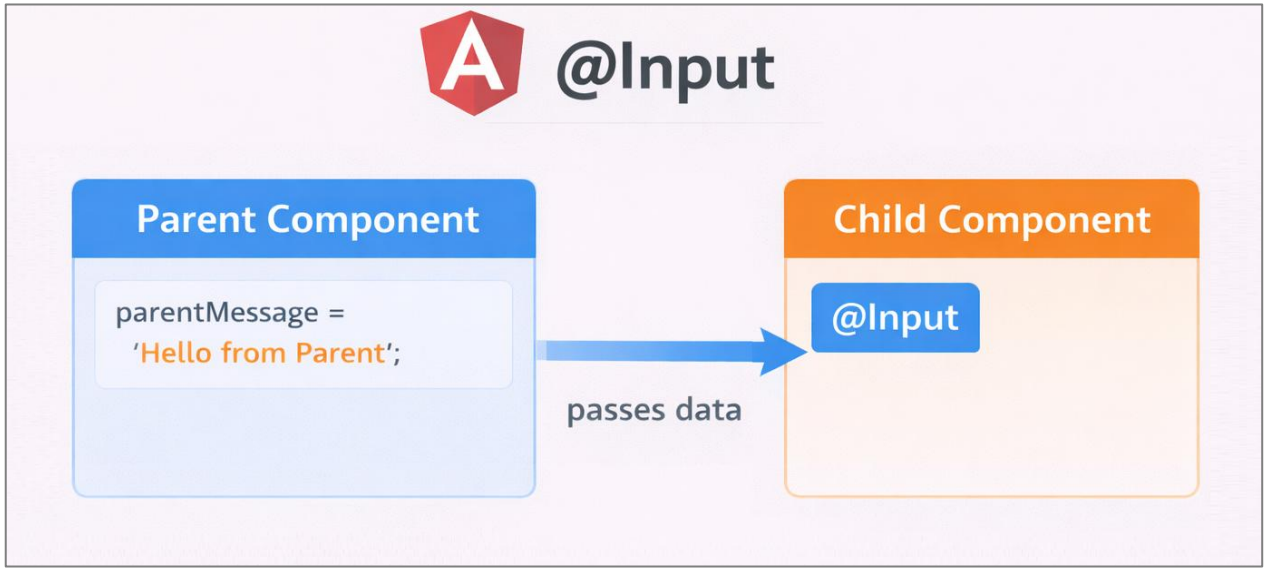
- Parent-Child relationship in Angular means **one component (child) is nested inside another component's template (parent)** by placing the selector tag of the child component in the parent component's template.
- Parent can send data to child (**@Input**).
- Child can send data back to parent (**@Output**).



Q. What is @Input decorator?



- @Input is a property decorator in Angular that allows a child component to **receive data from its parent** component.





Sending data from Parent to Child using @Input

Q. How to implement @Input decorator?

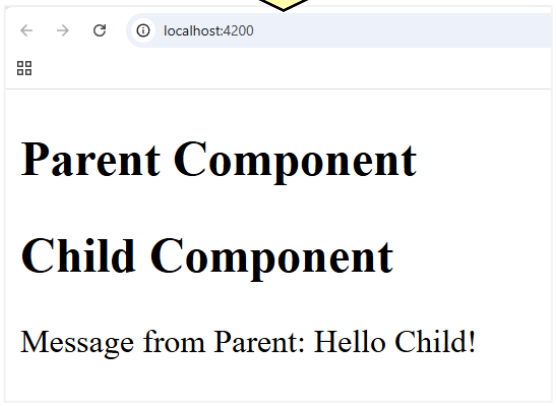


```ts
// parent.ts
export class Parent {
  parentMessage = 'Hello Child!';
}
```

```html
// parent.html
<h2>Parent Component</h2>
<app-child [message]="parentMessage"></app-child>
```

```ts
// child.ts
export class Child {
  // Receiving data from parent
  @Input() message  = '';
}
```

```html
// child.html
<h2>Child Component</h2>
<p>Message from Parent: {{ message }}</p>
```



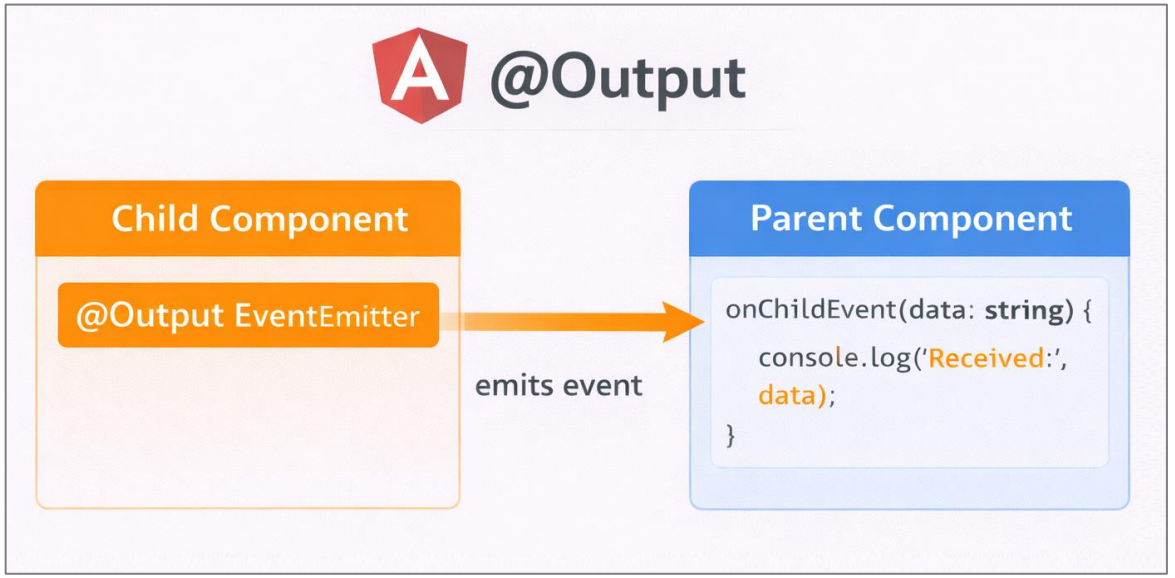
Q. What is @Output decorator?



- @Output is a property decorator in Angular **used in a child component to emit events** so that the parent component can listen to those events and receive data.



Sending Event & Data from Child to Parent using @Output



Q. What is EventEmitter?



- EventEmitter is an inbuilt Angular class **used to emit custom events** from a child component to a parent component.

```
// child.html
<p>child component</p>
<button (click)="sendMessage()">
  Send Message to Parent </button>
```

```
// child.ts
export class ChildOutput {
  notify = new EventEmitter();
  sendMessage() {
    this.notify.emit('Hello Parent');
  }
}
```

Q. How do @Output and EventEmitter send data from a child to a parent?



```
// child.html
<h3>Child Component</h3>
<p>{{ message }}</p>
<button (click)="send()">Send to Parent</button>
```

```
// child.ts
export class ChildOutput {
  @Output() notify = new EventEmitter();
  message = 'Hello Parent';
  send() {
    this.notify.emit(this.message);
 }}
```

```
// parent.html
<p>parent component</p>
<app-my-component
(notify)="receive($event)"></app-my-component>
<p>Message from child: {{ message }}</p>
```

Parent Component

Child Component

Hello Parent

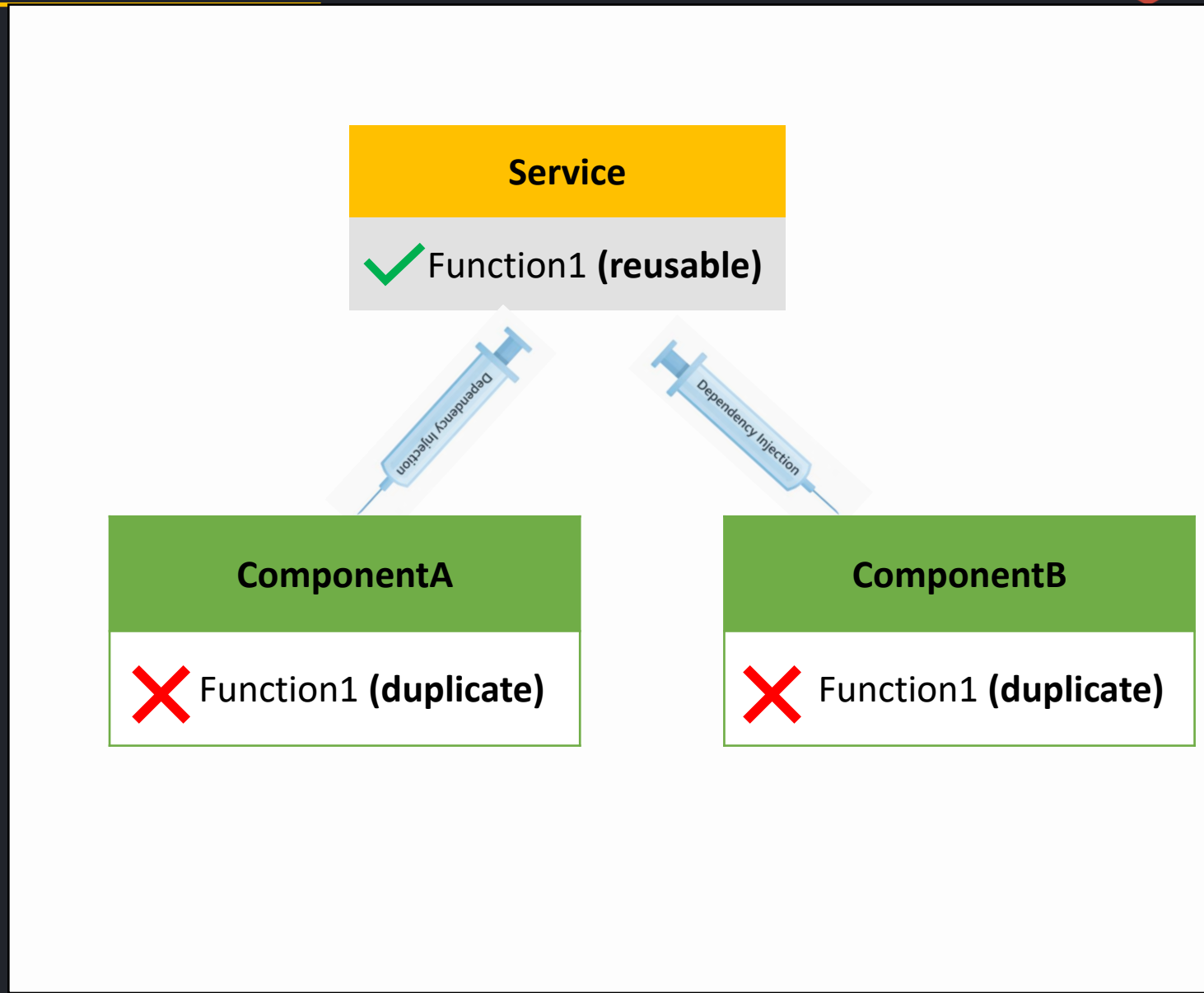
Send to Parent

Message from child: Hello Parent

```
// parent.ts
export class ParentOutput {
  message = '';
  receive(data: string) {
    this.message = data;
  }
}
```

Services & Dependency Injection

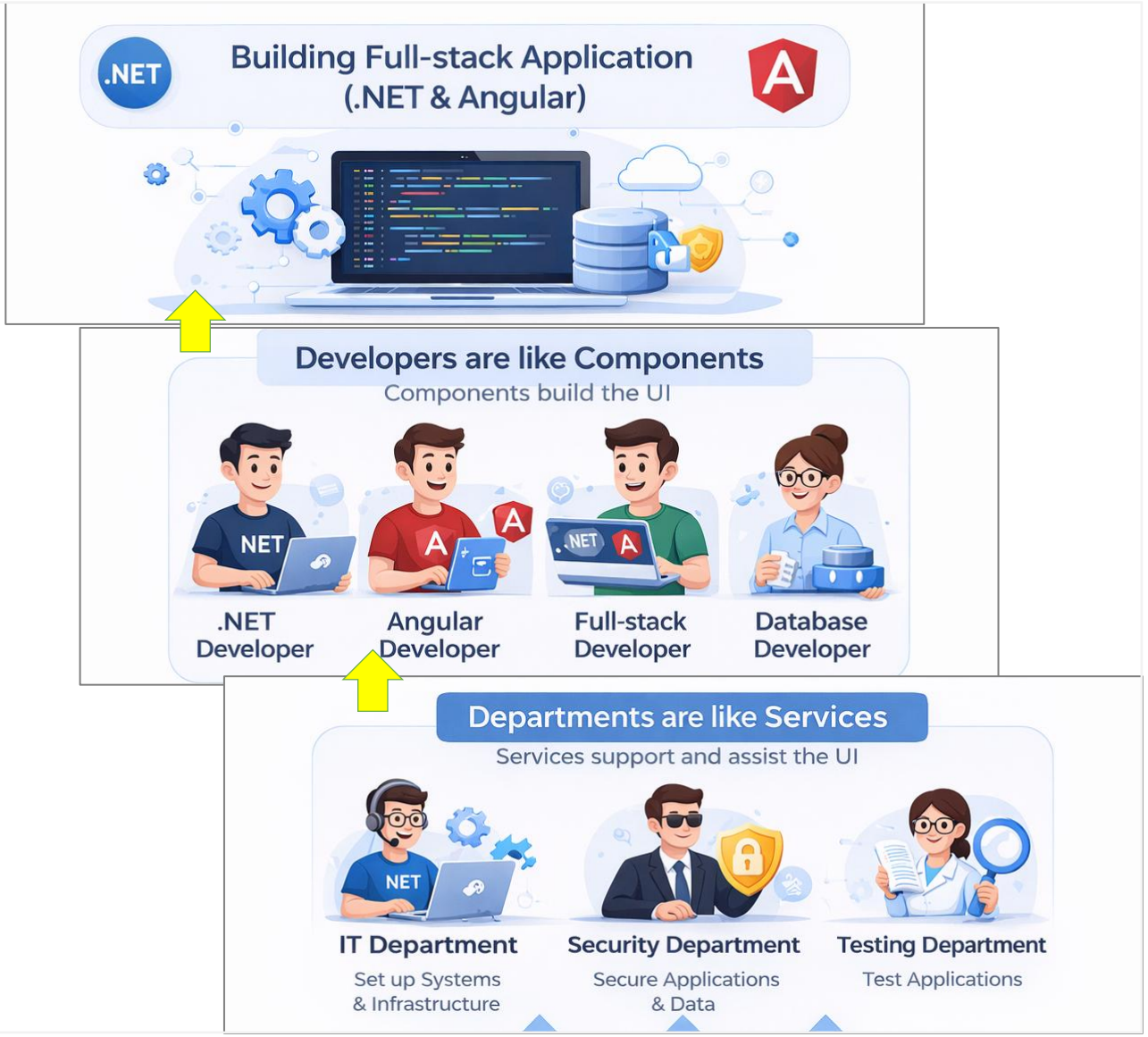
- 👉 Services Introduction
- 👉 Dependency Injection



Q. What are Services in Angular?



Understanding Angular Services with a Real Company Analogy

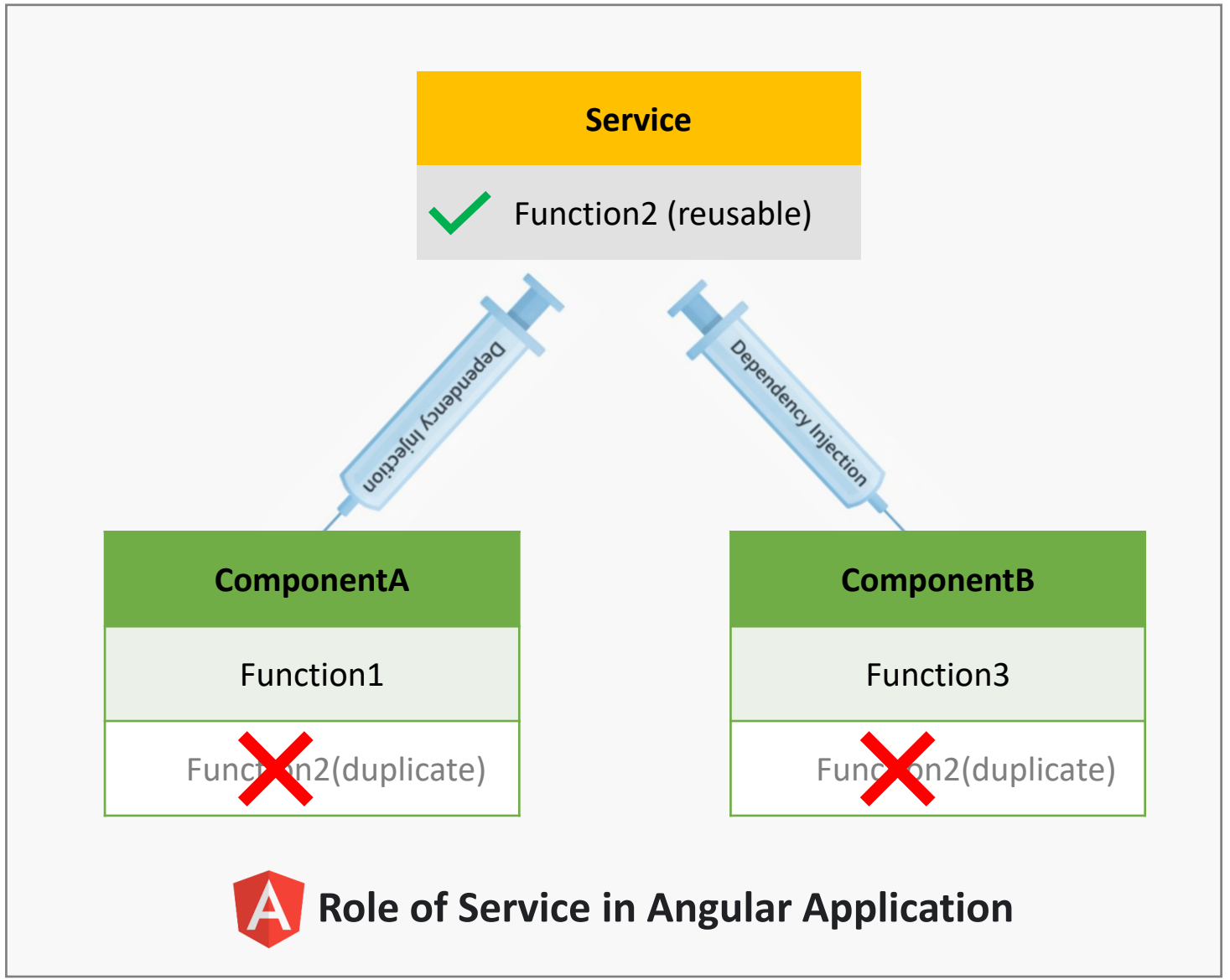


Q. What are Services in Angular? Why do we use them?



- A Service is a TypeScript class that contains **reusable code**(business logic), which can be shared across multiple components using DI.

- Dependency Injection is a mechanism used to **inject services into components**, directives, or other services in Angular.



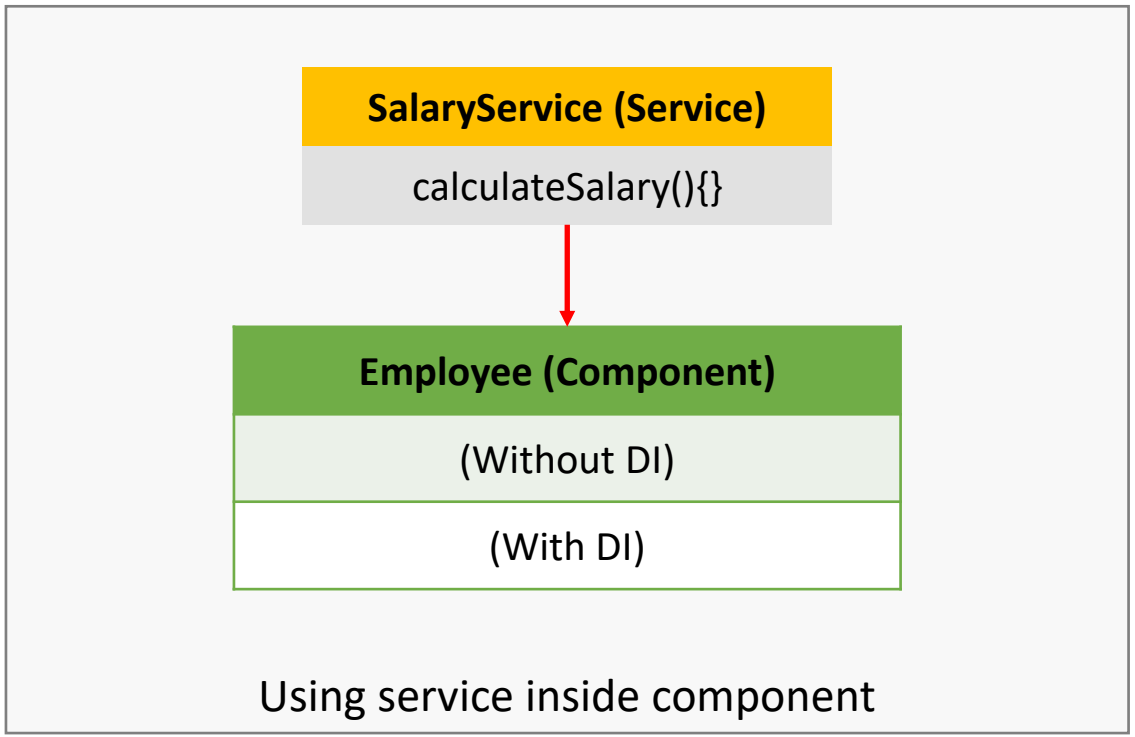
Q. How to Create Service in Angular?



- To create service, use Angular CLI Command:

  ng generate service service-name

  Or ng g s service-name




```
// salary.ts
import { Injectable } from '@angular/core';

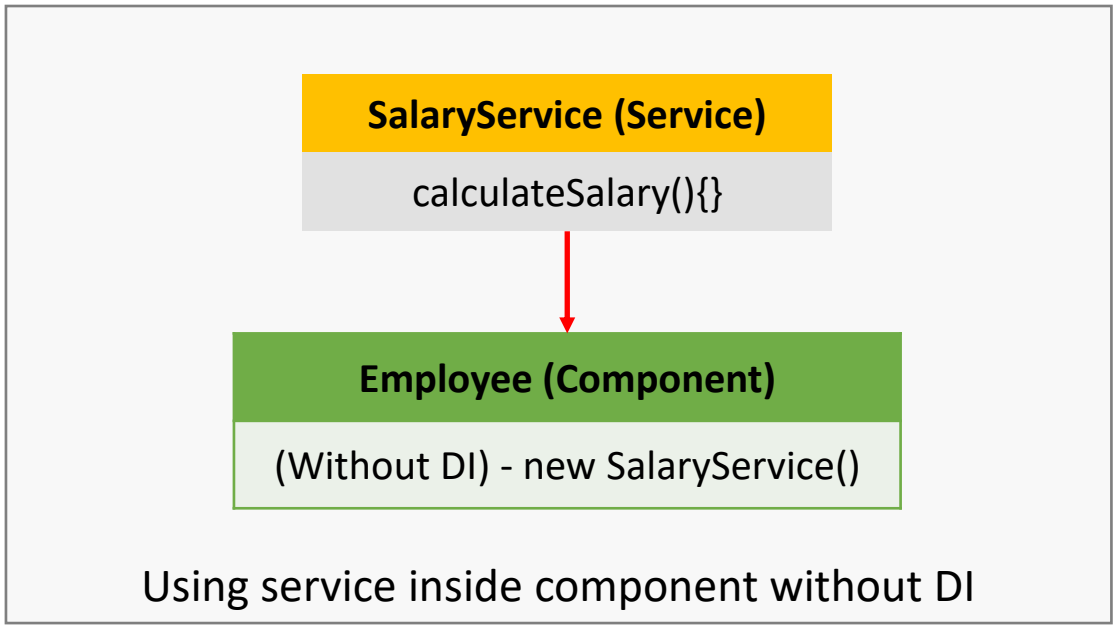
@Injectable({
  providedIn: 'root’
})

export class SalaryService {
  calculateSalary(base: number, bonus: number) {
    return base + bonus;
  }
}
```

Q. How to use an Angular service inside a component without Dependency Injection?



- How to use Angular Service inside Component Without DI:
  Create the service object manually inside the component using new.
  const service = new MyService();



Using service inside component without DI

```
export class SalaryService {
  calculateSalary(base: number, bonus: number) {
    return base + bonus;
   }
}
```

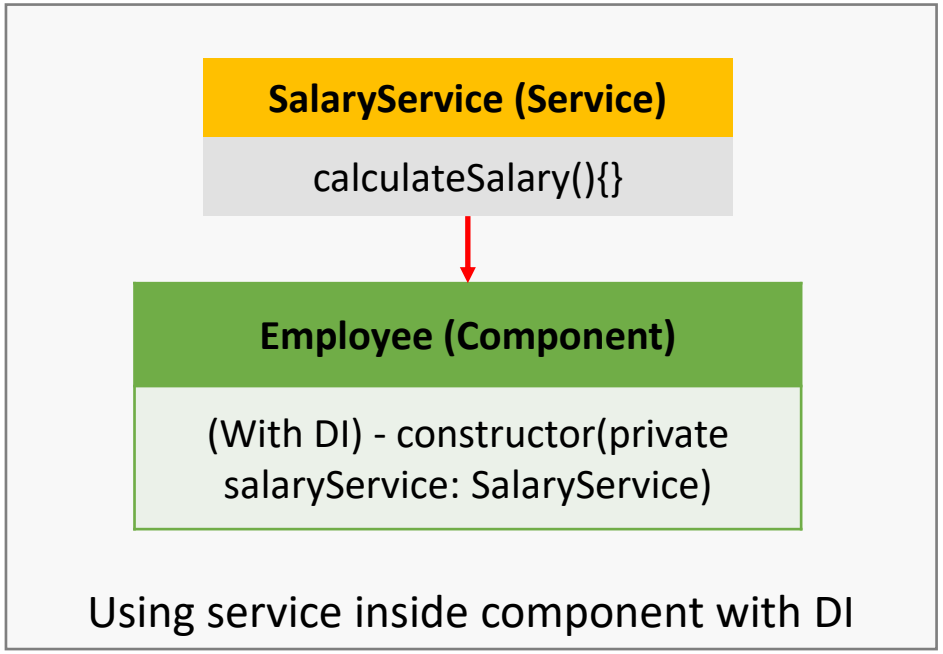
```
// Service implemented Without DI
export class Empoyee {
  base = 30000; bonus = 5000;
  finalSalary = 0;
  salaryService: SalaryService;

  constructor() {
    // Manually creating the service object
    this.salaryService = new SalaryService();
    this.finalSalary =
this.salaryService.calculateSalary(this.base,
this.bbonus);
  }
}
```

Q. What is Dependency Injection? How to implement it?



- Dependency Injection (DI) in Angular is a design pattern where **Angular automatically provides(inject) required services or objects to a component** instead of creating them manually.



Using service inside component with DI

```
export class SalaryService {
  calculateSalary(base: number, bonus: number) {
    return base + bonus;
   }
}
```

```
// Service implemented With DI
export class Employee {
  base = 30000; bonus = 5000;
  finalSalary = 0;

  constructor(private salaryService: SalaryService) {
    this.finalSalary =
this.salaryService.calculateSalary(this.base, this.bonus);
  }
}
```

Q. What are the Advantages of Dependency Injection? Disadvantages of creating a service using new in a component?



❖ Disadvantages of using **new keyword:**

1. Using new can create multiple service instances
2. Creates tight coupling btw component and service
3. Reduces flexibility - Makes testing and mocking more difficult

```
// Service implemented Without DI
export class Empoyee {
  base = 30000; bonus = 5000; finalSalary = 0;
  salaryService: SalaryService;

  constructor() {
    // Manually creating the service object
    this.salaryService = new SalaryService();

    this.finalSalary =
this.salaryService.calculateSalary(this.base,
this.bbonus);
  }
}
```

❖ Advantages of using **Dependency Injection:**

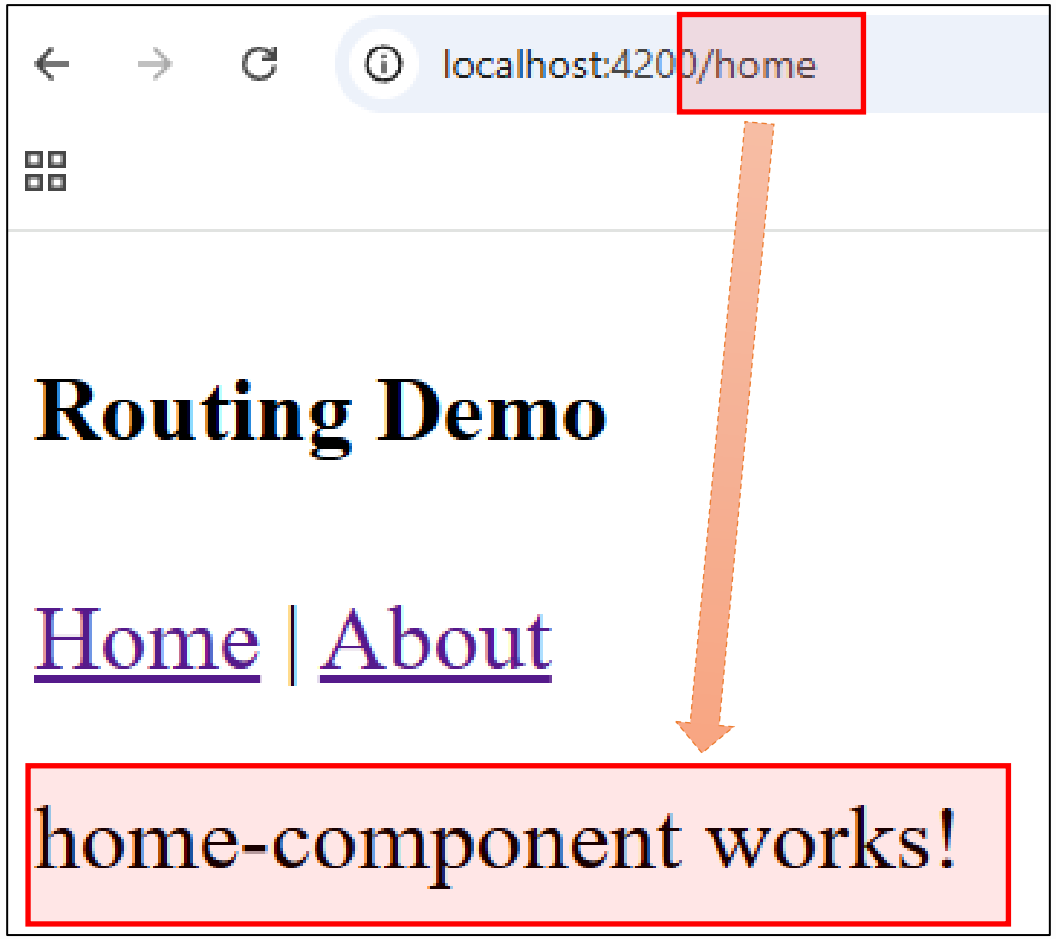
1. Angular provides centralized object management (singleton)
2. Loose coupling between components and services
3. Enhances flexibility - Makes testing and mocking easier

```
// Service implemented With DI
export class Employee {
  base = 30000; bonus = 5000; finalSalary = 0;

  constructor(private salaryService: SalaryService) {
    this.finalSalary =
this.salaryService.calculateSalary(this.base, this.bonus);
  }
}
```

Angular Routing

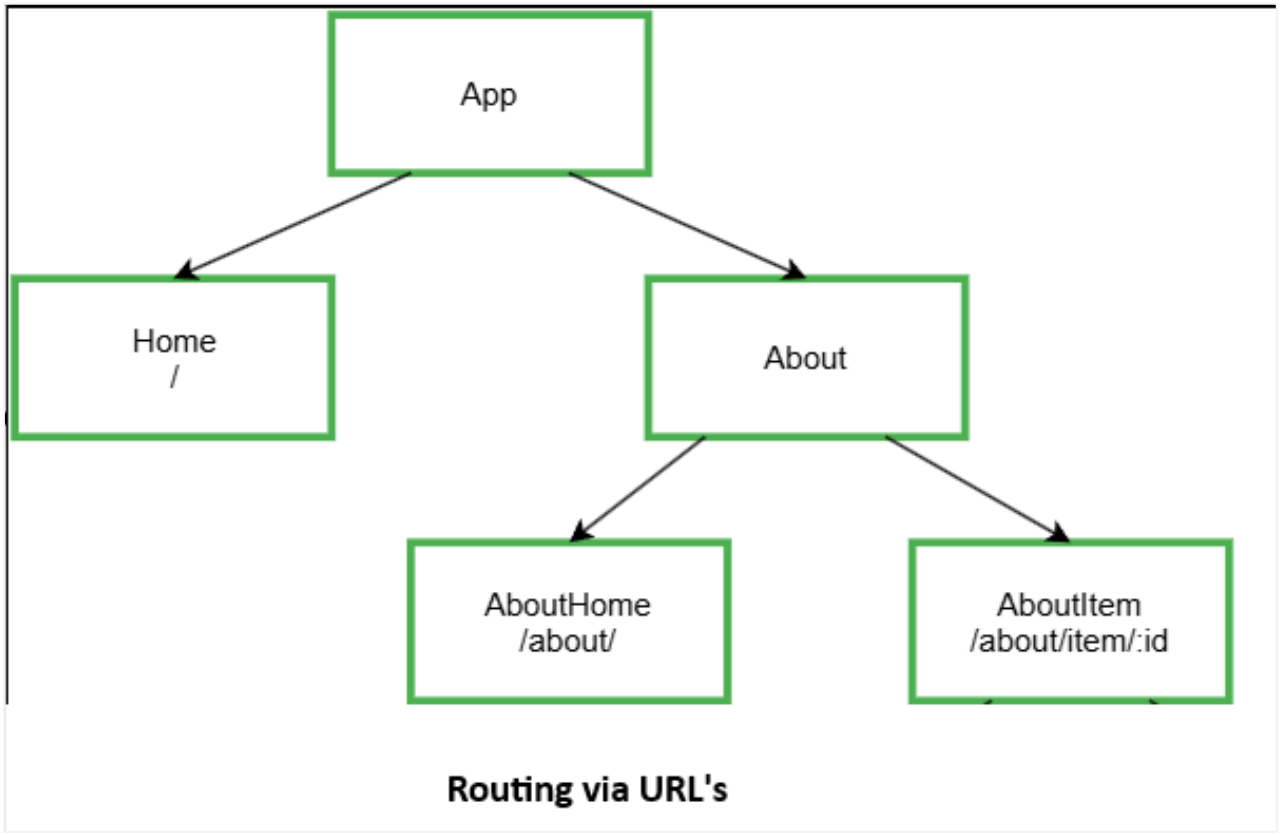
- 👉 Routing Introduction
- 👉 provideRouter
- 👉 app.routes.ts
- 👉 routerLink & router-outlet
- 👉 routerLinkActive
- 👉 Wildcard Route



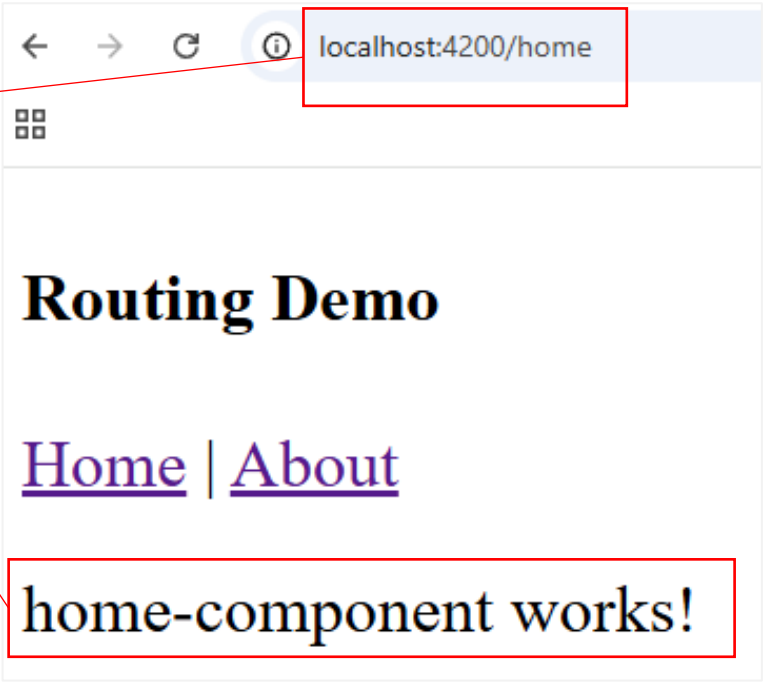
Q. What is Angular Routing?



- **Routing** is a feature in Angular that allows navigation between different views or **components using URLs**, without reloading the entire web page.

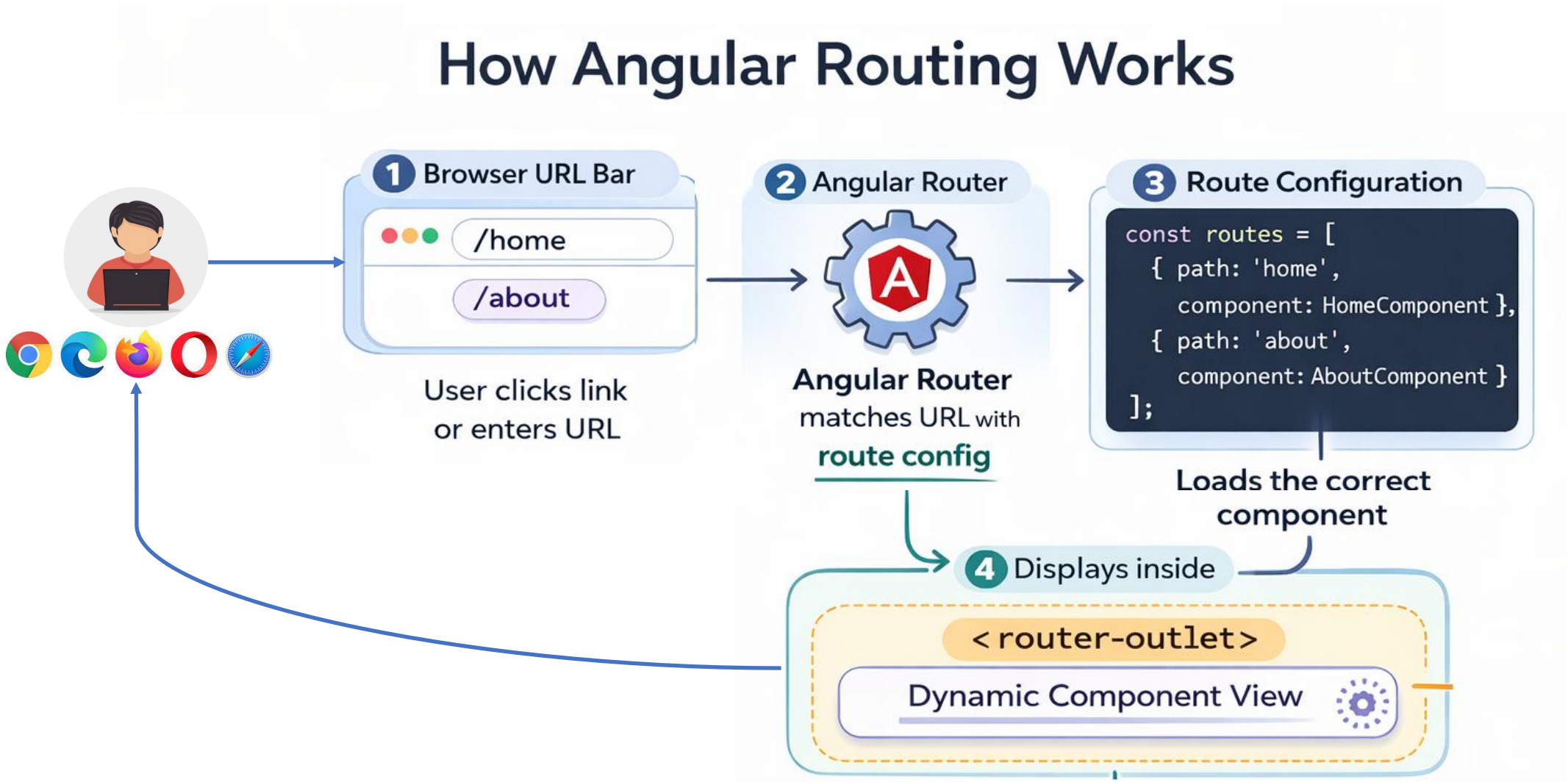


Routing via URL's



Q. How Angular Routing Works? V. IMP.





Q. How do we implement Routing? What is the role of provideRouter?



- **provideRouter** is a function used in Angular to **enable routing** by providing route configuration during application startup.

```
// app.config.ts
export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideZoneChangeDetection({ eventCoalescing: true }),
    provideRouter(routes)
  ]
};
```

Q. What is the role of app.route.ts in routing implementation?



- **app.routes.ts** is the main routing configuration file in Angular that **contains the Routes array** mapping URL paths to their respective components.
- **Routes Array** is an Angular configuration array where each route object defines a **URL path and the component** to be loaded for that path.
- **default route** is used to redirect users to a specific component when no URL path is provided.

```
// app.route.ts
export const routes: Routes = [
  { path: 'home', component: HomeComponent },
  { path: 'about', component: AboutComponent },
  //   default route
  { path: '', redirectTo: 'home', pathMatch: 'full' }
];
```

Q. What is the role of routerLink and router-outlet in routing implementation?



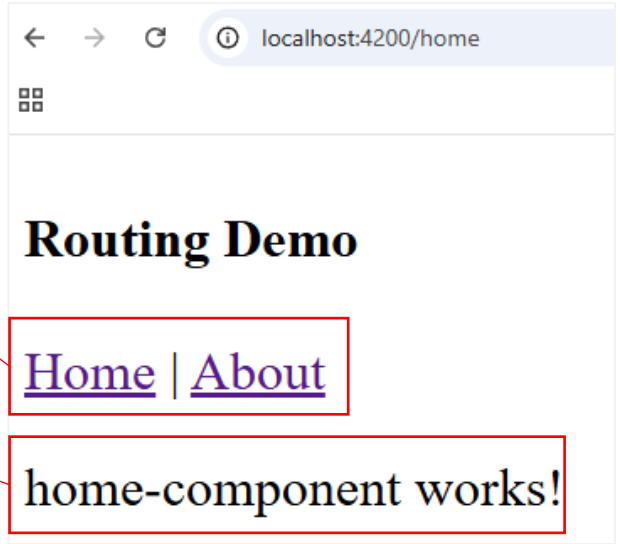
- **routerLink** is an Angular directive used to **navigate between components using URLs** without refreshing the entire page.

- **router-outlet** is an Angular directive that acts as a **placeholder** where routed components are displayed based on the current URL.

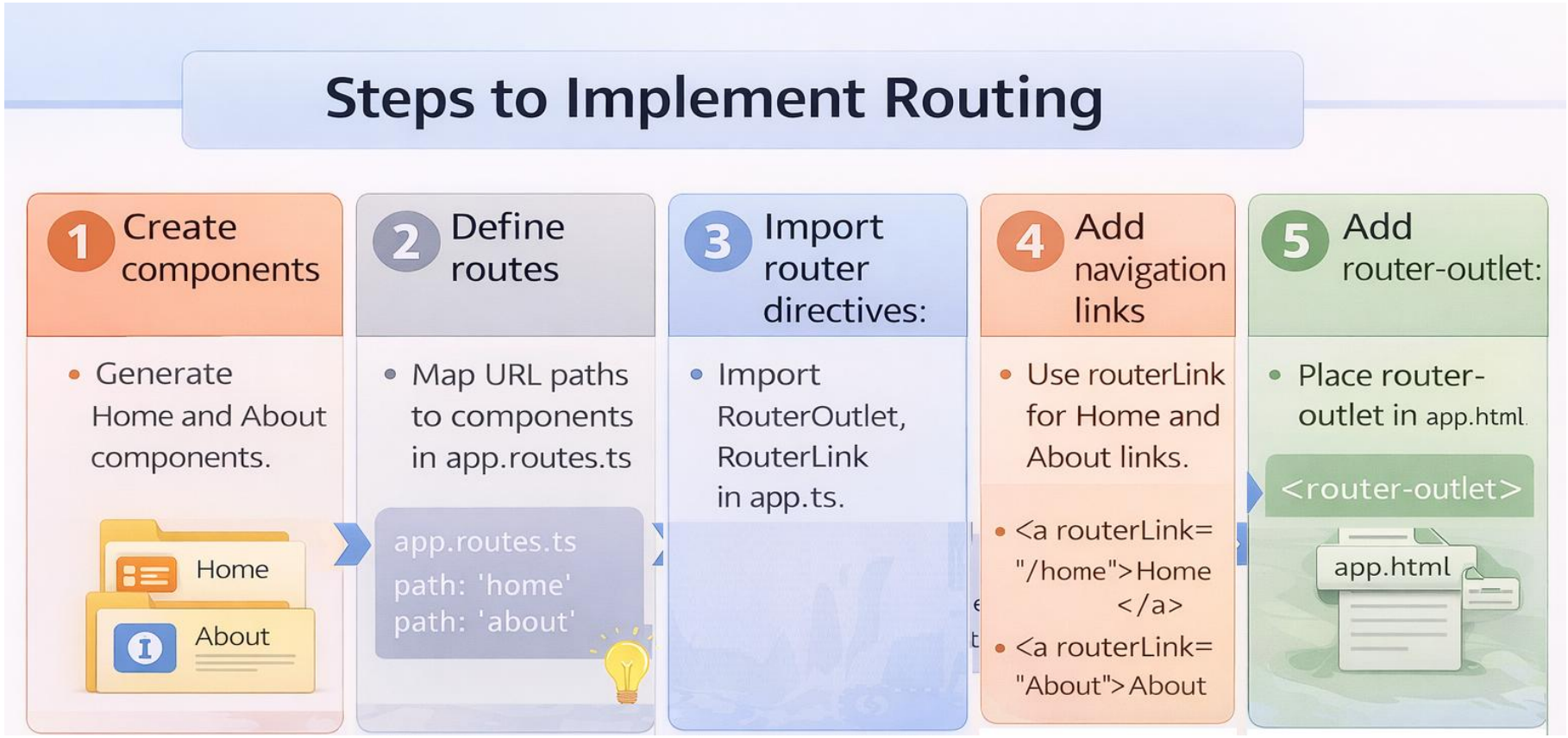
```
// app.html

<a routerLink="/home">Home</a> |
<a routerLink="/about">About</a>

<router-outlet></router-outlet>
```



Q. Summarize Angular Routing Working?



Q. What is the role of routerLinkActive?



- **RouterLinkActive** is an Angular directive that **automatically adds a CSS class to a link** when its associated route is currently active.

```
@Component({
  selector: 'app-root',
  imports: [RouterOutlet, RouterLink, RouterLinkActive],
  templateUrl: './app.html',
  styleUrl: './app.css'
})
```

```
<h4>Routing Demo</h4>

<a routerLink="/home" routerLinkActive="active">Home</a> |
<a routerLink="/about" routerLinkActive="active">About</a>

<router-outlet></router-outlet>
```

```
.active {
  font-weight: bold;
  color: red;
}
```

Home | About

about-component works!

Q. What is a Wildcard Route (**)?

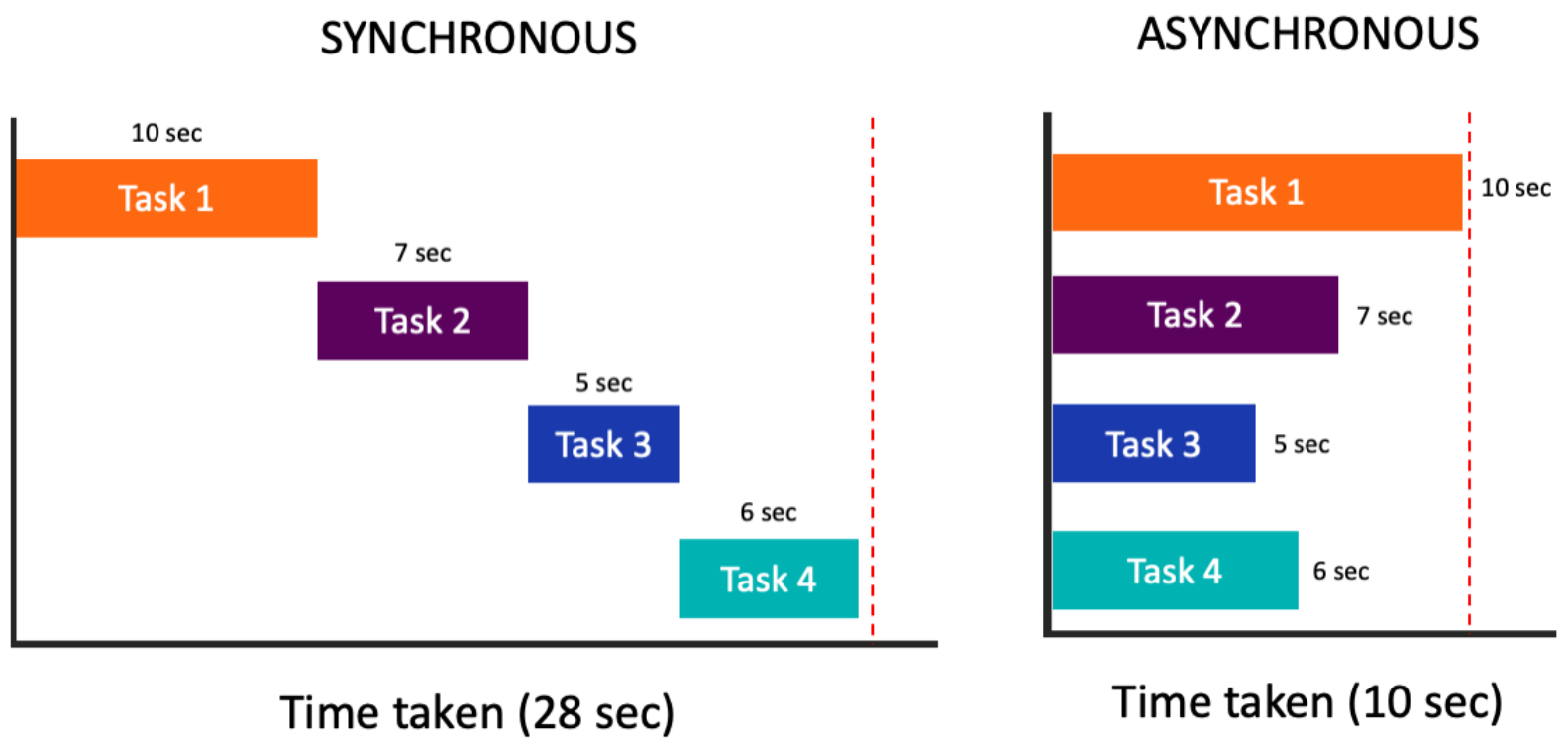


- A wildcard route is used **handle invalid or unknown URLs.**
- Mostly it is used to redirects users to a 404 page or default page.
- Always place the wildcard route at the end of the routes array.

```
// app.routes.ts
export const routes: Routes = [
  { path: 'home', component: Home },
  { path: 'about', component: About },
  // default route
  { path: '', redirectTo: 'home', pathMatch: 'full' },
  // wildcard **
  { path: '**', component: PageNotFound },
];
```

Asynchronous programming

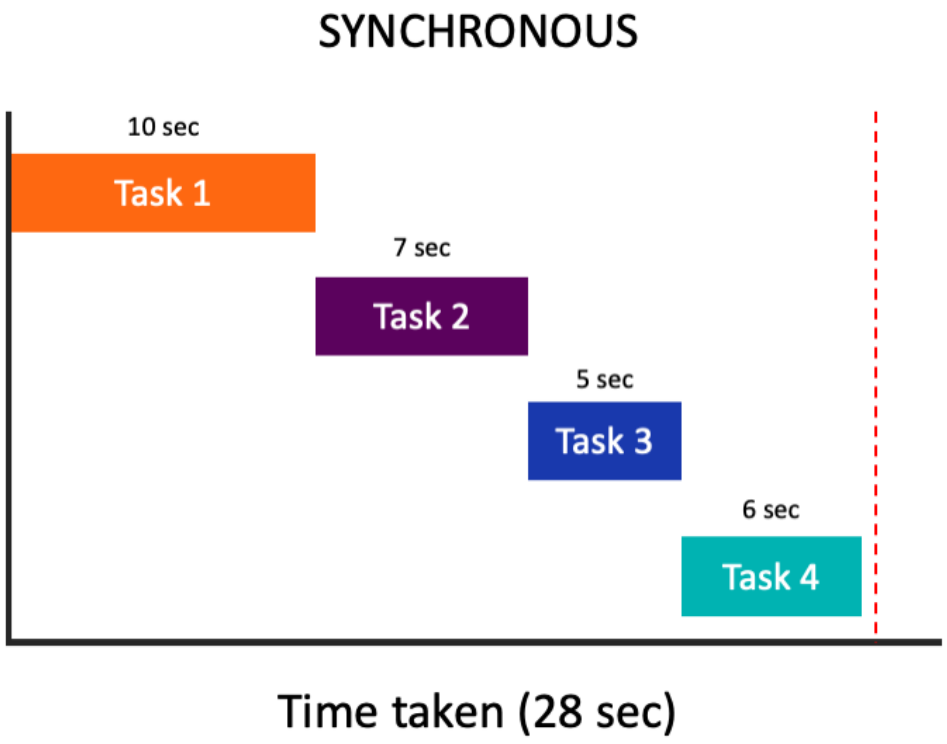
- 👉 Synchronous vs Asynchronous
- 👉 Asynchronous Techniques



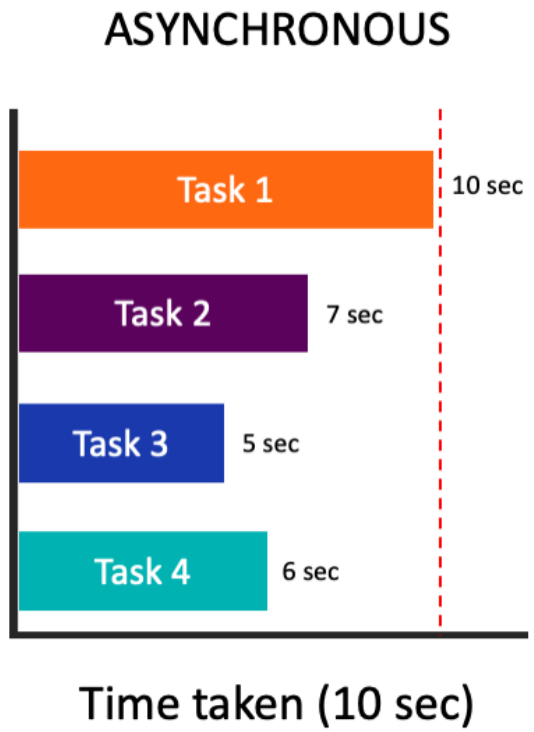
Q. What is Synchronous and Asynchronous programming? V. IMP.



- Synchronous programming means the code runs one line at a time, in order. A task must finish first before the next task starts.

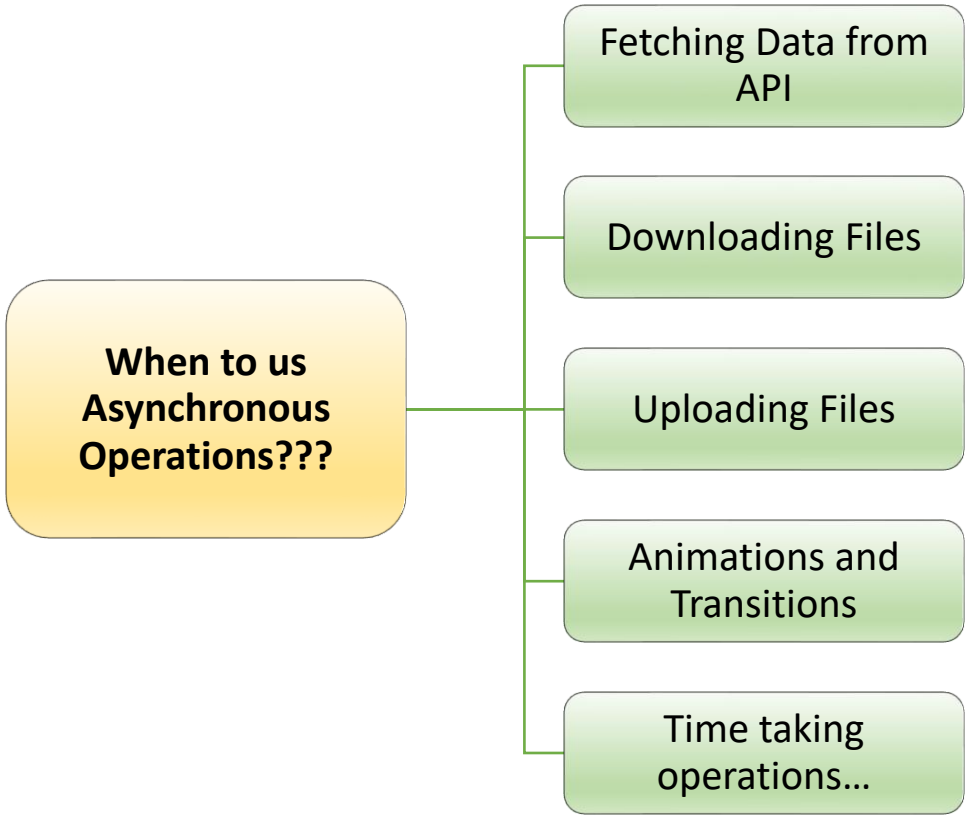


- Asynchronous programming allows tasks to start and continue without waiting for other tasks to finish.



Q. When to use Asynchronous programming in real applications?






```
// Synchronous Programming
// Not efficient

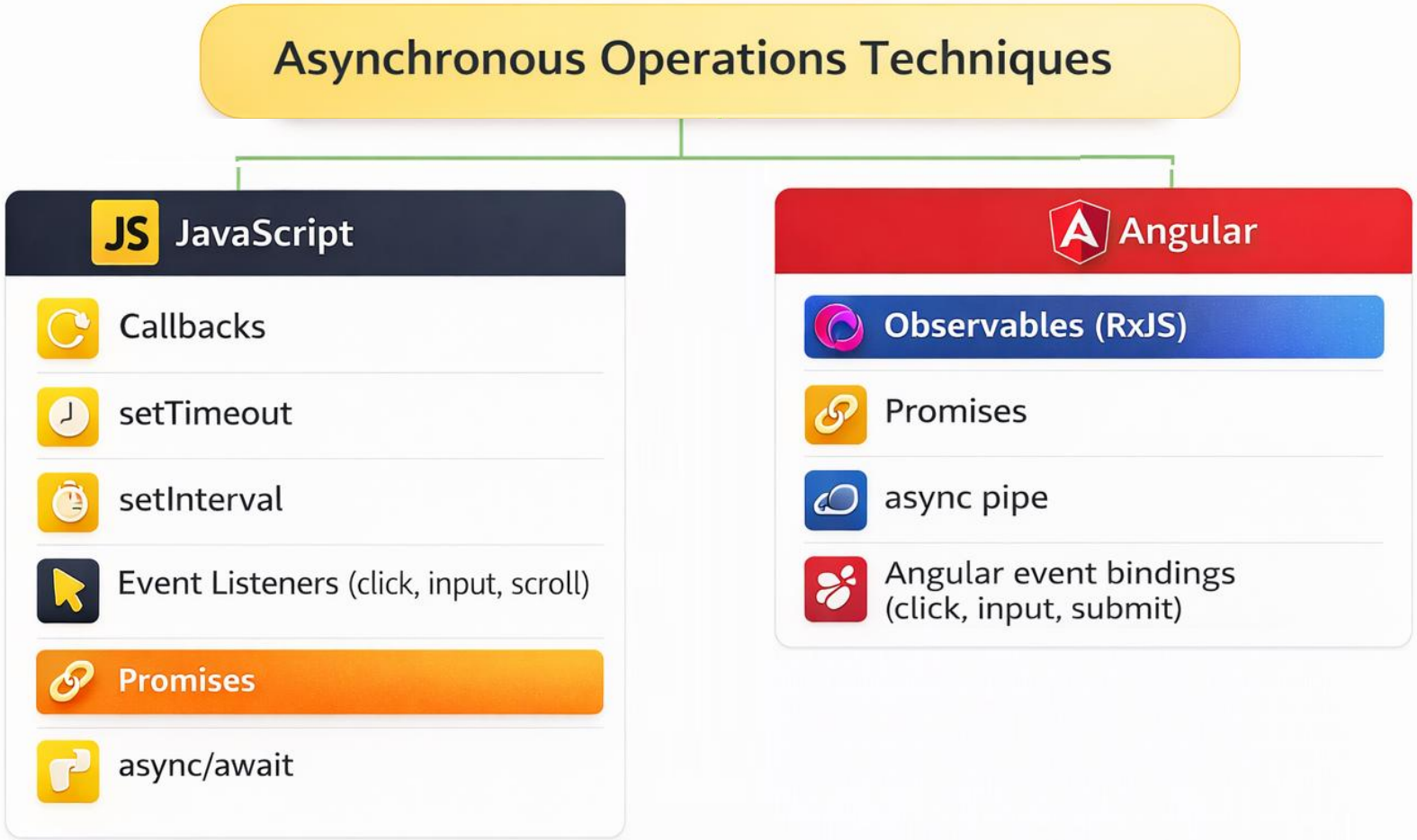
Function1();
Function2();
```

```
// Time taking function
function Function1() {
  // Loading Data from an API
  // Uploading Files
  // Animations
}
```

```
function Function2() {
  console.log(100 + 50);
}
```

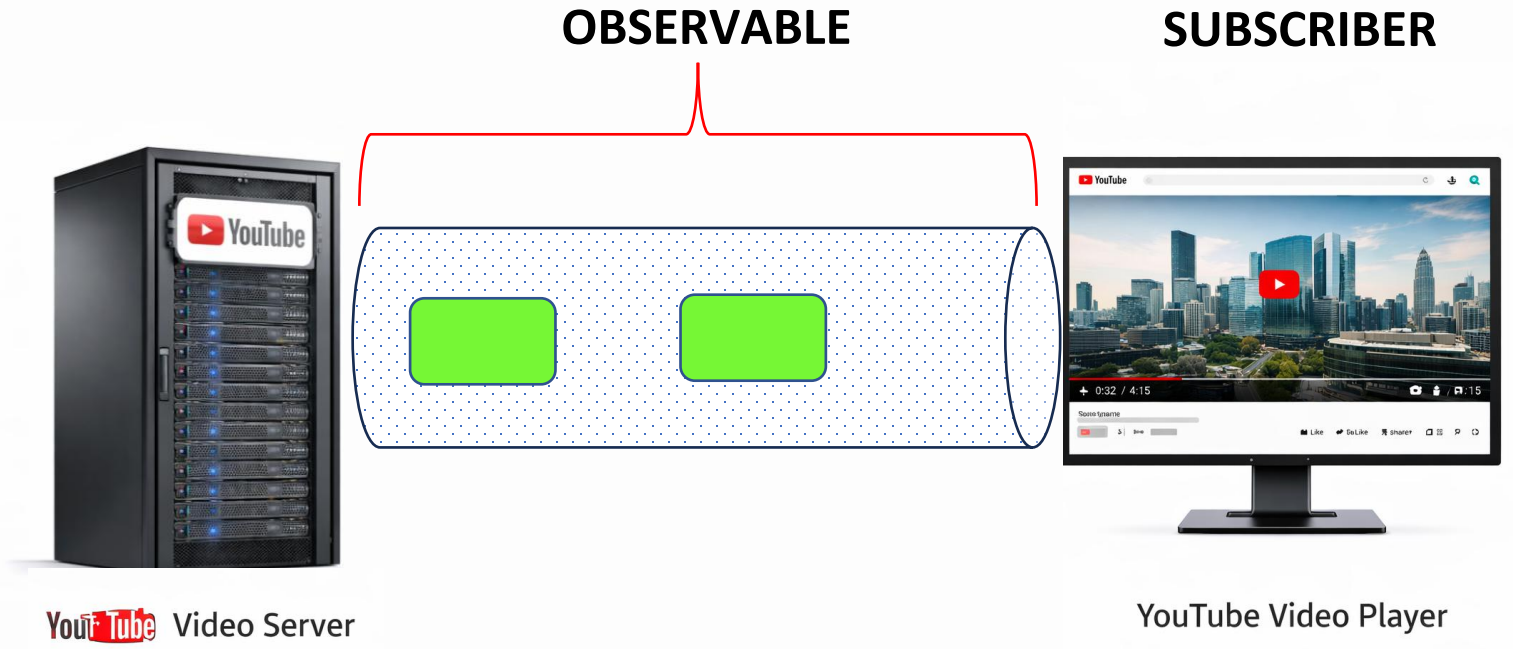
Q. What are the techniques for achieving asynchronous operations in JS and Angular?





Observables

- 👉 RxJS
- 👉 Observables
- 👉 Observer
- 👉 Subscriber



Q. What are RxJS and Observables?



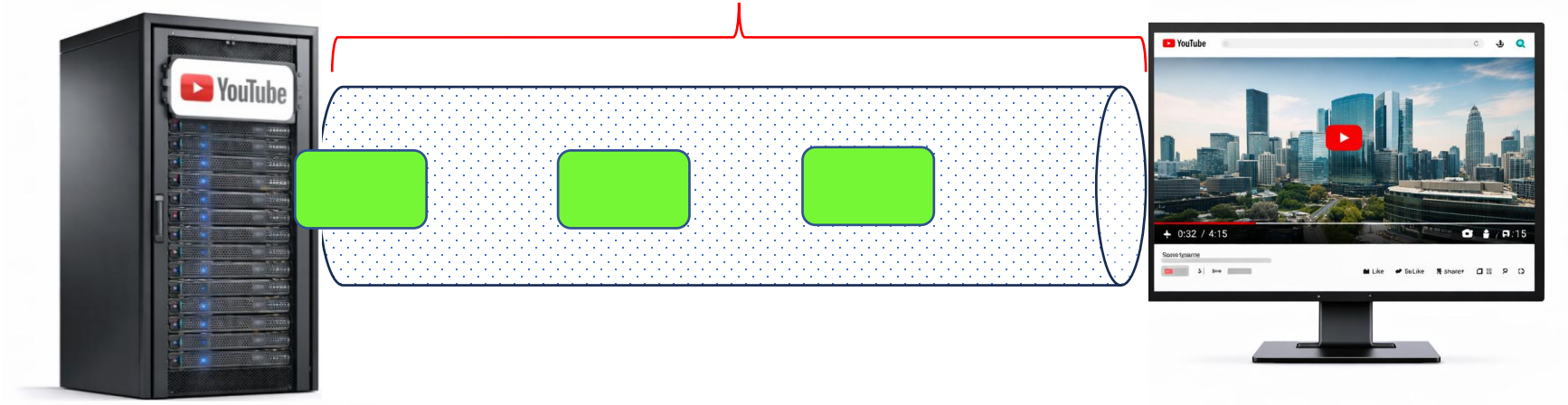
- RxJS(Reactive Extensions for JavaScript) is a reactive programming library that manages continuous data streams using Observables.

YouTube Streaming – Understanding RxJS

Data Source
Origin where data is generated or emitted

Observable
Represents a stream of data that emits values over time.

Subscriber
Subscribes to the Observable and receives the data emitted by it.



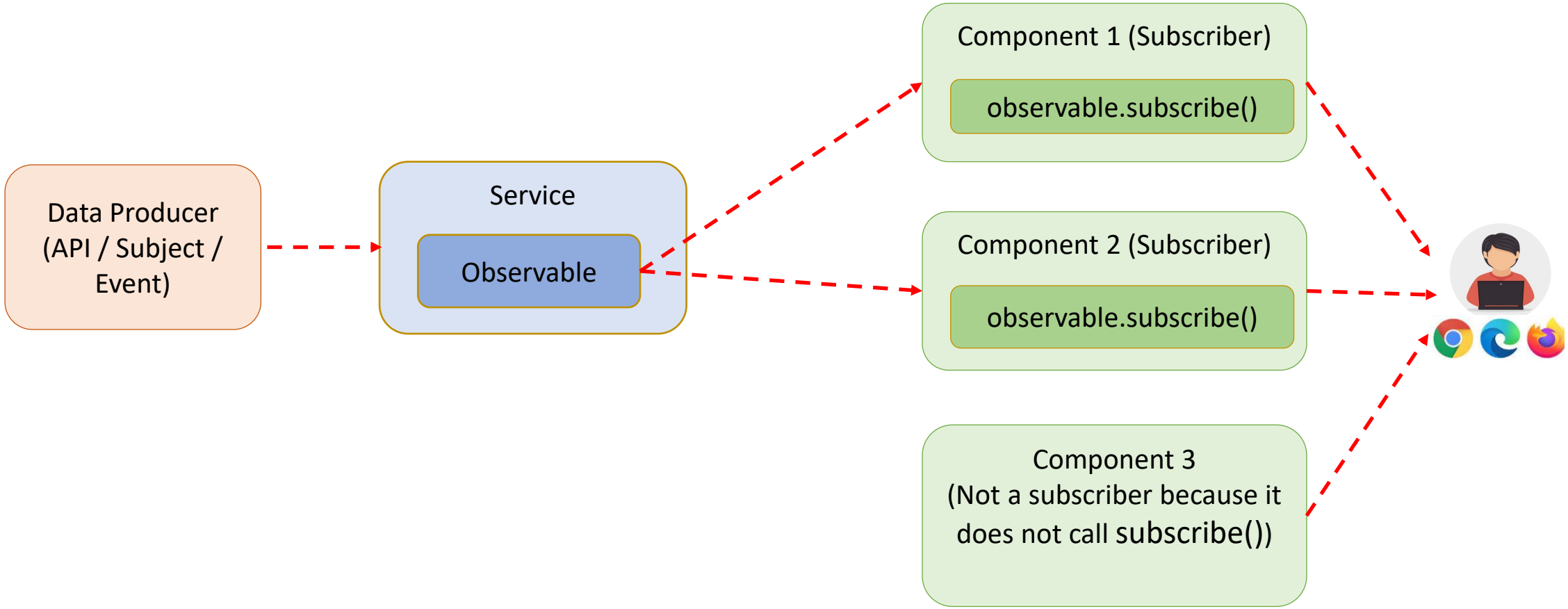
YouTube Video Server

YouTube Video Player

Q. Explain Data Flow between Observables and Subscriber in detail?



Observable – Subscriber: Data Flow Explained



Q. How do we implement Observable and Subscriber?



- How to setup Observable & Subscriber:

1. Create an Observable in a service and return it from a method.
2. In the component, inject the service and call that method.
3. Then use subscribe() on the Observable.
4. **By calling subscribe()**, the component becomes the Subscriber and can receive the emitted data.

```
// service.ts
import { Injectable } from '@angular/core';
import { Observable } from 'rxjs';
@Injectable({
  providedIn: 'root’
})
export class ObservableService {// Create & return Observable
  getNumber(): Observable<number> {
    return new Observable();
  }
}
```

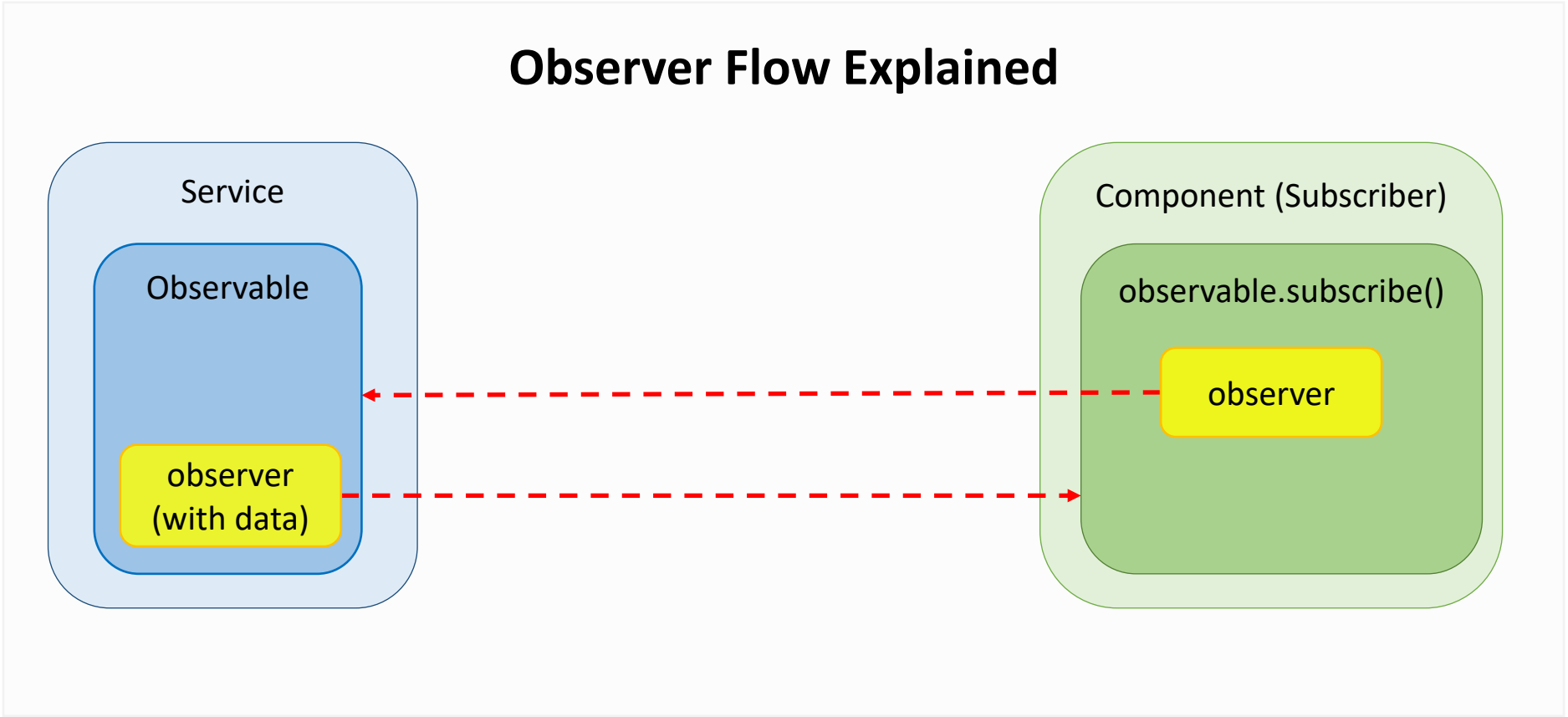
```
// my-component.ts
export class MyComponent {

  constructor(private service: ObservableService) {
    // Subscribing the observable
    this.service.getNumber().subscribe();
  }
}
```

Q. What is Observer? V. IMP.



- Observer is an object used by the Subscriber to receive data from the Observable.



Q. How to implement observer?



❖ How Observer Works Step-by-Step

1. Component creates Observer object.
2. Observer is passed to subscribe().
3. RxJS connects Observer to Observable.
4. Observable emits data using observer.next().
5. Observer receives data in next() handler.

```ts
// component.ts
export class MyComponent implements OnInit {
  items: number[] = [];
  constructor(private service: ObservableService) {
    this.service.getNumber().subscribe({
      next: (data) => {
        this.items.push(data);
      },
    });
 }
```

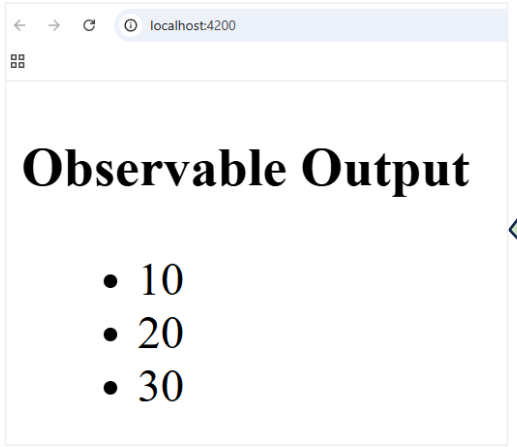
```ts
// service.ts
export class ObservableService {
  getNumber(): Observable<number> {
    return new Observable((observer) => {
      observer.next(10);
      observer.next(20);
      observer.next(30);
      observer.complete();
    });
  }}
```

Q. How does Subscriber display data received from an Observable?



- Display the data in the component template file **using *ngFor directive.**

```
// component.ts
export class MyComponent {
  items: number[] = [];
  constructor(private service: ObservableService) {
    this.service.getNumber().subscribe({
      next: (data) => {
        this.items.push(data);
      },
    });
  }
```




```
// component.html
<h3>Observable Output</h3>
<ul>
  <li *ngFor="let item of items">
    {{ item }}
  </li>
</ul>
```

Q. Why do we use ngOnInit() instead of the constructor to subscribe to an Observable?



- Constructor runs when the component class is created, before ngOnInit() and before the template is fully initialized.
- It is good for injecting services, but not ideal for subscribing to Observables.

```
// component.ts
export class MyComponent {
  items: number[] = [];

  constructor(private service: ObservableService) {
    this.service.getNumber().subscribe({
      next: (data) => {
        this.items.push(data);
      },
    });
  }
```

- ngOnInit() is an Angular lifecycle hook that runs **after Angular initializes the component** and its template, so the component can safely receive data and display it in the UI.

```
// component.ts
export class MyComponent implements OnInit {
  items: number[] = [];
  constructor(private service: ObservableService) {}

  ngOnInit() {// Component HTML is fully iniatialized
    this.service.getNumber().subscribe({
      next: (data) => {
        this.items.push(data);
      },
    });
  }
}
```

Q. Summarize the implementation of Observable and Subscriber?



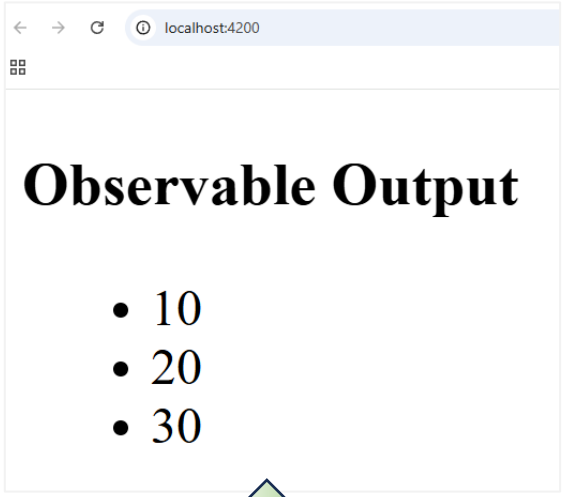
1. Create service and return Observable
2. Inject service in component
3. Subscribe inside ngOnInit()
4. Send data using observer.next()
5. Store and display using *ngFor*

```ts
// service.ts
export class ObservableService {
  getNumber(): Observable<number> {
    return new Observable((observer) => {
      observer.next(10);
      observer.next(20);
      observer.complete();
    });
 }}
```

```ts
// component.ts
export class MyComponent {
  items: number[] = [];
  constructor(private service: ObservableService) {}
  ngOnInit() {
    this.service.getNumber().subscribe({
      next: (data) => {
        this.items.push(data);
      },
    });
  }}
```

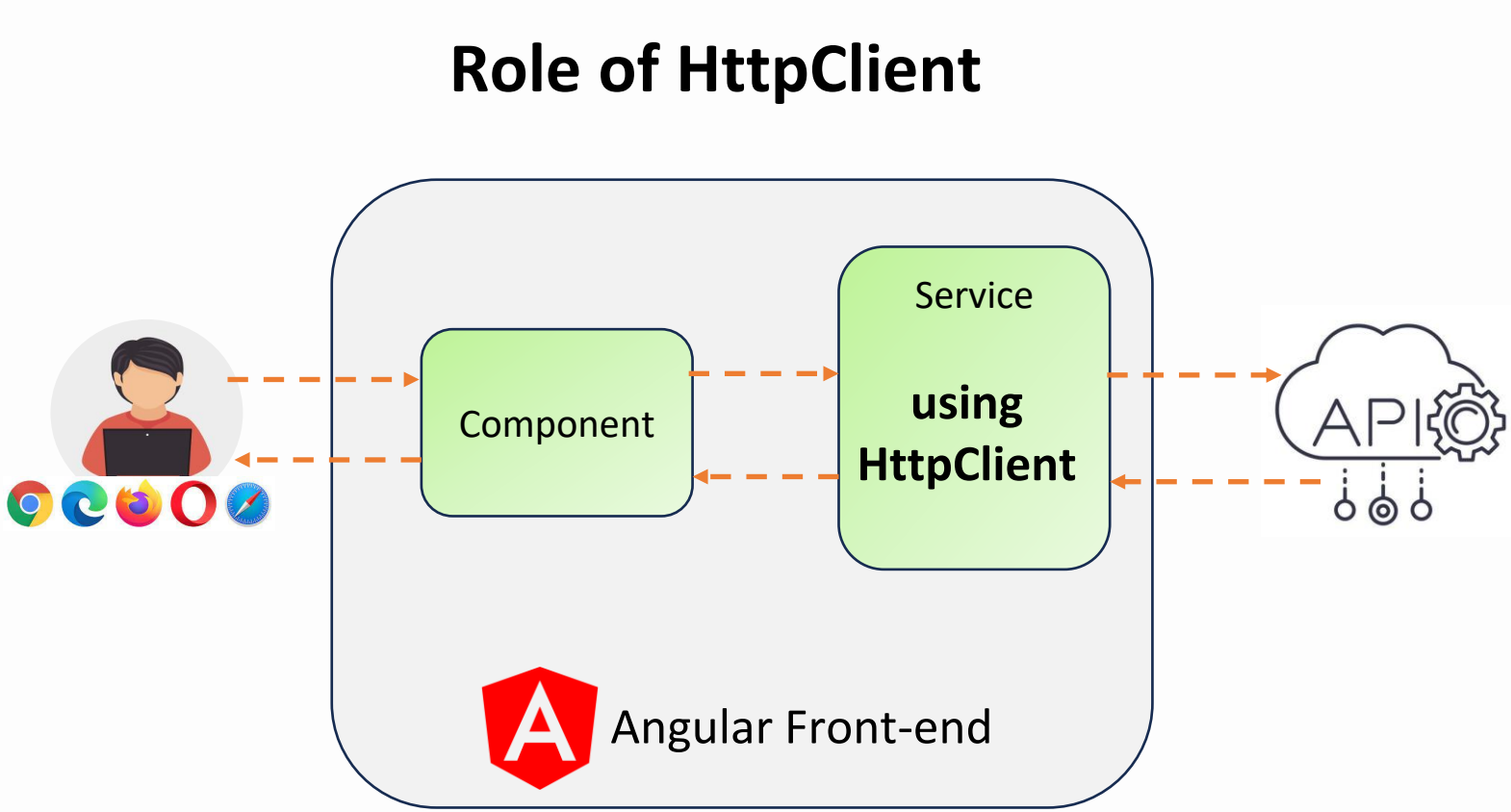
```html
// component.html
<h3>Observable Output</h3>

<ul>
  <li *ngFor="let v of values">
    {{ v }}
  </li>
</ul>
```



HttpClient

- 👉 HttpClient Introduction
- 👉 HttpClient Methods
- 👉 provideHttpClient

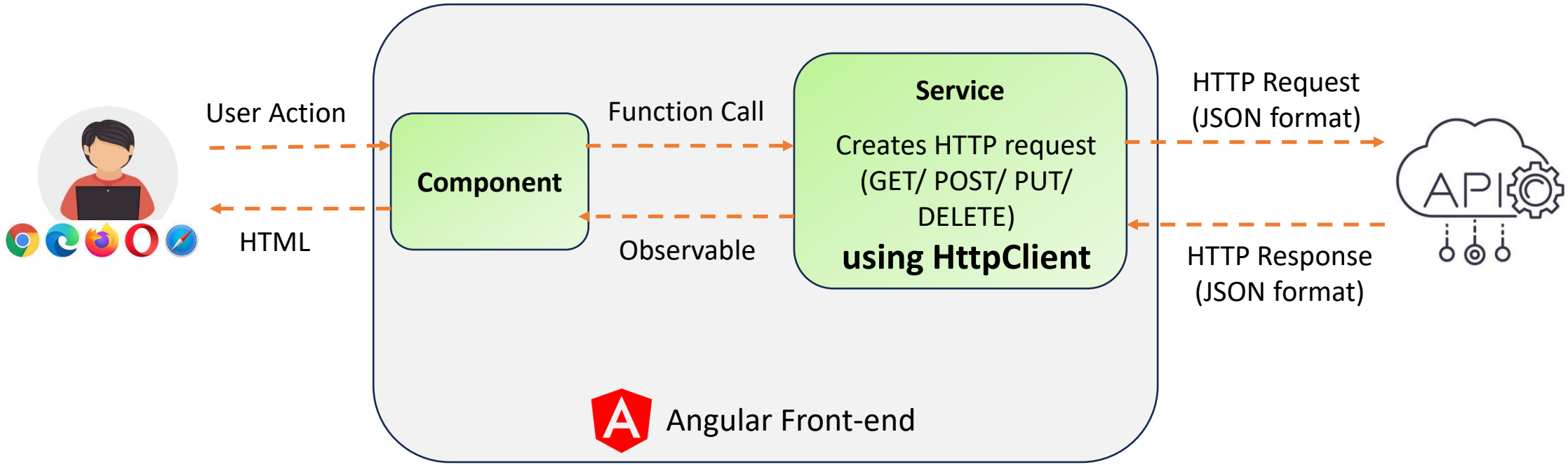


Q. What is the role of HttpClient in Angular?

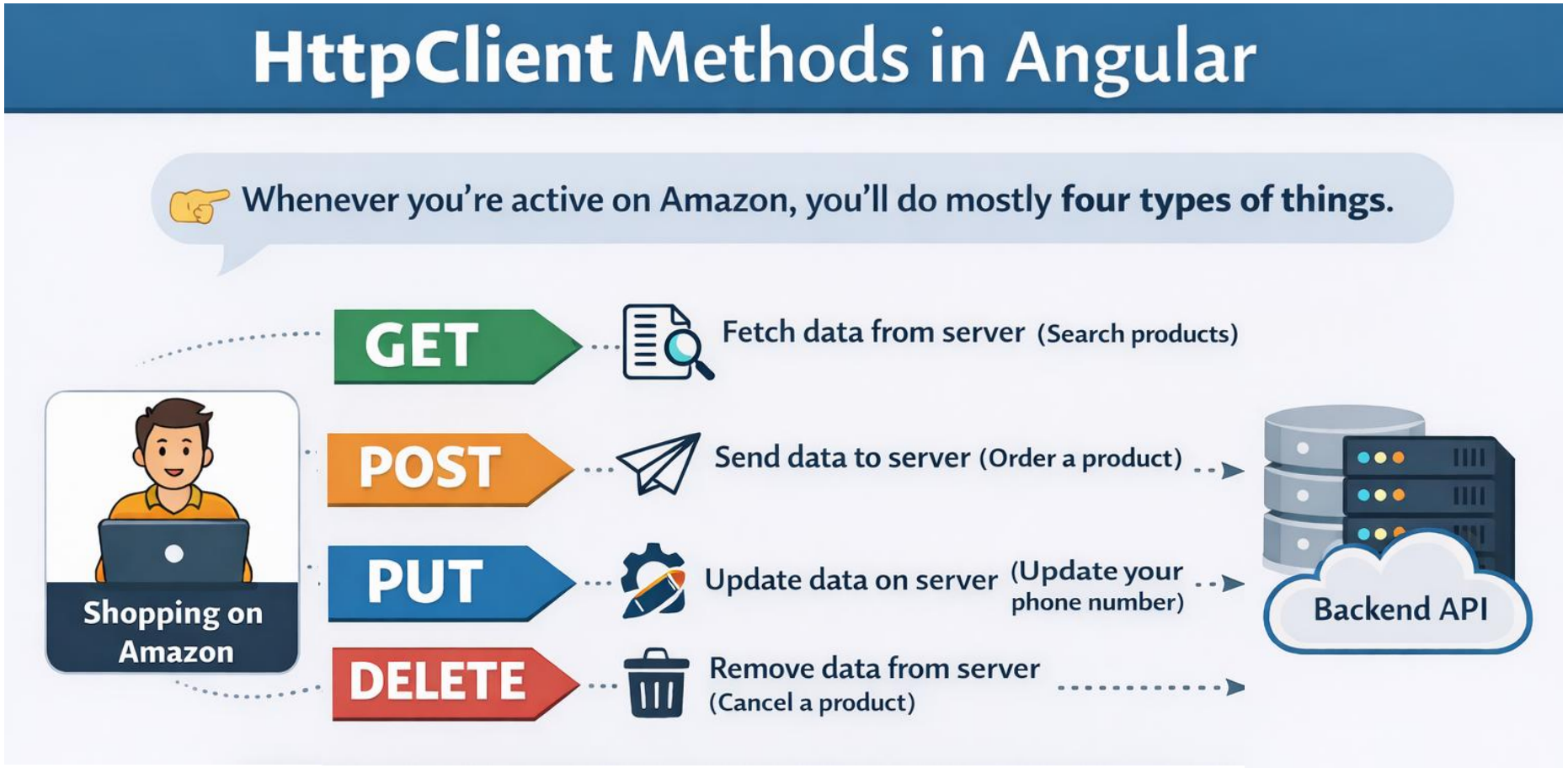


- HttpClient is an Angular service used to make **HTTP calls** like GET, POST, PUT, DELETE to a backend server or API.

Role of HttpClient



Q. What are the main methods of HttpClient?



Q. What is the role of provideHttpClient (HttpClient Implementation)?



- **app.config.ts** is a configuration file used in standalone Angular applications to **register global providers** and application-level settings before the app starts.

- **provideHttpClient()** is a function used to register the HttpClient service in Angular's Dependency Injection system so that the application can make HTTP requests to APIs.

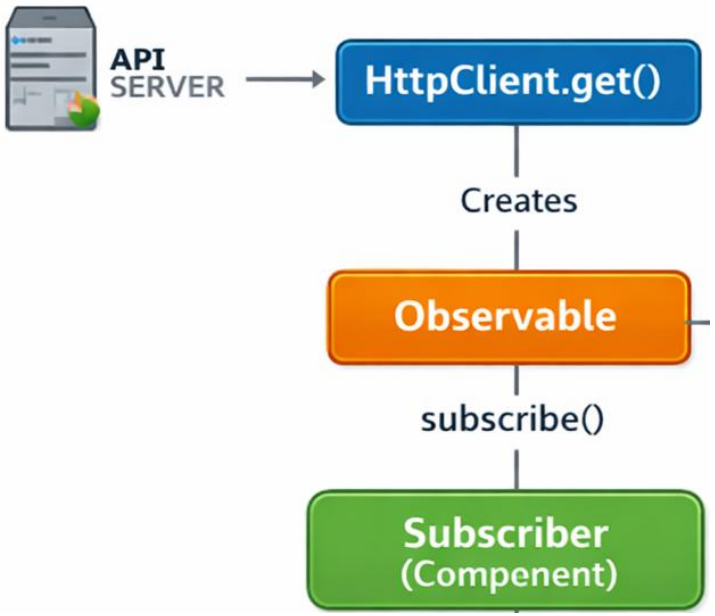
```ts
// app.config.ts
export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideRouter(routes),
    provideHttpClient()
  ],
};
```

Q. How do you inject HttpClient (HttpClient Implementation)?



- To inject HttpClient, first register it using **provideHttpClient()** in app.config.ts, then inject it using constructor dependency injection.

- HttpClient internally uses RxJS to **create an Observable** that handles the HTTP response.




```
// service
export class UserService {
  constructor(private http: HttpClient) {}

  getUsers(): Observable<any> {
    return
this.http.get('https://jsonplaceholder.typicode.com/users
');
  }
}
```

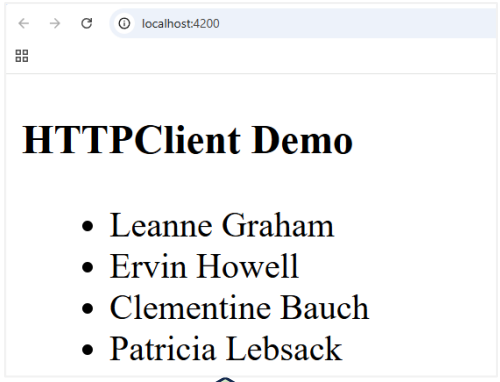
Q. How to display API data using HttpClient (HttpClient Implementation)?



```
// app.config.ts
export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient()
   ],};
```

```
// service.ts
export class UserService {
  constructor(private http: HttpClient) {}
  getUsers(): Observable<any> {
    return
this.http.get('https://jsonplaceholder.typicode.com/users');
  }}
```

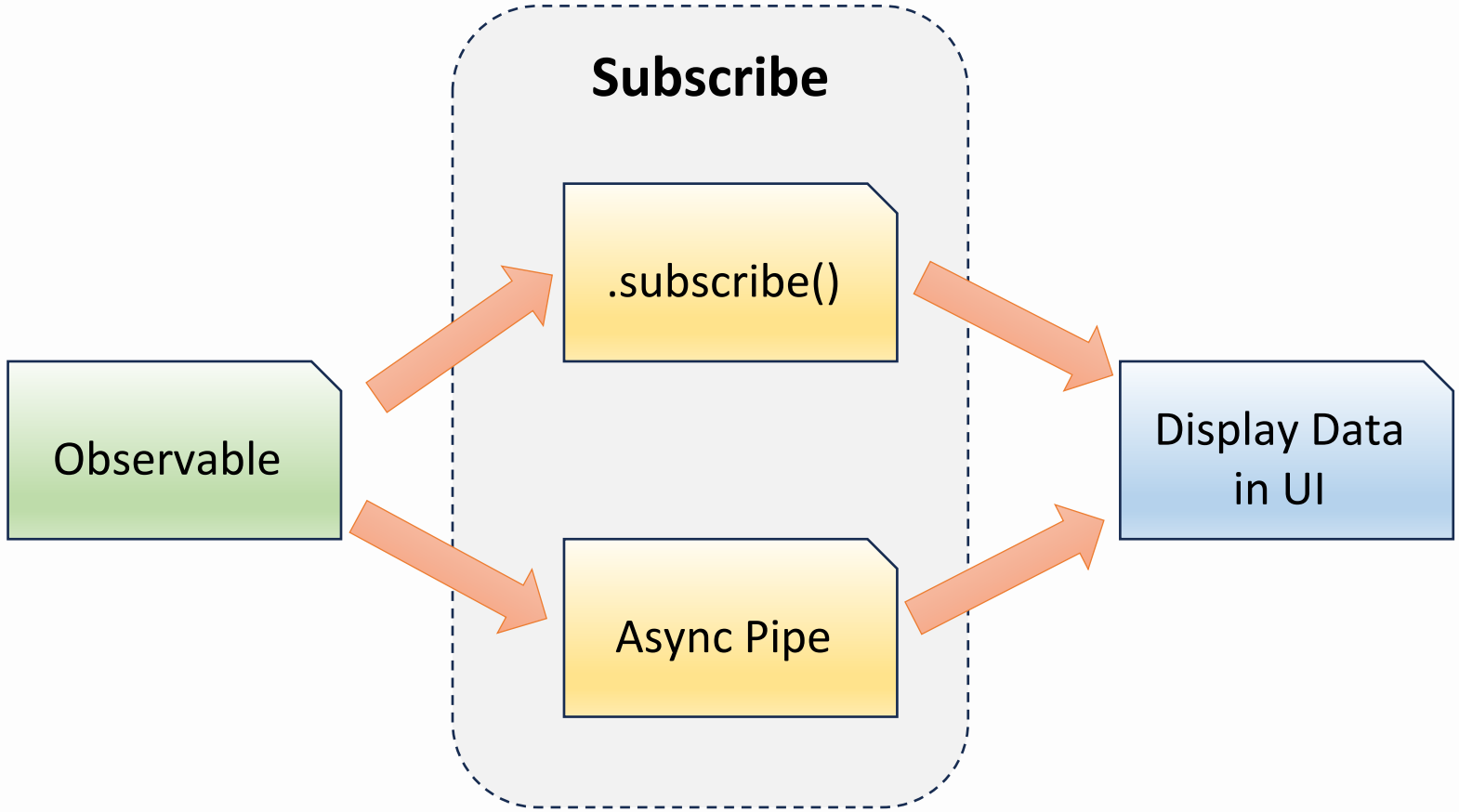
```
// component.ts
export class User implements OnInit {
  users: any[] = [];
  constructor(private userService: UserService) {}
  ngOnInit(): void {
    this.userService.getUsers().subscribe({
      next: (data) => {
        this.users = data;
      }
    });
  }}
```




```
// component.html
<h2>Users</h2>
<ul>
    <li *ngFor="let u of users">{{ u.name }}</li>
</ul>
```

Async Pipe

- 👉 Async Pipe



Q. What is Async Pipe?



- Async Pipe is a built-in Angular pipe that **automatically subscribes to an Observable** and gives the latest emitted value directly to the template.

```ts
// component.ts
export class MyComponent {
  users$: any;
  constructor(private userService: UserService) {}

  ngOnInit(): void {
    this.users$ = this.userService.getUsers();
  }
}
```

```html
// component.html
<h2>Async Pipe Example</h2>
<ul>
  <li *ngFor="let u of users$ | async">
    {{ u | json }}
  </li>
</ul>
```

Q. What are the advantage of using Async Pipe?



- Advantages of Async Pipe over subscribe function:

1. No need to manually call subscribe()
2. Automatically subscribes in the template
3. Cleaner and more concise code
4. Automatically unsubscribes on component destroy, preventing memory leaks

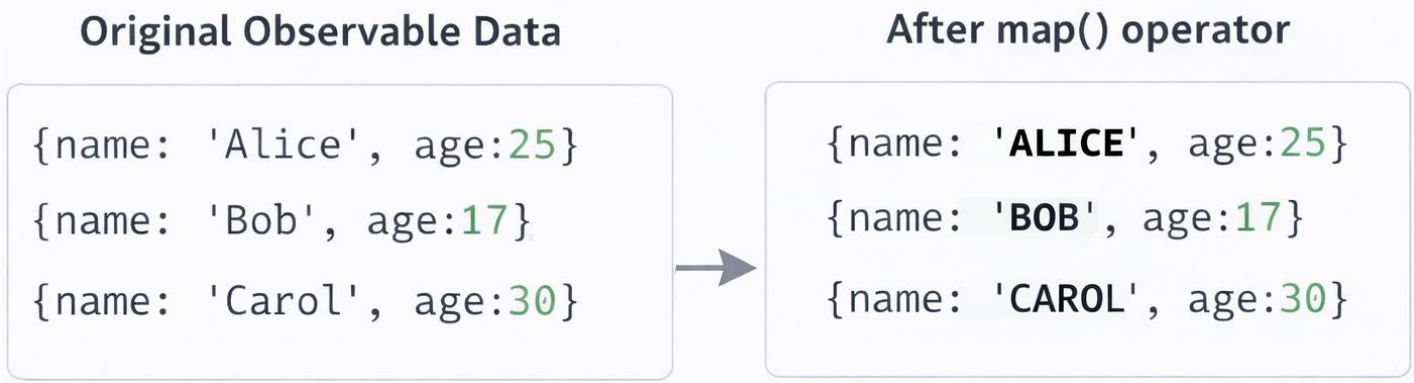
```ts
// component.ts
export class MyComponent {
  users$: any;
  constructor(private userService: UserService) {}

  ngOnInit(): void {
    this.users$ = this.userService.getUsers();
  }
}
```

```html
// component.html
<h2>Async Pipe Example</h2>
<ul>
  <li *ngFor="let u of users$ | async">
    {{ u | json }}
  </li>
</ul>
```

RxJS Operators

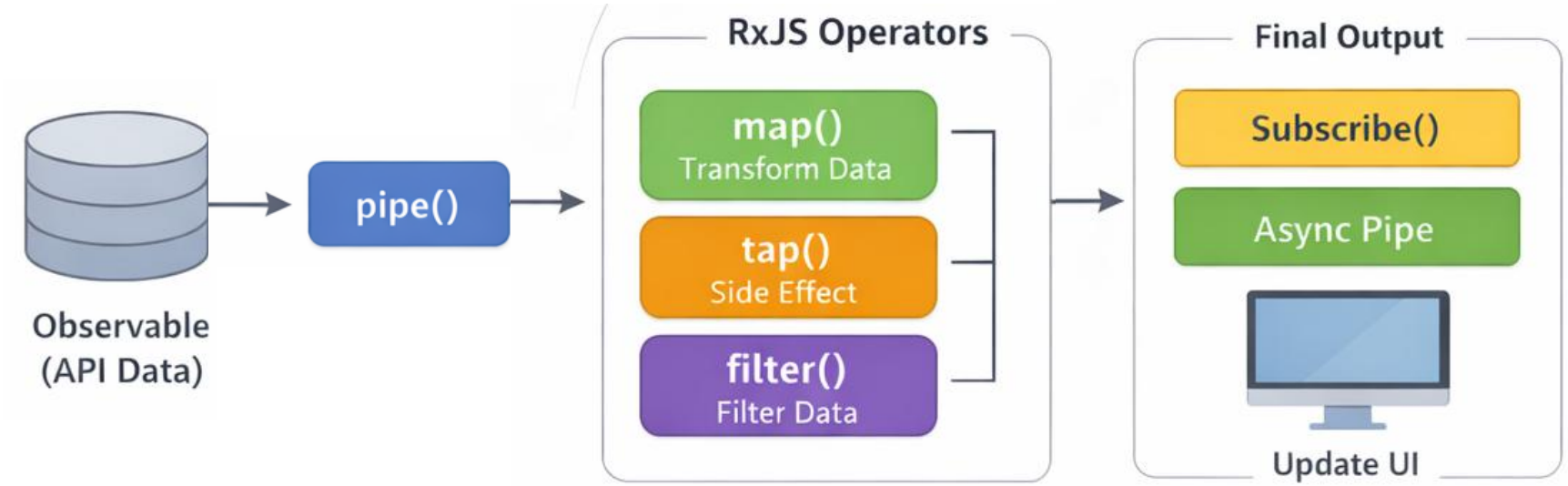
- 👉 RxJS Operators Introduction
- 👉 map Operator
- 👉 filter Operator
- 👉 tap Operator



Q. What are RxJS Operators?



- In RxJS, operators are functions that act on the values emitted by an Observable to transform(map), filter(filter), or perform side effects(tap), and return a new Observable.



Original Observable Data

```
{name: 'Alice', age:25}
{name: 'Bob', age:17}
{name: 'Carol', age:30}
```

After map() operator

```
{name: 'ALICE', age:25}
{name: 'BOB', age:17}
{name: 'CAROL', age:30}
```

Q. What is RxJS map Operator? What is pipe function?



- **map operator is an RxJS operator** that transforms the data emitted by an Observable and returns a new Observable.

- **Pipe is a method** that lets you apply RxJS operators on an Observable.

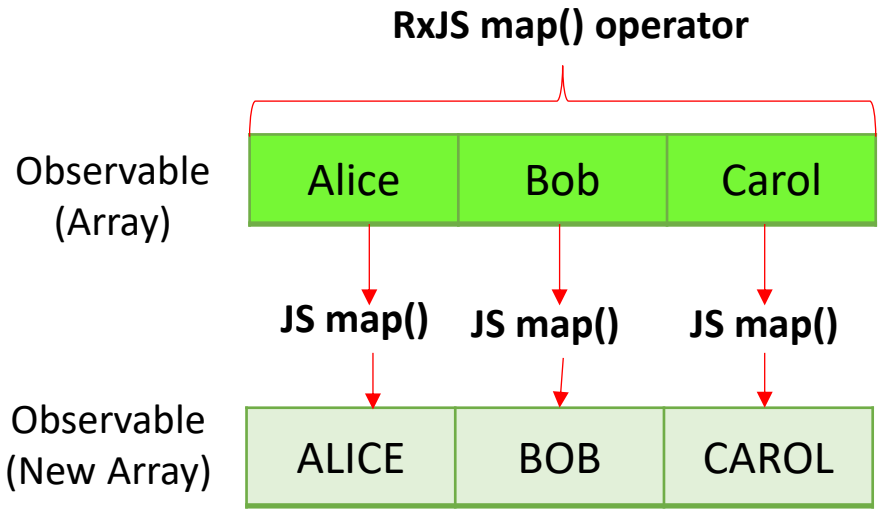
Original Observable Data

```
{name: 'Alice', age:25}
{name: 'Bob', age:17}
{name: 'Carol', age:30}
```

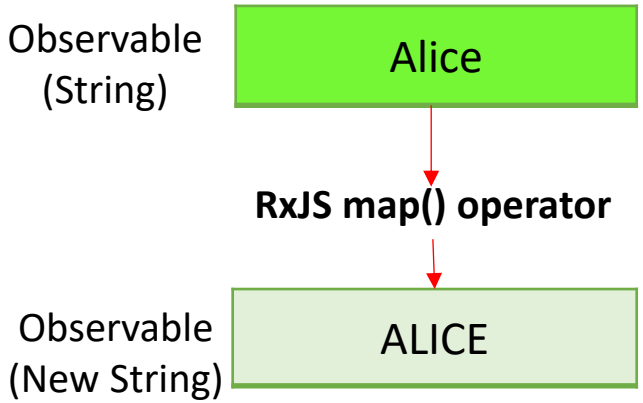
After map() operator & map() function

```
{name: 'ALICE', age:25}
{name: 'BOB', age:17}
{name: 'CAROL', age:30}
```

map() Operator on an Array Emitted by an Observable



map() Operator on a Single Value Emitted by an Observable



Q. How to transform data using RxJS map operator and JS map function?



- **map function is a JS array method** that transforms each element of an array and returns a new array with the transformed values.

- **spread operator (...)** is used to expand elements of an array or properties of an object into individual elements.

```
export class UserService {
  constructor(private http: HttpClient) {}
  getUsers() {
    return
this.http.get<any[]>('https://jsonplaceholder.typicode.co
m/users').pipe(map((users) => users.map((u) => ({
          name: u.name.toUpperCase(),
          ...u,})),
),);
}}
```

Q. What is JS filter function?



- **filter function is a JavaScript array method** that returns a new array containing only those elements that satisfy a given condition.

Original Observable Data

```
{name: 'Alice', age:25}
{name: 'Bob', age:17}
{name: 'Carol', age:30}
{name: 'Dave', age:16}
```

After filter()

```
{name: 'Carol', age:30}
```

```
export class UserService {
  constructor(private http: HttpClient) {}

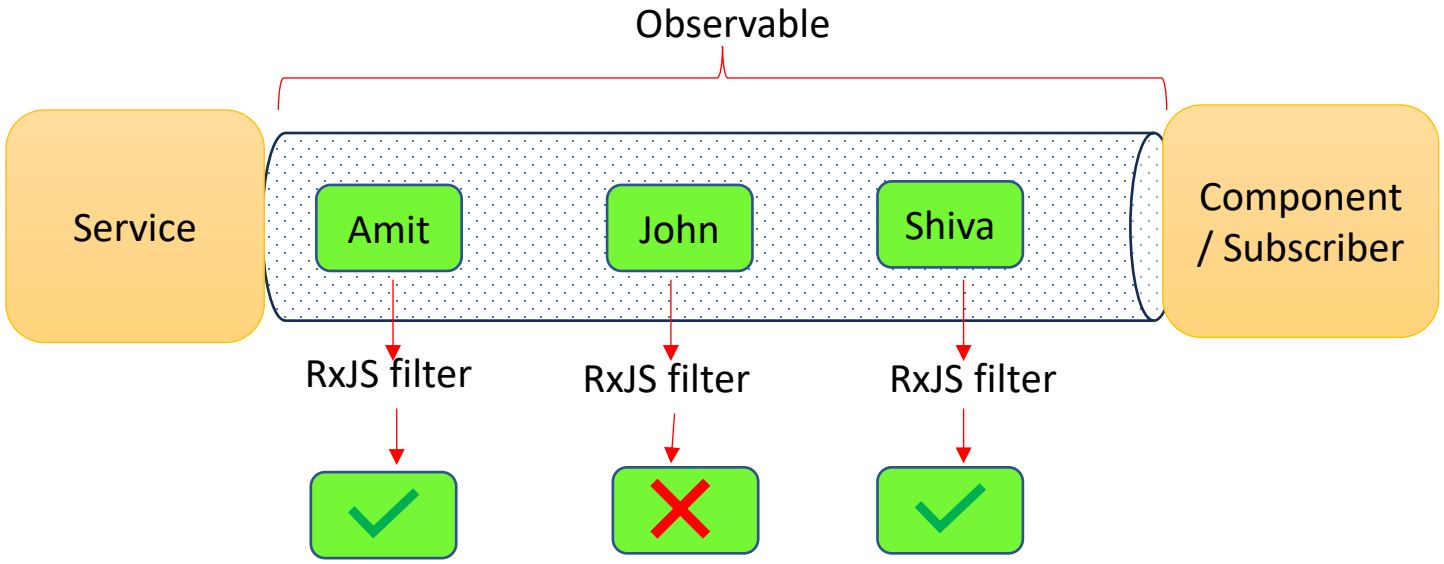
  getUsers() {
    return this.http
      .get<any[]>('https://jsonplaceholder.typicode.com/users')
      .pipe(map((users) => users.filter((u) =>
u.name.startsWith('C'))));
  }
}
```

Q. What is RxJS filter Operator?



- **RxJS filter operator** is used to decide whether the entire value emitted by an Observable should pass or be blocked.

```
export class UserService {
  constructor(private http: HttpClient) {}
  getUsers() {
    return this.http
      .get<any[]>('https://jsonplaceholder.typicode.com/users')
      .pipe(filter((users) => users.length > 0));
}}
```



Q. What is RxJS tap Operator?



- **tap is an RxJS operator** that lets you perform **side actions**, such as logging or debugging, without modifying the data coming from the Observable.

```
export class UserService {
  constructor(private http: HttpClient) {}

  getUsers() {
    return this.http
      .get<any[]>('https://jsonplaceholder.typicode.com/users')
      .pipe(tap((users) => console.log('Users:', users)));
  }
}
```

Main Lifecycle Hooks

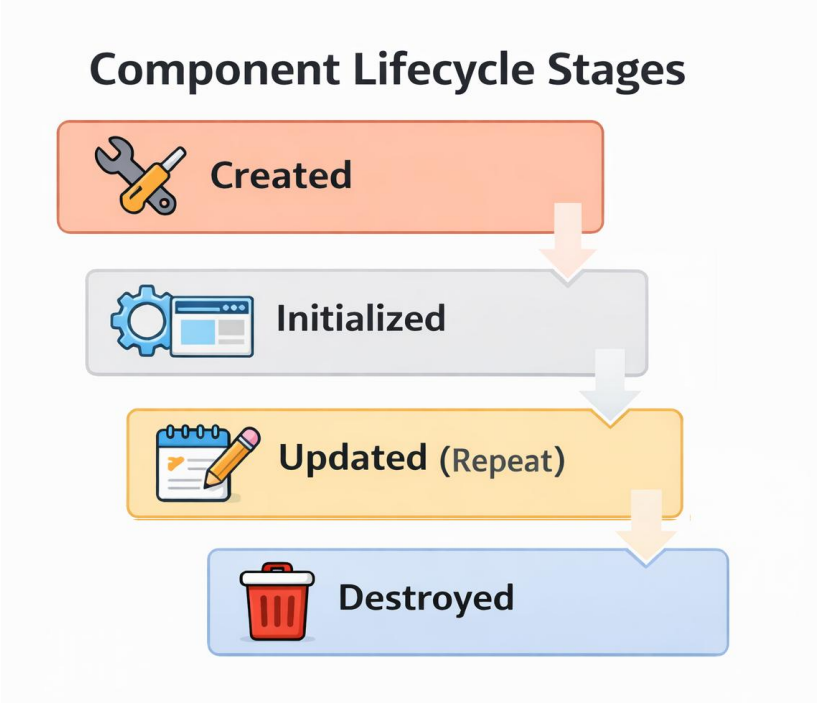
- 👉 Lifecycle Hooks
- 👉 ngOnInit
- 👉 ngOnDestroy



Q. What are Lifecycle Hooks in Angular?



- Lifecycle hooks are predefined methods that Angular calls at different stages of a component's lifecycle.





Q. What is ngOnInit Lifecycle Hook? How is it different from constructor?



- Constructor runs when the component class is created, before ngOnInit() and before the template is fully initialized.
- It is good for injecting services.

```
// component.ts
export class MyComponent {
  users: any[] = [];

  constructor(private service: ObservableService) {
    this.service.getUsers().subscribe({
      next: (data) => {
        this.users = data;
      },
    });
  }
```

- ngOnInit() is an Angular lifecycle hook that is called once after the **component is initialized.**
- Its good for subscribing to Observables, fetching the data from API's etc.

```
// component.ts
export class MyComponent implements OnInit {
  users: any[] = [];
  constructor(private service: ObservableService) {}

  ngOnInit() {// Component HTML is fully iniatialized
    this.service.getUsers().subscribe({
      next: (data) => {
        this.users = data;
      },
    });
  }
}
```

Q. What is ngOnDestroy Lifecycle Hook? When to use it?



- ngOnDestroy() is an Angular lifecycle hook that is called **just before a component is destroyed** (removed from the screen).
- It is used to **clean up resources** before the component is destroyed.

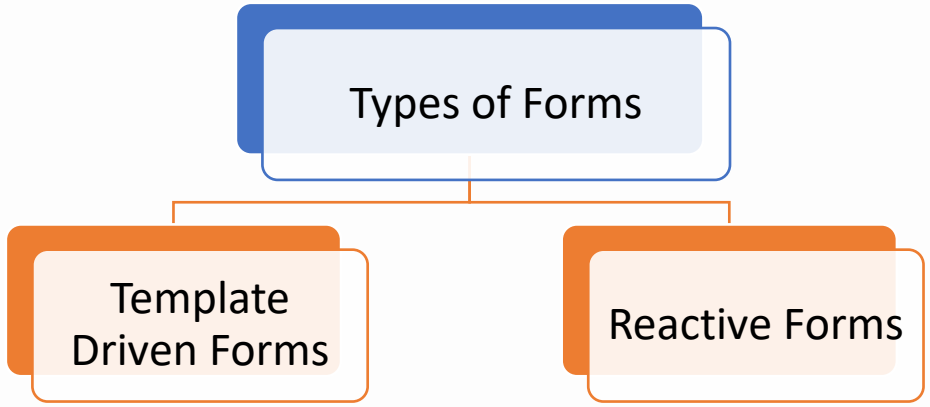
```
// component.ts
export class MyComponent implements OnInit, OnDestroy {
  users: any[] = [];
  subscription!: Subscription;
  constructor(private service: ObservableService) {}

  ngOnInit() {
    this.subscription = this.service.getUsers().subscribe({
      next: (data) => {
        this.users = data;
      },
    });
  }

  ngOnDestroy() {
    this.subscription.unsubscribe();
  }
}
```

Angular Forms - Template Driven

- 👉 Angular Forms Introduction
- 👉 Template Driven Forms
- 👉 ngModel
- 👉 Template Reference Variable
- 👉 ngForm

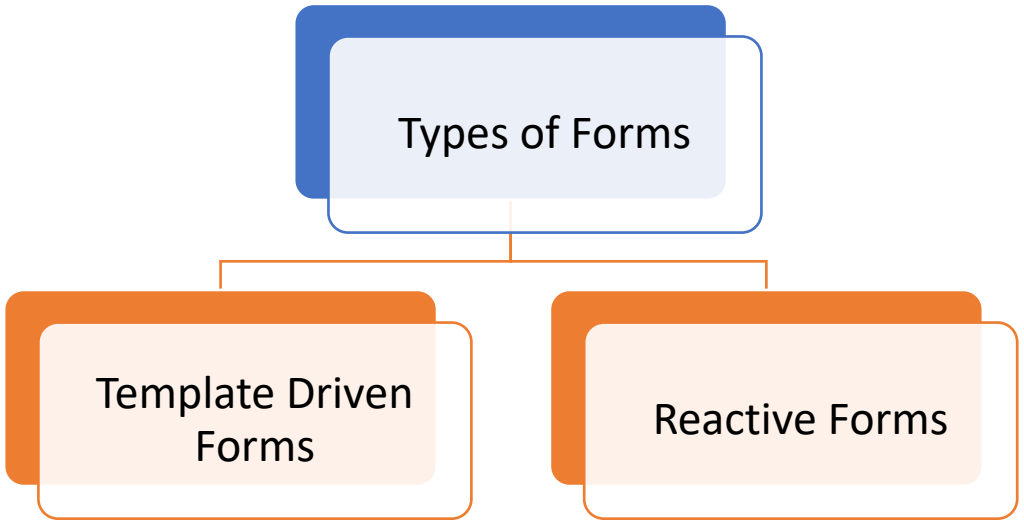


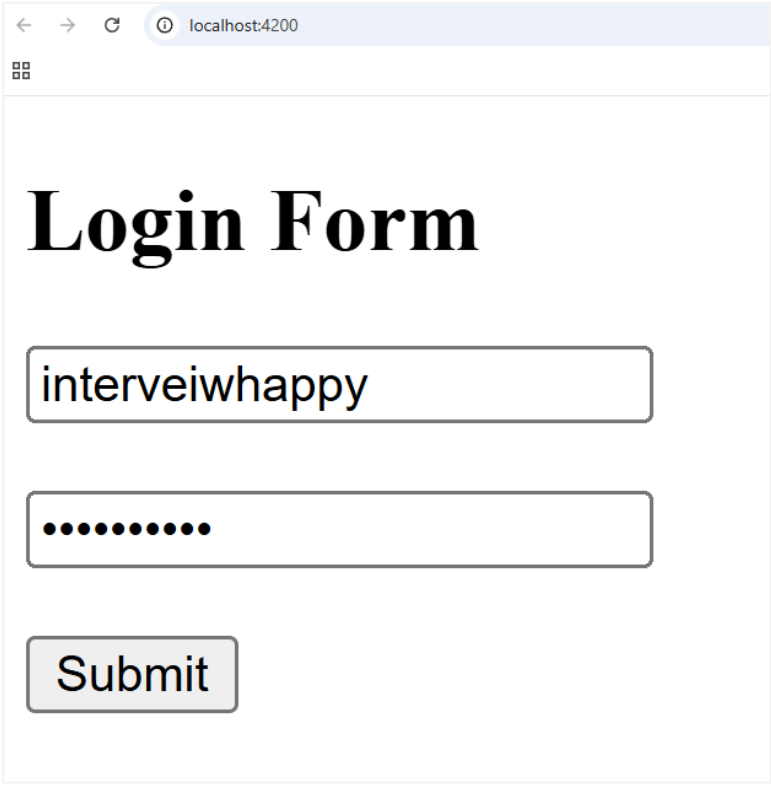


Q. What are Angular Forms?



- Angular Forms are a feature in Angular used to create and **manage forms**.

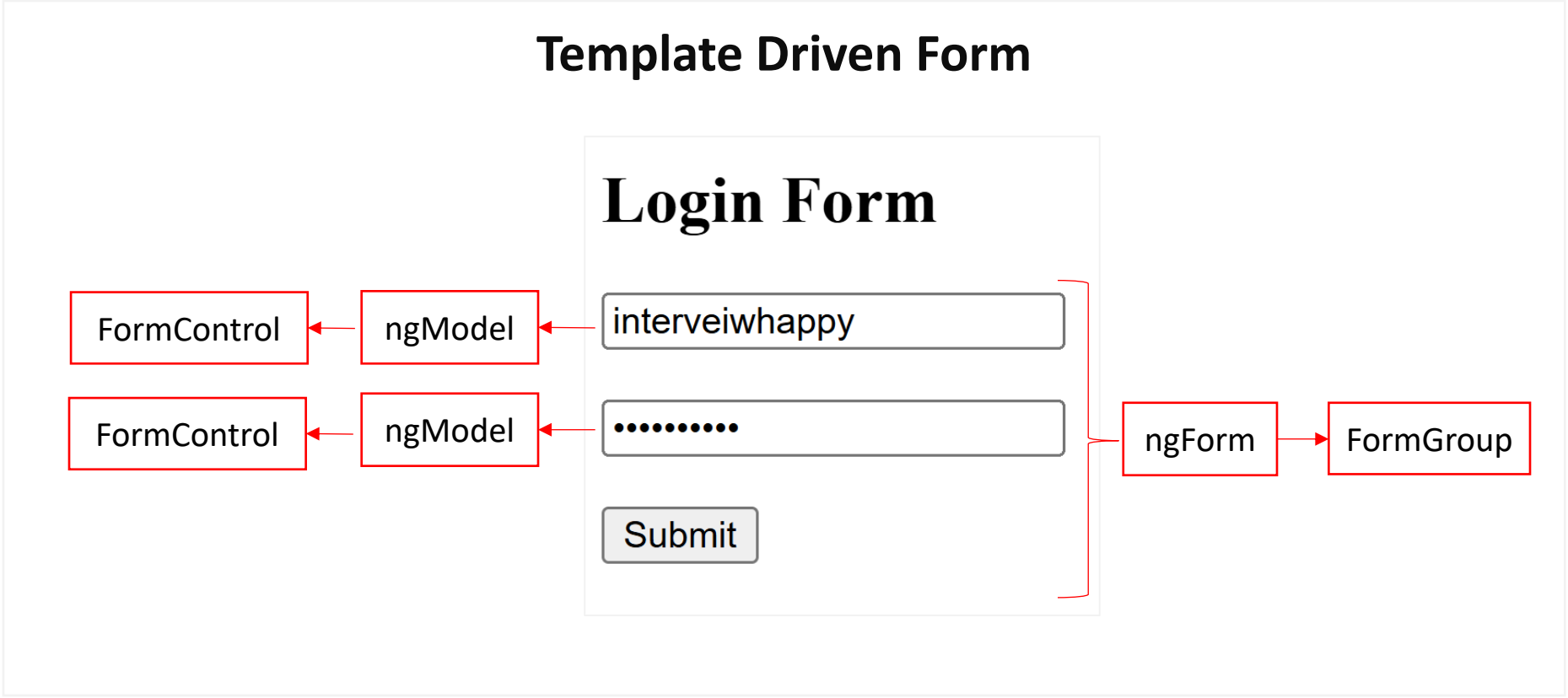




Q. What are Template Driven Forms?



- Template-Driven Forms are Angular forms where most of the form logic and validation are **written in the HTML template** using directives like ngModel and ngForm.

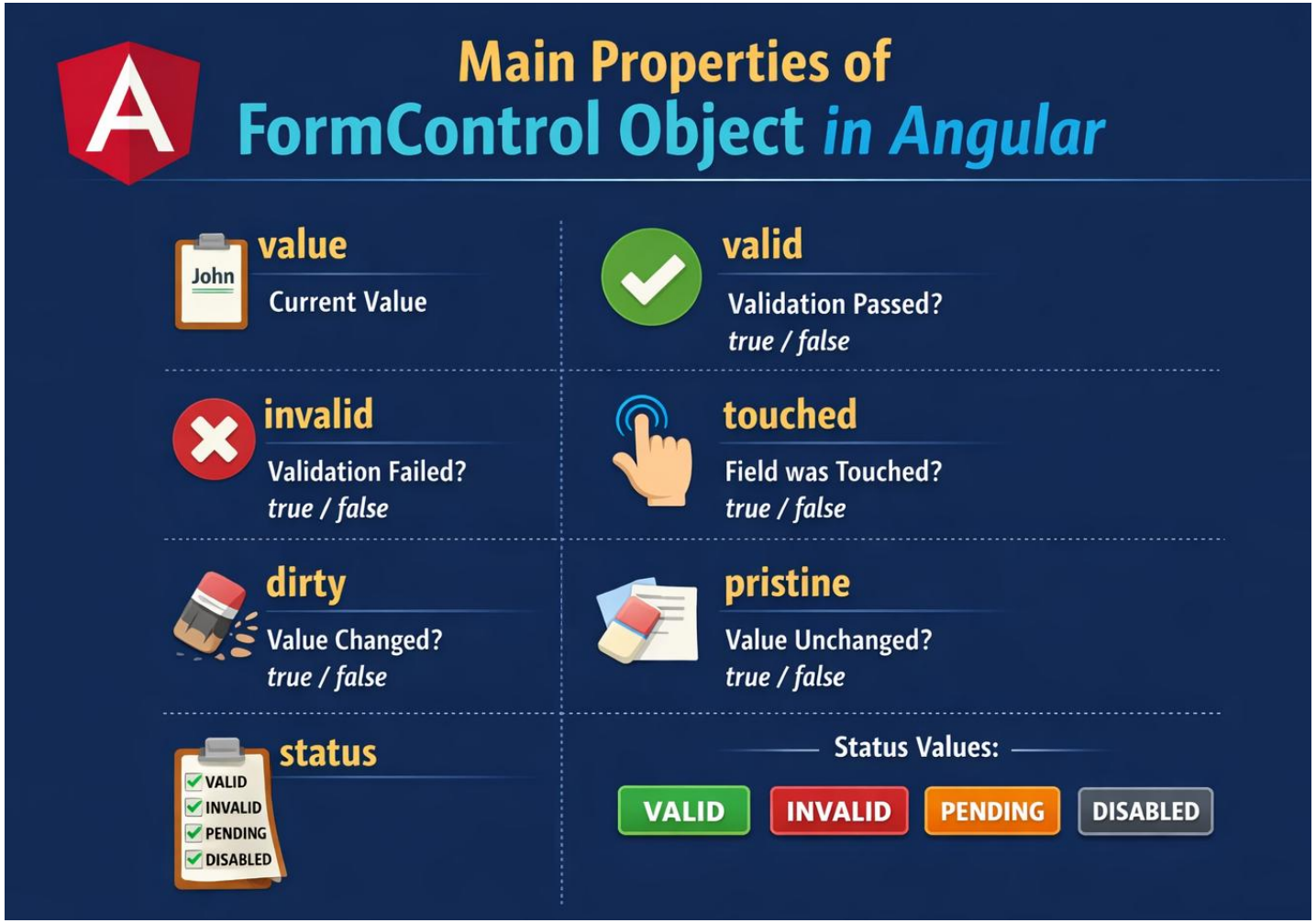


Q. What are ngModel and FormControl?



- ngModel is an Angular directive that converts an HTML input element into an Angular form control and **tracks elements** value and validation state.

- Angular Form Control is **an object** that represents and manages a single input field in a form.



Q. What is Template Reference Variable?



- Template reference variable(# symbol) is used to create a variable in the template to **access an element** or directive.

```
<h2>With Angular Template Form</h2>
<form>
  <input
    type="text"
    name="username"
    ngModel
    minlength="3"
    #username="ngModel"
    placeholder="Enter username">

  <p *ngIf="username.invalid" style="color: red;">
    Username must be at least 3 characters
  </p>
</form>
```

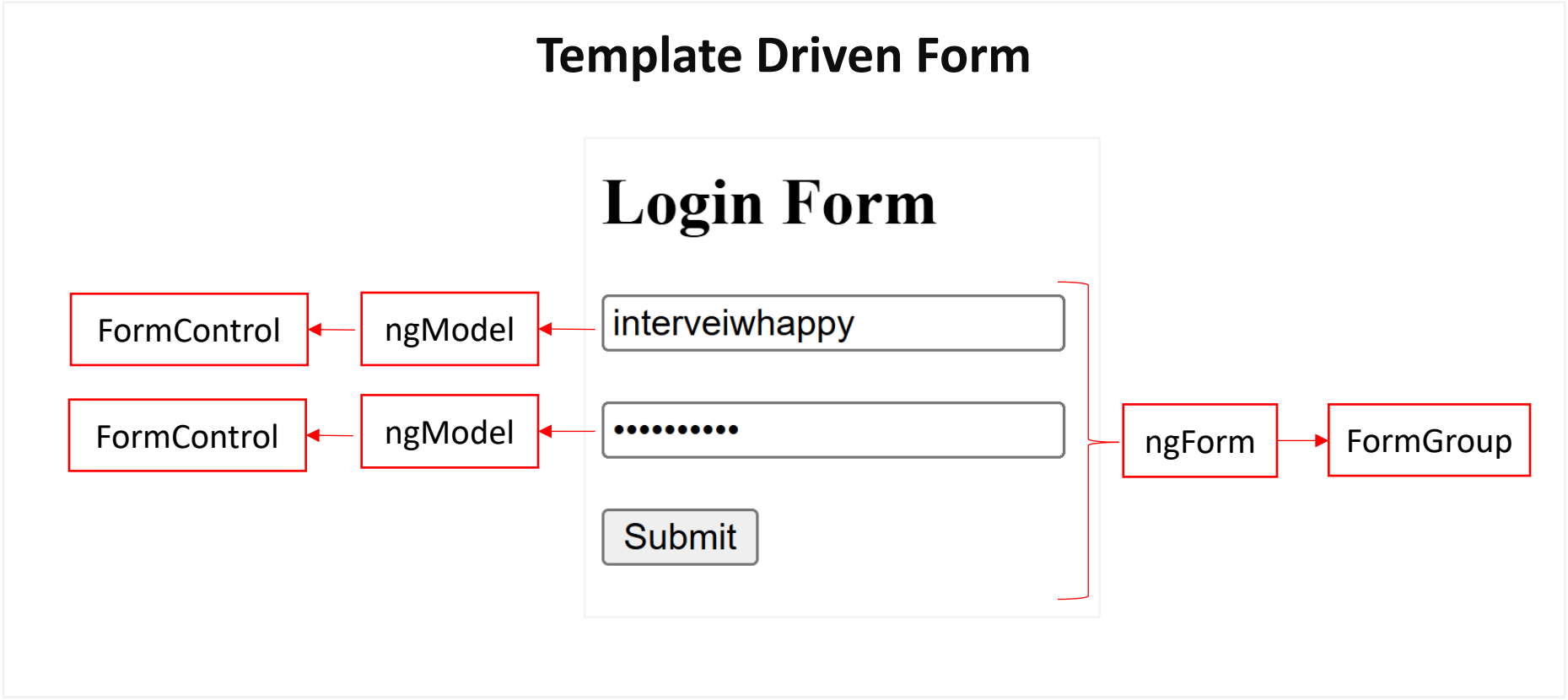
With Angular Template Form

a

Username must be at least 3 characters

Q. What is ngForm and FormGroup?





Q. What is ngForm and FormGroup?



- ngForm is an Angular directive that **manages the entire form** and tracks the value and validation state of all its input fields.

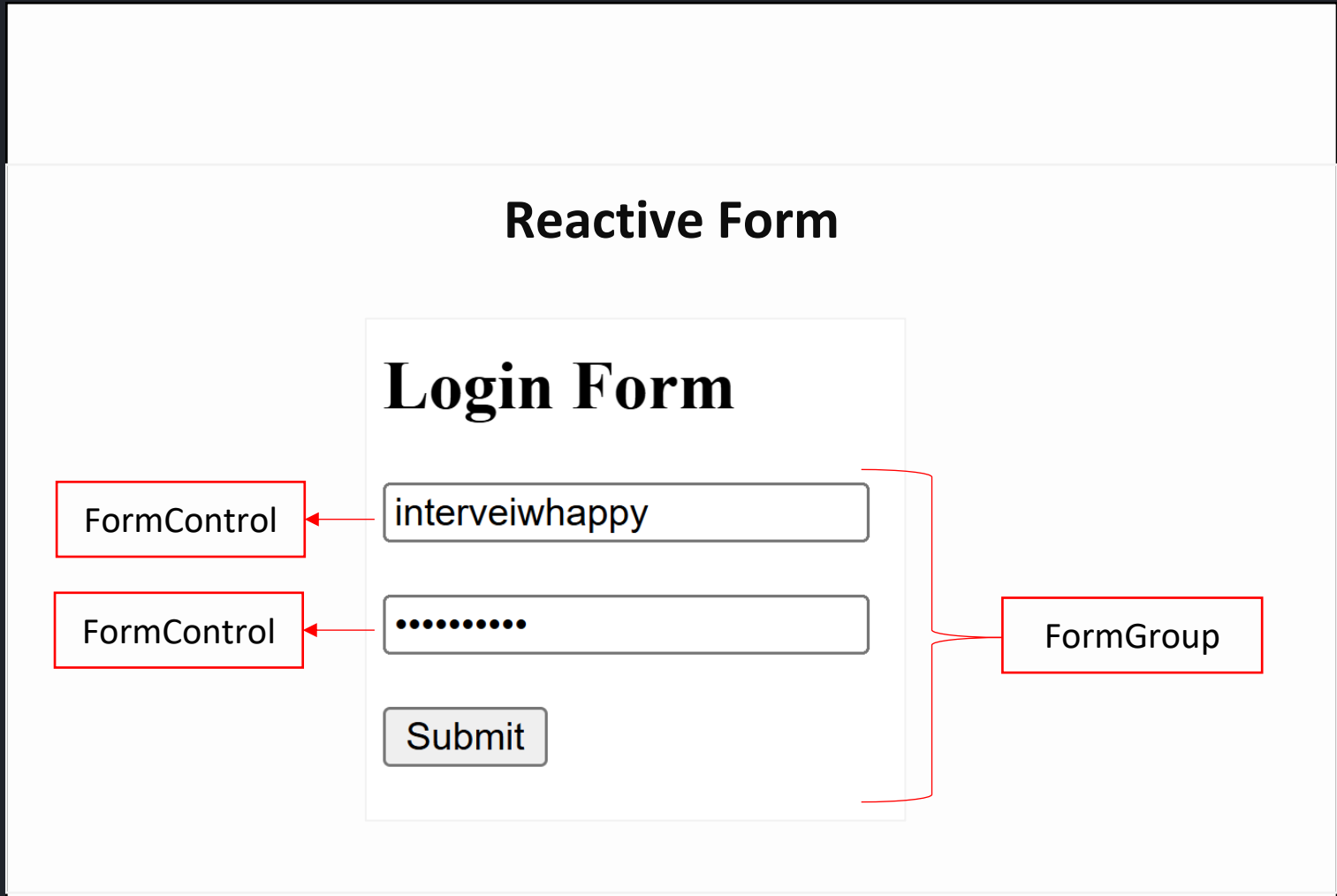
- A FormGroup is an Angular object that **represents the entire form** and manages a collection of FormControl objects.

```
<h2>Angular Template Form</h2>
<form #f="ngForm">
  <input
    type="text"
    name="username"
    ngModel
    minlength="3"
    #username="ngModel"
    placeholder="Enter username">

  <p *ngIf="username.invalid" style="color: red;">
    Username must be at least 3 characters
  </p>
  <p *ngIf="f.invalid" style="color: blue;">
    Form is invalid
  </p>
</form>
```

Angular Forms - Reactive Forms

- 👉 Reactive Forms
- 👉 FormControl & FormGroup
- 👉 Validators
- 👉 formControlName
- 👉 Template vs Reactive Forms



Q. What are Reactive Forms?



- Reactive Forms are Angular forms where the form structure and logic are **defined in the component class** using classes like FormControl and FormGroup.



Q. How to implement Reactive Form using FormGroup and FormControl?



- In Reactive Forms, FormGroup is a class used to create a FormGroup object that **contains multiple FormControl objects** in a single form.

- In Reactive Forms, FormControl is a class used to create a FormControl object that represents and **manages a single form field**.

```ts
// .ts
export class ReactiveForm {

  usernameForm = new FormGroup({

    username: new FormControl(''),

  });

}
```

Q. How to use Validators class in Reactive Forms?



- In Reactive Forms, Validators is a class used **to apply validation rules** on FormControl or FormGroup.

```ts
// .ts
export class ReactiveForm {
  usernameForm = new FormGroup({
    username: new FormControl('', [Validators.minLength(3)]),
  });
}
```

Q. How to bind Reactive Form to HTML using formGroup and formControlName?



- formGroup is an Angular directive used to bind a FormGroup object to an HTML form element.

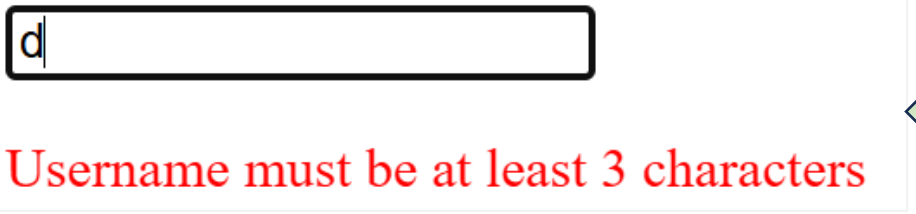
- formControlName is an Angular directive used to bind an input element to a specific FormControl inside a FormGroup using a string key.

```ts
// .ts
export class ReactiveForm {
  usernameForm = new FormGroup({
    username: new FormControl('', [Validators.minLength(3)]),
  });
}
```

```html
// .html
<form [formGroup]="usernameForm">

  <input type="text"
    formControlName="username" placeholder="Enter username">

  <p *ngIf="usernameForm.get('username')?.invalid"
style="color: red;">
    Username must be at least 3 characters
  </p>
</form>
```



Q. How to simplify FormControl access using a getter in Reactive Forms?



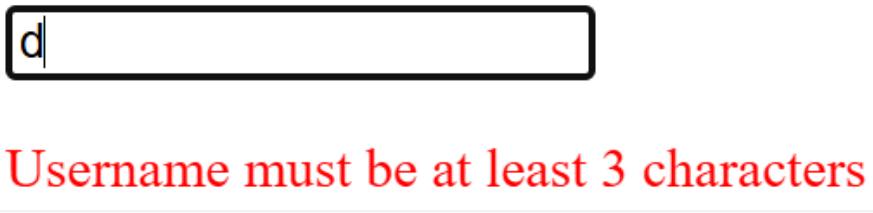
- In Reactive Forms, A getter is used to **easily access FormControl from FormGroup in template**

```ts
// .ts
export class ReactiveForm {
  usernameForm = new FormGroup({
    username: new FormControl('', [Validators.minLength(3)]),
  });
  get username() {
    return this.usernameForm.get('username');
  }
}
```

```html
// .html
<form [formGroup]="usernameForm">

  <input type="text"
    formControlName="username" placeholder="Enter username">

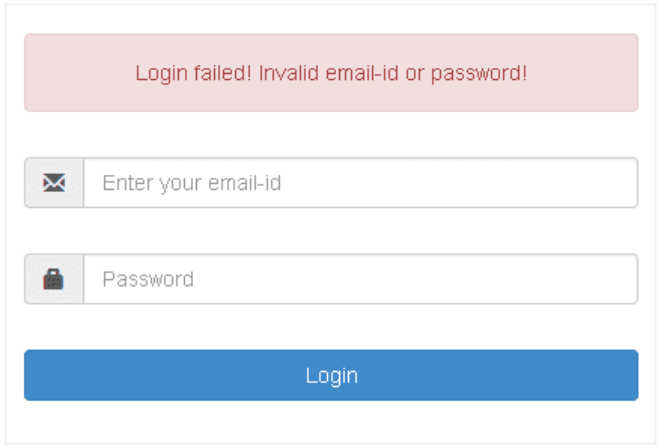
  <p *ngIf="username?.invalid" style="color: red;">
    Username must be at least 3 characters
  </p>
</form>
```



Q. When to use Template driven forms and when to use Reactive forms?



- Use Template Forms when the form is **small, simple**, and not very dynamic.
- For example, Login form, Contact form, Simple feedback form, Simple registration form with 2–5 input fields.

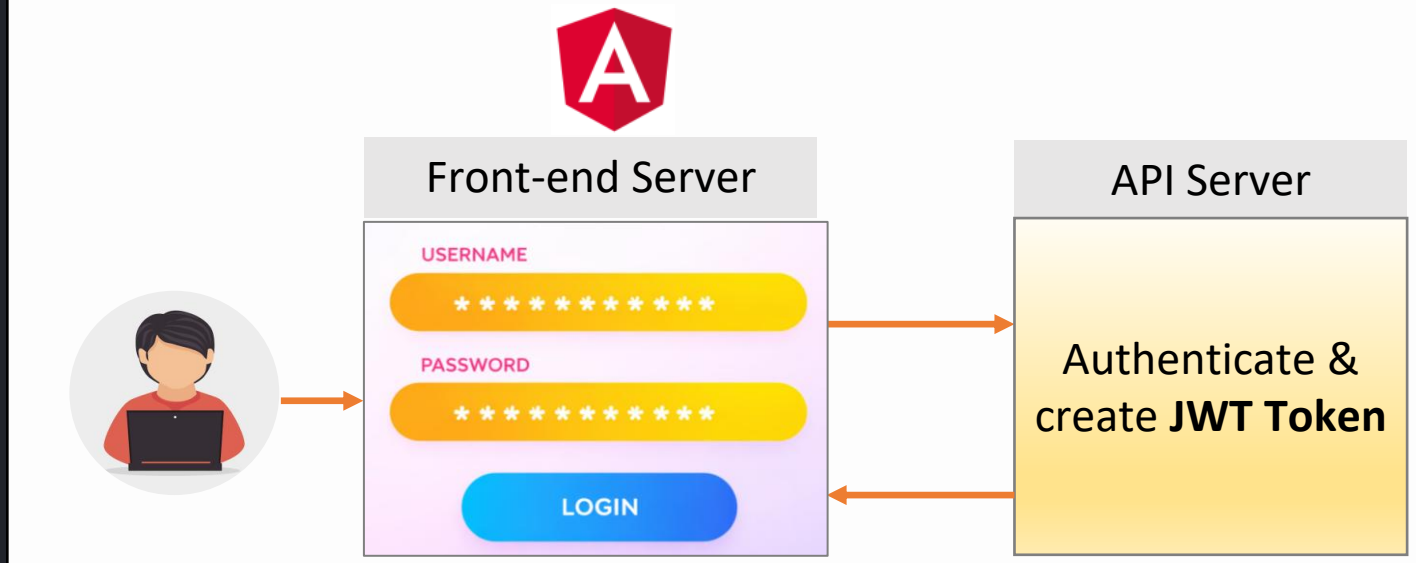


- Use Reactive Forms when the form is big, complex and dynamic.
- For example, complex registration forms, multi-step signup forms, dynamic forms (add/remove fields).



JWT Authentication

- 👉 Authentication vs Authorization
- 👉 Types of Authentication
- 👉 JWT Authentication

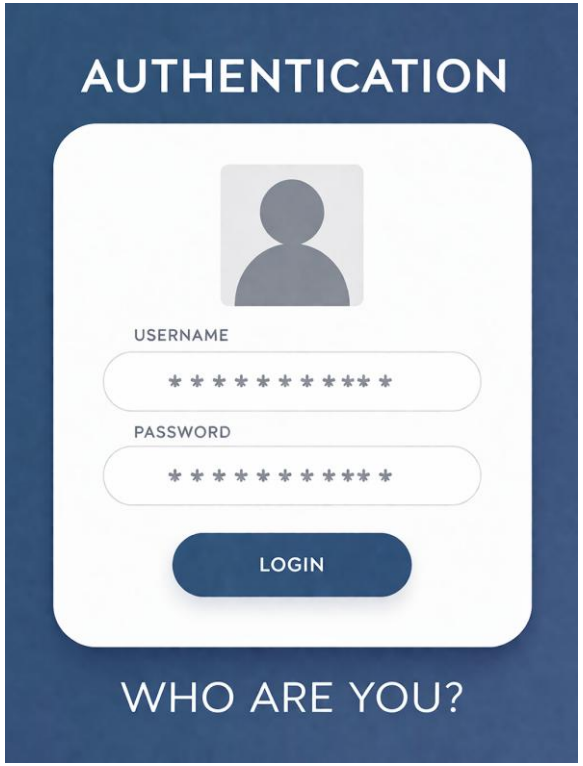


Q. What is the difference between Authentication and Authorization?

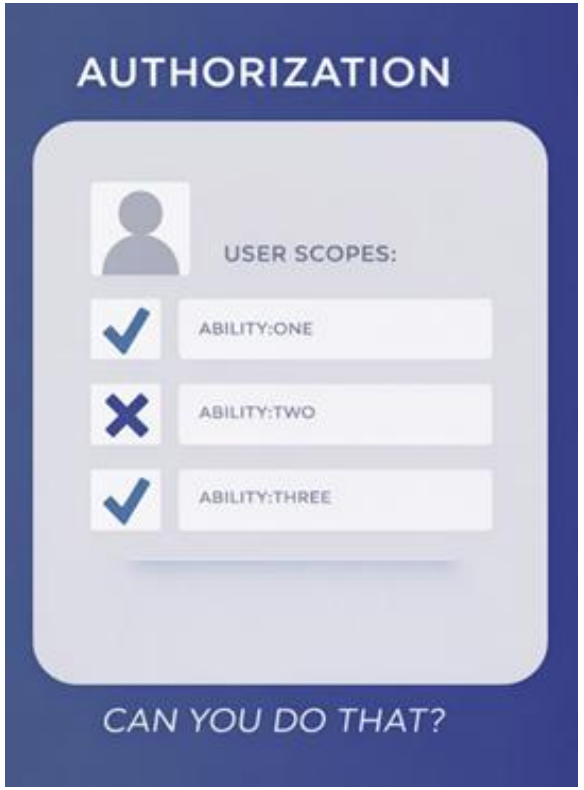


- Authentication is the process to verify the **user's identity**.

- Authorization is the process of allowing a user to **access resources as per the role**.

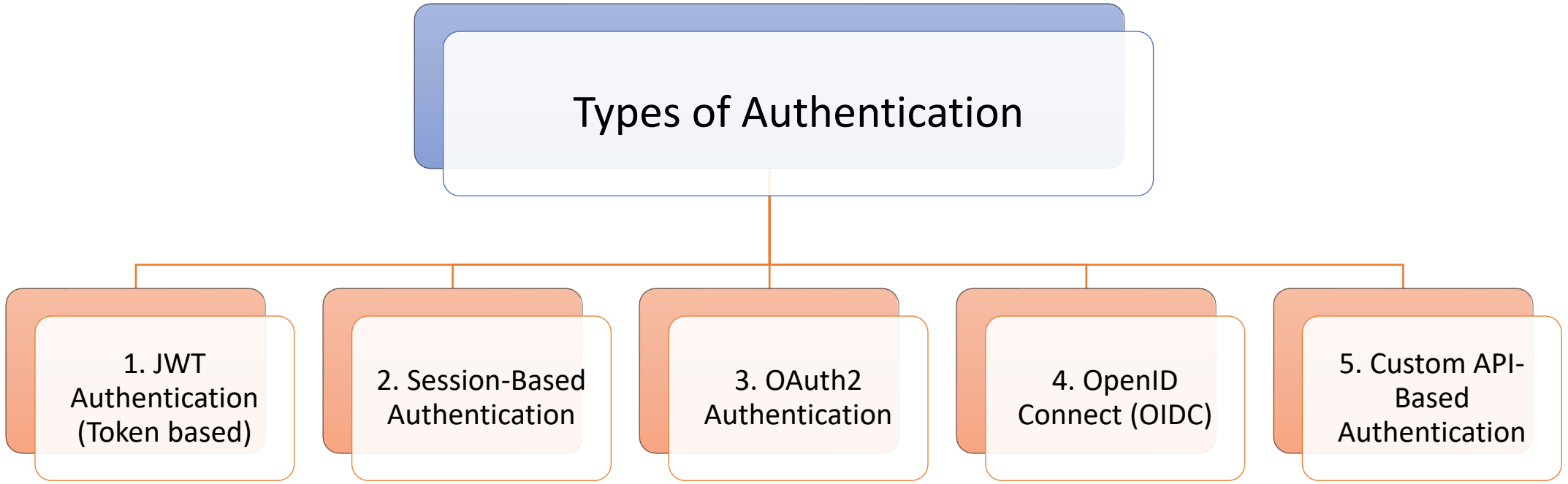


After



Q. What Types of Authentications can be used in Angular?





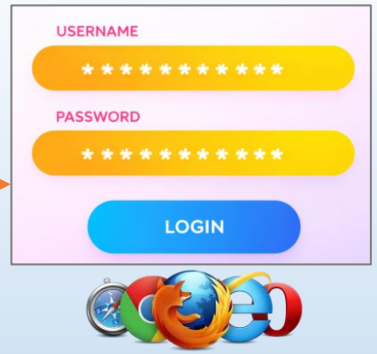
Q. What is JWT (Token based) Authentication? V. IMP.



Front-end/ Client-side

Middleware/ Server-side/ Backend





1. POST: {username, password}



2. Authenticate & create JWT **Token**

3. Return Response {JWT token}

4. Store JWT token at local storage & User Login

5. Request Data {JWT token: Header}

6. Validate token signature

7. Send Data

8. Display data on browser

Q. What is JWT (Token based) Authentication? V. IMP.

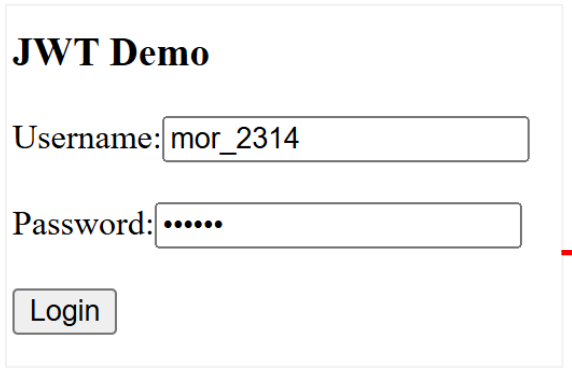


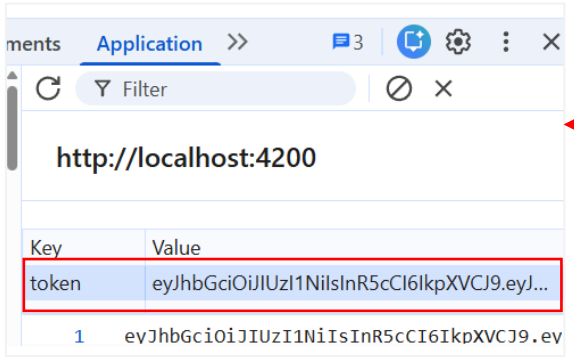
- JWT Authentication is a type of authentication and login method where the server issues a signed token after verifying credentials, and the client uses that token for all future authorized API requests.

Q. How to implement JWT Authentication?



JWT Authentication Implementation in Angular





username, password

Component

1. Login Form (UI)
2. Capture credential
3. Call service.login(). subscribe()

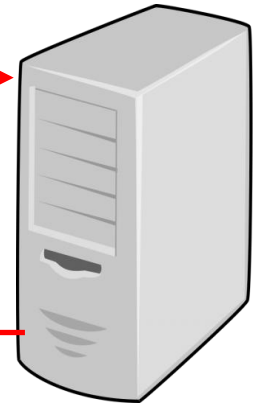
username, password

Service

1. Call API (login request) - http.post()
2. Return Observable
3. Store token

Observable (with Token)

API Server



JSON Response (with Token)

Q. How to set up the Login Component for JWT Authentication?



❖ **Login Component for JWT:**

1. Displays login form (username & password)
2. Captures user input
3. Calls - authService.login(username, password)
4. Subscribes to response (Observable)
5. Handles success/ error: Redirects user (e.g., dashboard) OR shows error message (invalid credentials)

```
// component.html
<h3>JWT Demo</h3>
<form (ngSubmit)="onLogin()">
  <label>Username:</label>
  <input type="text" [(ngModel)]="username" name="username" />
  <br /><br/>
  <label>Password:</label>
  <input type="password" [(ngModel)]="password" name="password" />
  <br /><br />
  <button type="submit">Login</button>
</form>
```

```
// component.ts
export class MyComponent {
  username = '';
  password = '';
  constructor(public auth: AuthService) {}
  onLogin() {
    this.auth.login(this.username, this.password).subscribe();
  }}
```

Q. How to set up the Authentication Service for JWT Authentication?



❖ **Auth Service for JWT:**

1. Sends login request to API → POST /auth/login with username & password
2. Returns Observable to component
3. Stores JWT token (e.g., localStorage)

```
// authService.ts

export class AuthService {

  constructor(private http: HttpClient) {}

  login(username: string, password: string) {

    return this.http
      .post<any>('https://fakestoreapi.com/auth/login', { username,
password })
      .pipe(tap((res) => localStorage.setItem('token', res.token)));

  }
}
```

Q. How do we store the JWT token in the Component? Is it recommended?



- Storing the token in components is not recommended because it causes duplication across multiple components.
- It’s better to store it once in the service using pipe() and tap() for centralized handling.

```
export class MyComponent {
  username = '';
  password = '';

  constructor(public auth: AuthService) {}

  onLogin() {
    this.auth
      .login(this.username, this.password)
      .subscribe((res) => localStorage.setItem('token', res.token));
  }
}
```

Q. How do we store the JWT token in the Service?



- JWT token should be stored in a service because it **centralizes authentication logic** and allows reuse token across multiple components.
- **pipe() method** is used to **connect operators**(tap, map, filter) that process data from an Observable.
- **tap()** is an RxJS operator used to **perform side effects** (like logging or storing data) without changing the original data.

```
// authService.ts
// https://fakestoreapi.com/auth/login
// "username": "mor_2314",
//  "password": "83r5^_"

export class AuthService {

  constructor(private http: HttpClient) {}

  login(username: string, password: string) {

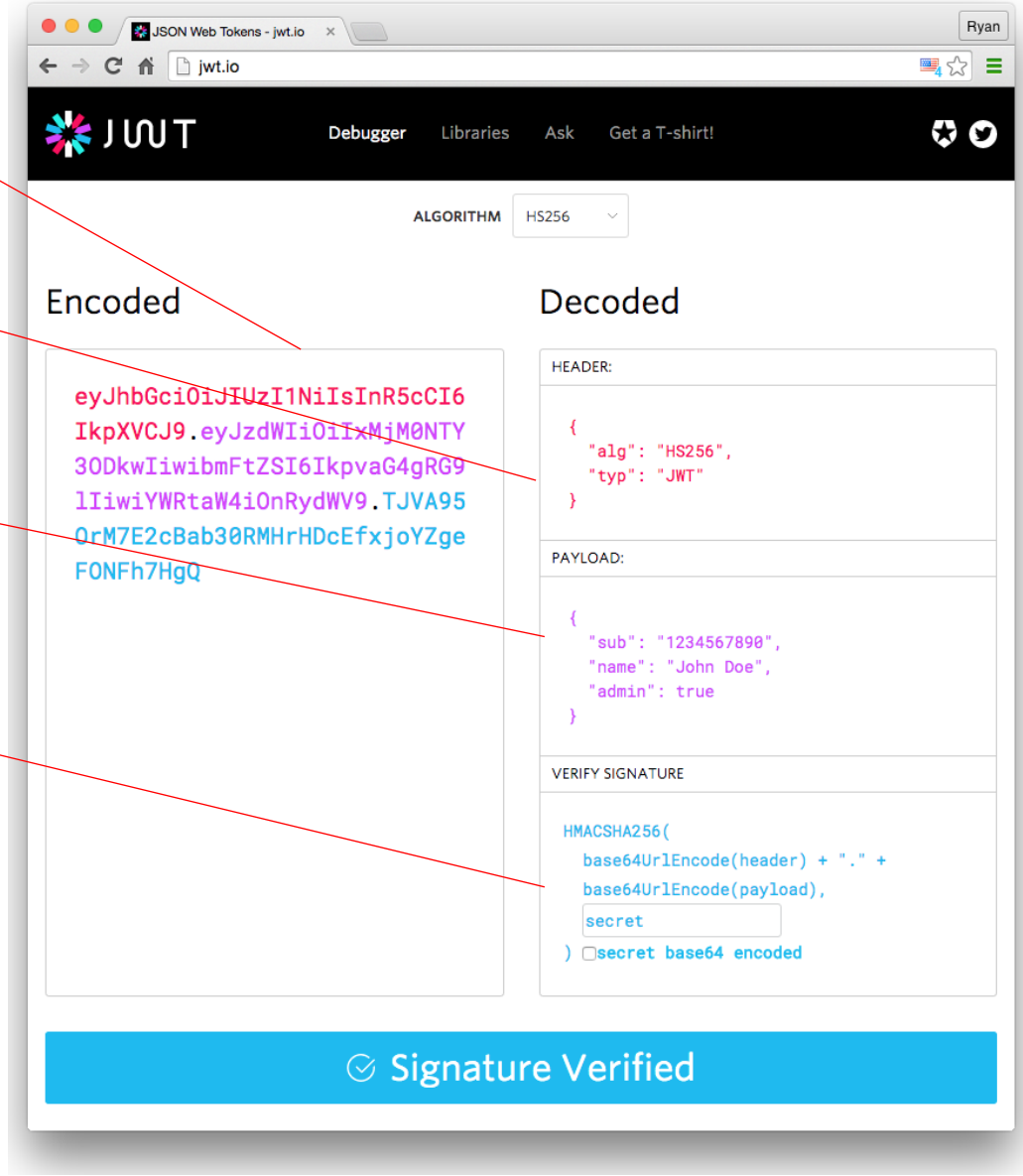
    return this.http
      .post<any>('https://fakestoreapi.com/auth/login', { username,
password })
      .pipe(tap((res) => localStorage.setItem('token', res.token)));

  }
}
```

Q. What are the parts of JWT Token?

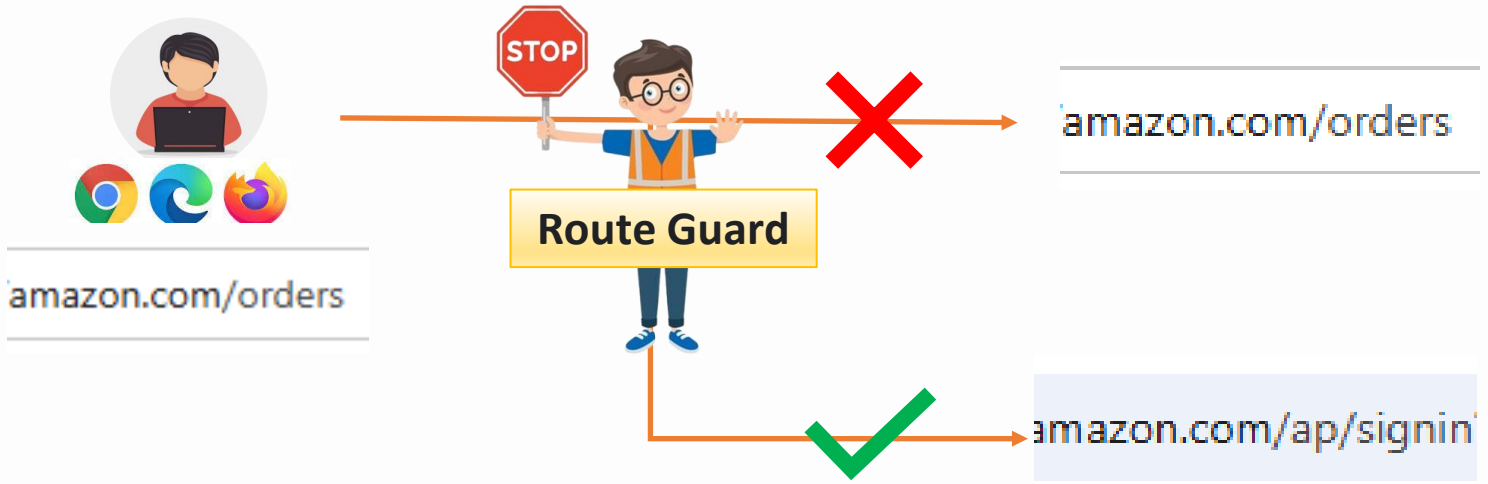


- A JWT (JSON Web Token) is a secure, compact, and URL-safe token used for authentication.
- **Header** contains token type (JWT) and algorithm (HS256, RS256).
- **Payload** contains user data / claims (e.g., userId/ username of logged-in user, roles, expiry).
- **Signature** is created using the header, payload, and a secret key. The server uses it to check that the token is not changed.



Route Guards

- 👉 Route Guards Introduction
- 👉 Types of Route Guards
- 👉 AuthGuard
- 👉 CanActivate
- 👉 CanActivateFn



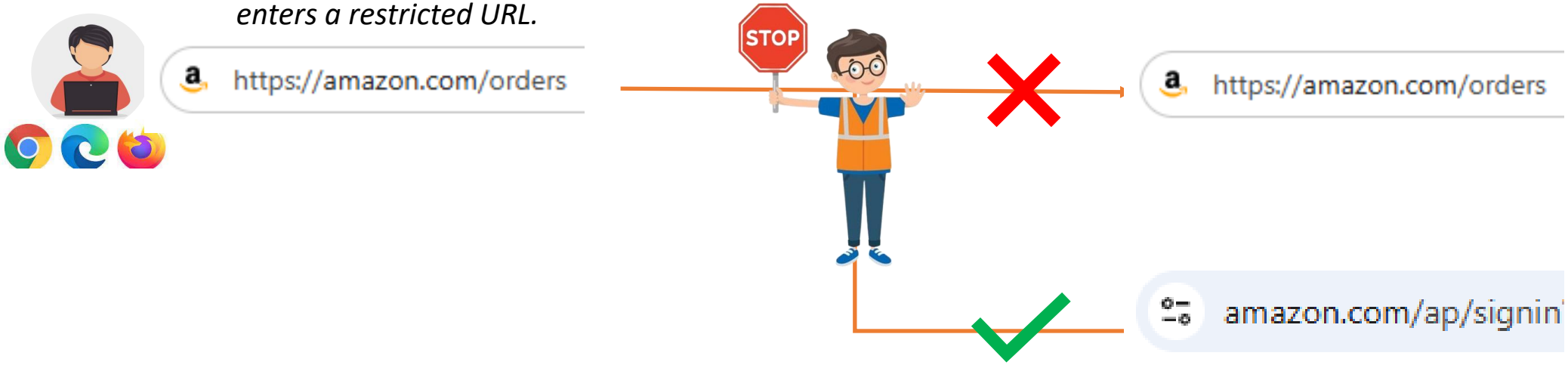
Q. What are Route Guards?



- Route Guards control whether a user can **access a route** or not.

Route Guards

A user without login access manually enters a restricted URL.



Q. What are the Types of Route Guards?

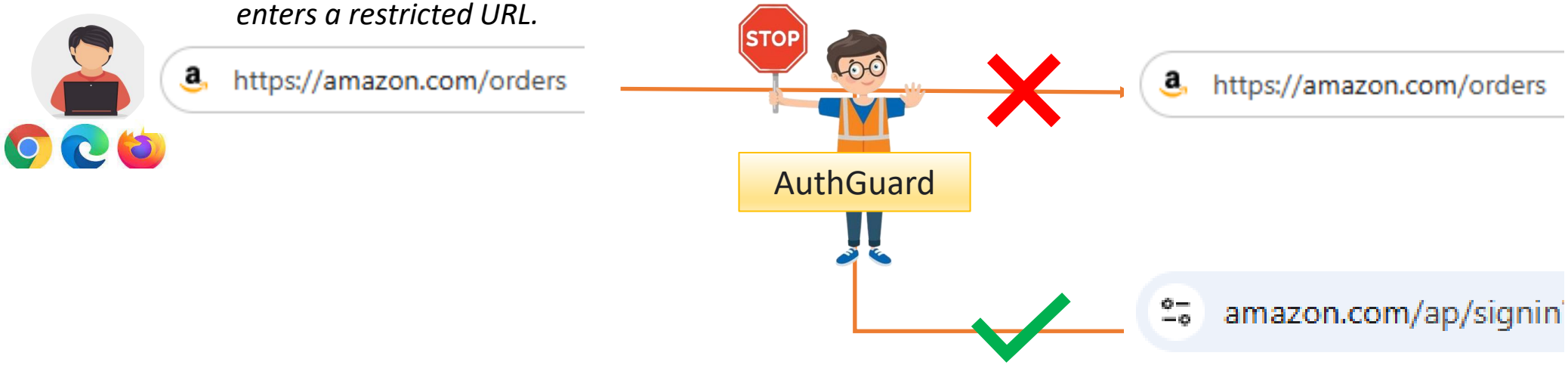


Q. What is AuthGuard? How is it different from Route Guard?



- Route Guard is a general Angular feature used to control navigation between routes, while AuthGuard is a **specific implementation of a Route Guard used for authentication**, typically using **CanActivate**.

A user without login access manually enters a restricted URL.

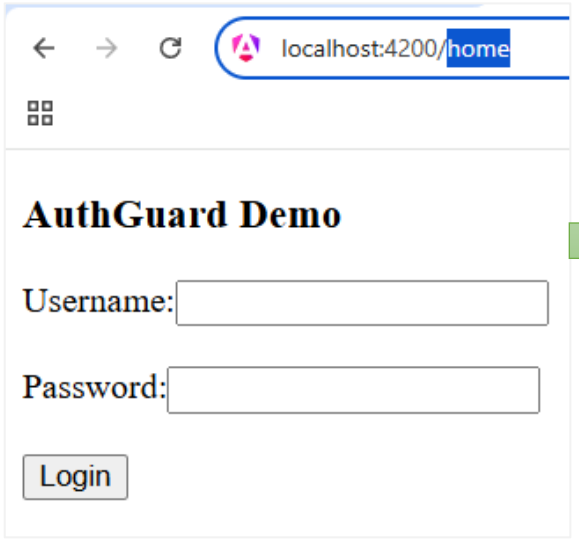


Q. How to implement AuthGuard?



AuthGuard Implementaion

manually entering restricted(home) url in browser



main.ts

bootstrapApplication(App, appConfig)

App.config.ts

provideRouter(routes)

app.routes.ts

{path: 'home', component: Home, **canActivate: [AuthGuard]**,}

auth.guard.ts

Valid token: Allow

No token:

this.router.navigate(['/login']);

Q. How to generate and configure AuthGuard.ts file?



- Command used for generating AuthGuard:
- *Ng generate guard auth*

```
Page reload sent to client(s).
PS D:\Angular-Practical\my-first-app> ng generate guard auth
? Which type of guard would you like to create?
❯◉ CanActivate
 ◯ CanActivateChild
 ◯ CanDeactivate
 ◯ CanMatch
```

Q. How to implement AuthGuard using the CanActivate interface?



- CanActivate interface is an Angular interface used to control whether a user can access (enter) a route or not.

```
// app.routes.ts
export const routes: Routes = [
  { path: '', redirectTo: 'login', pathMatch: 'full' },
  { path: 'login', component: MyComponent },
  { path: 'home', component: Home, canActivate: [AuthGuard] },
];
```

```
// auth-guard.ts
export class AuthGuard implements CanActivate {
  constructor(private router: Router) {}

  canActivate(): boolean {
    const token = localStorage.getItem('token');
    if (token) {
      return true;
    }
    this.router.navigate(['/login']);
    return false;
  }
}
```

Q. How to implement AuthGuard using the CanActivateFn? What is inject function?



- **CanActivateFn** is a functional interface in Angular used to create route guards as functions.

- **Inject()** is a function in Angular used to get instances/ object of services or dependencies inside functions, without using a constructor.

```
// app.routes.ts
export const routes: Routes = [
  { path: '', redirectTo: 'login', pathMatch: 'full' },
  { path: 'login', component: MyComponent },
  { path: 'home', component: Home, canActivate: [authGuard] },
];
```

```
// auth-guard.ts
export const authGuard: CanActivateFn = (route, state) => {

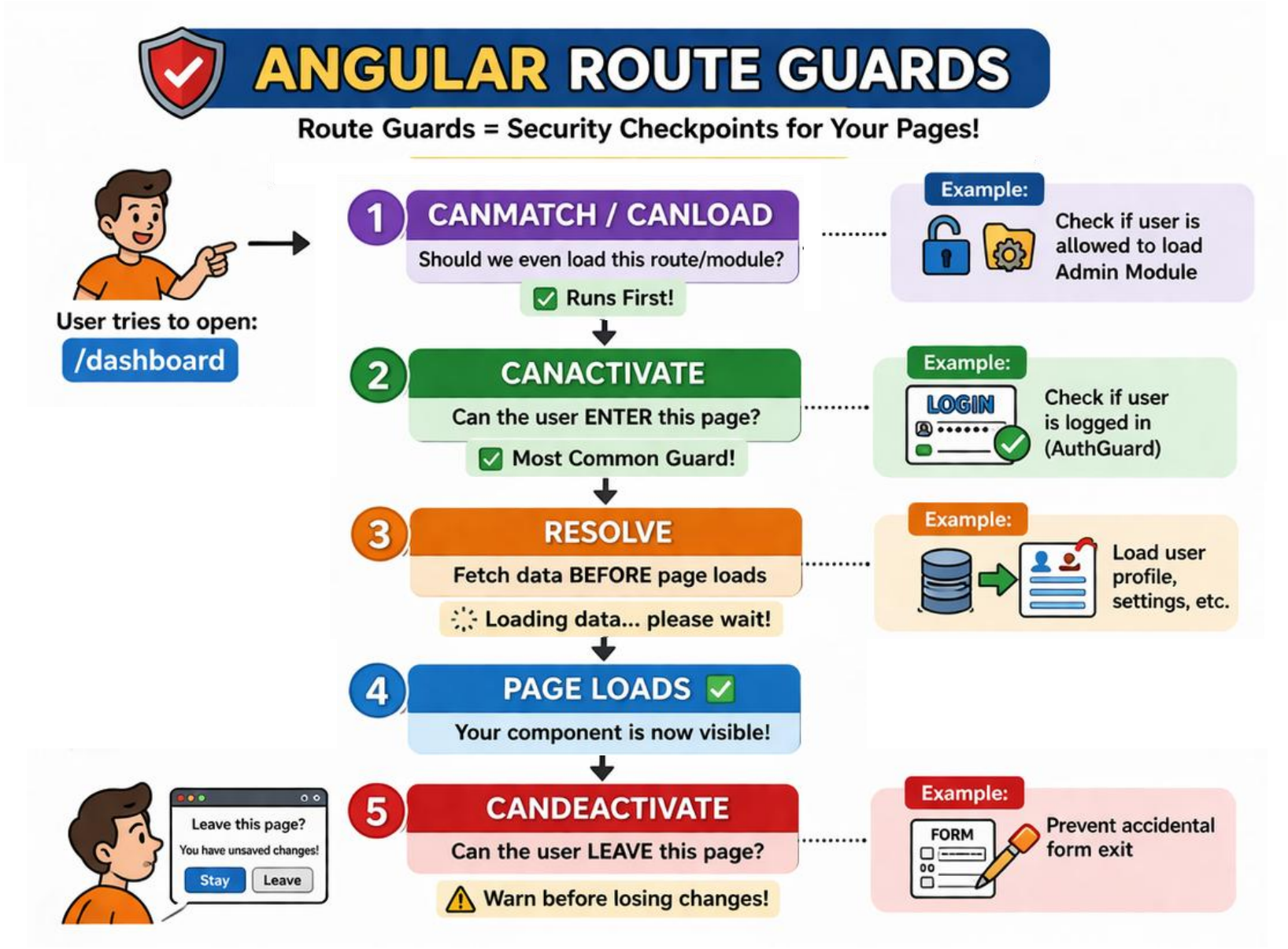
    const router = inject(Router);

    const token = localStorage.getItem('token');

    if (token) {
      return true;
    }
    router.navigate(['/login']);
    return false;
};
```

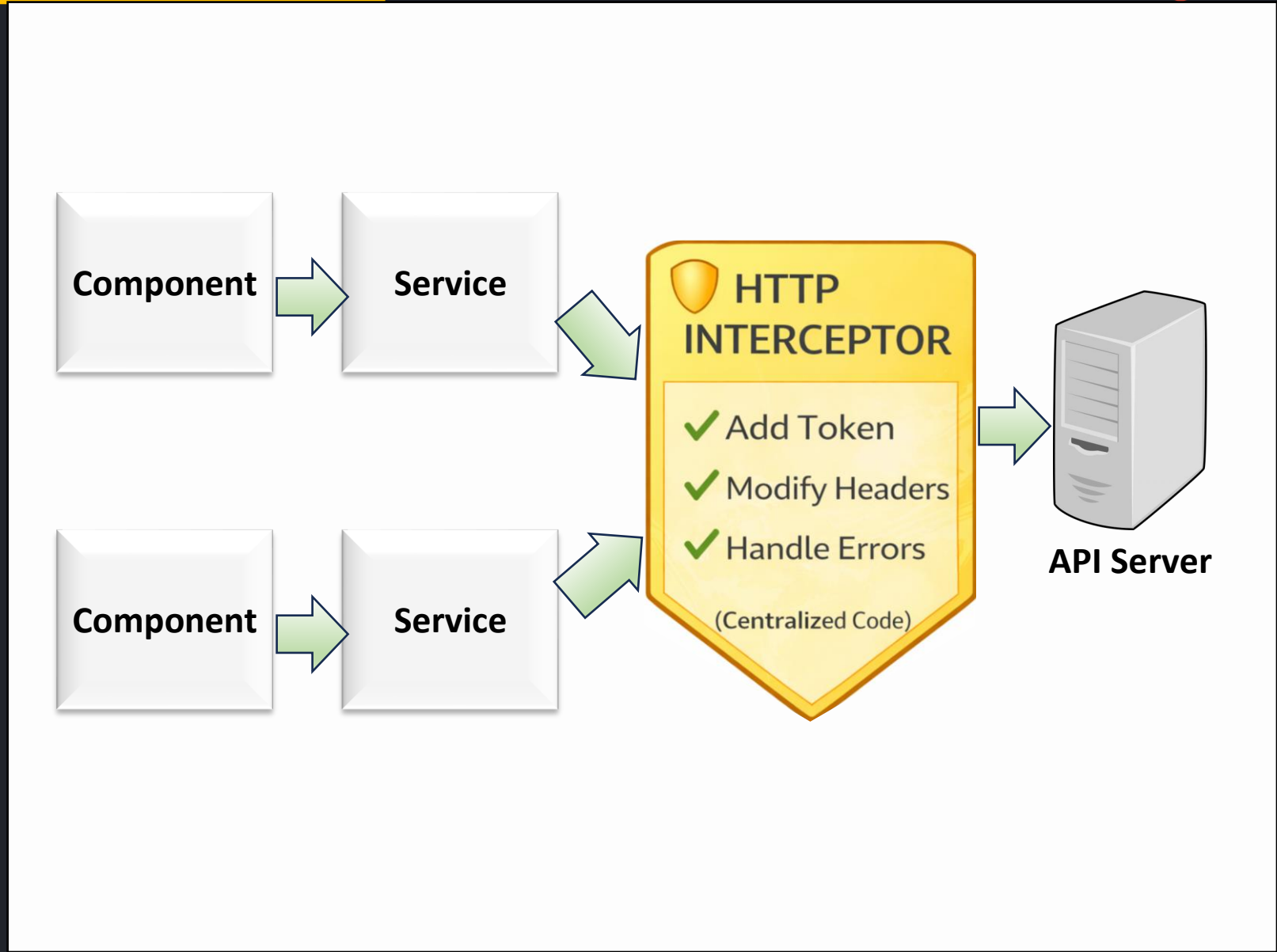
Q. Explain the Types of Route Guards in detail?





HTTP Interceptors

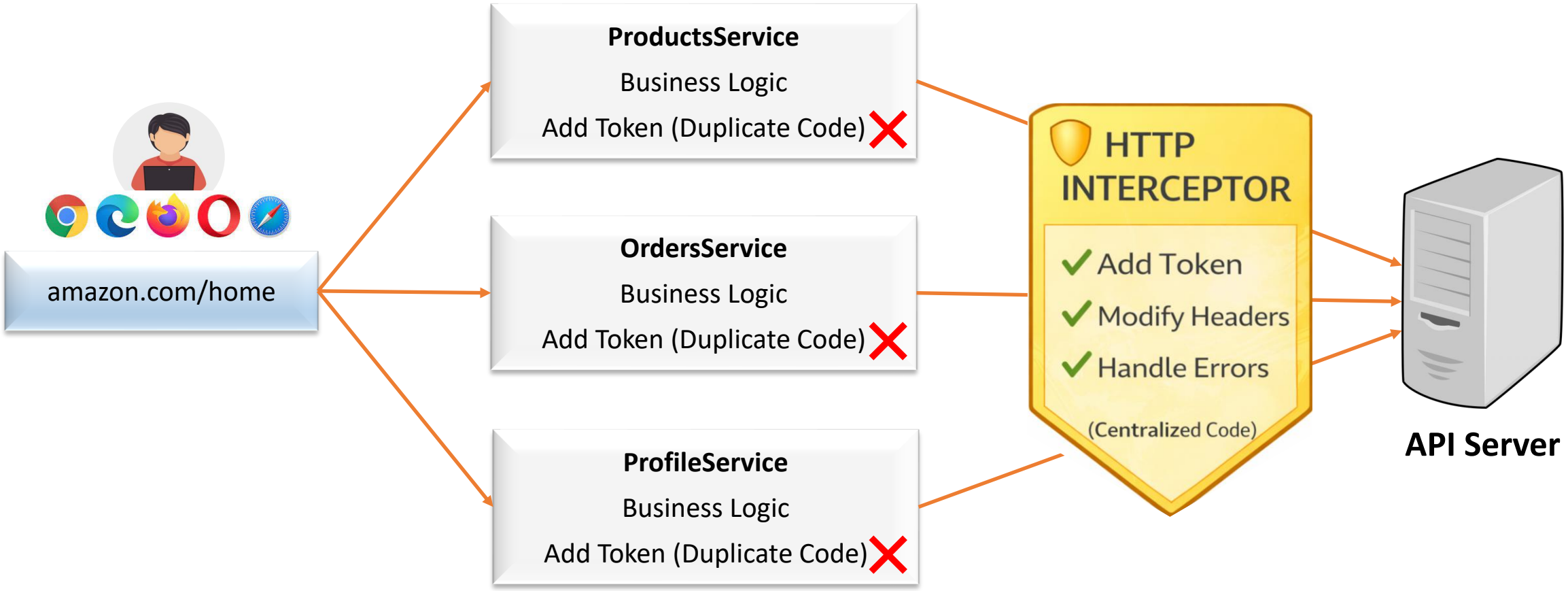
- 👉 Interceptors Introduction
- 👉 Interceptors Implementation



Q. What are HttpInterceptors?



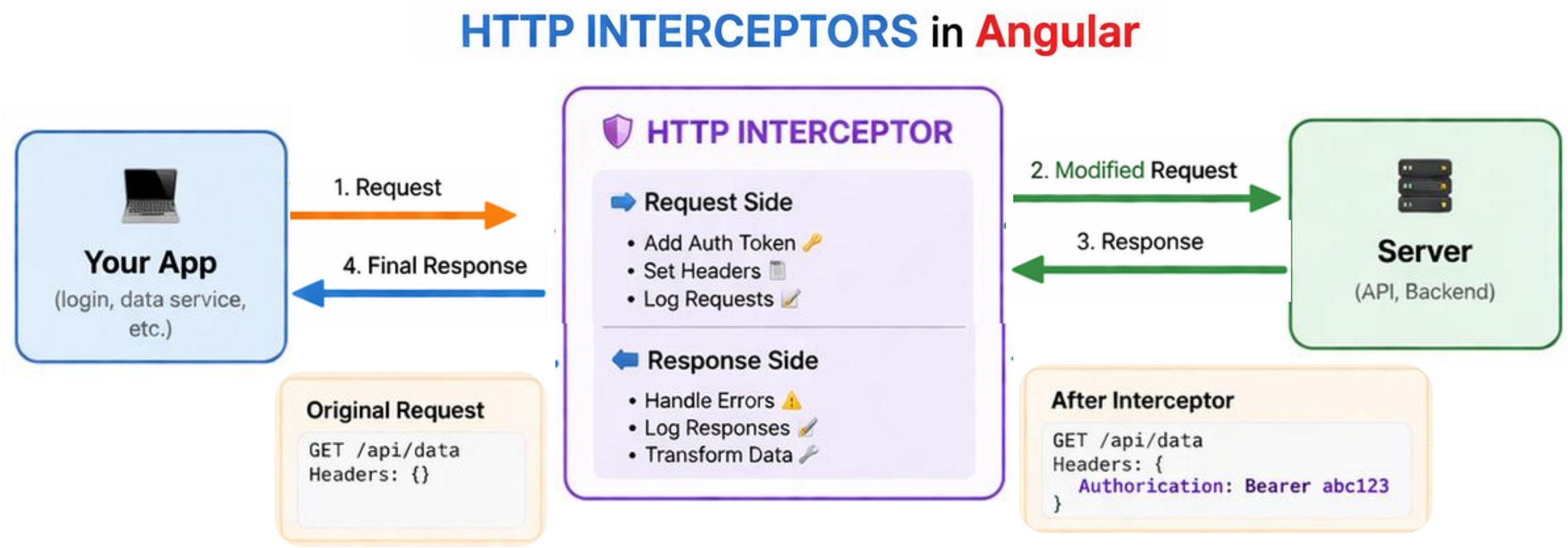
HTTP Interceptor - One Place to Add Token



Q. What are HttpInterceptors?



- HTTP Interceptor is a centralized mechanism in Angular used to **intercept and modify all outgoing HTTP requests and incoming responses**.



Q. How do you call an API from an Angular component to fetch and display the data?



- Steps to fetch data:

1. User clicks the button
2. getData() method is called
3. Angular sends HTTP request to the API
4. Server sends response back
5. Response is received inside subscribe()
6. Data is stored in this.data
7. Angular updates the UI with data

```html
// .html
<h2>Home Page</h2>
<button (click)="getData()">Get Data</button>
<div style="white-space: pre-wrap">
  {{ data | json }}
</div>
```

```ts
// .ts
export class Home {
  token: string = '';
  data: any;
  constructor(private http: HttpClient) {}
  getData() {
    this.http.get('https://fakestoreapi.com/products').subscribe((res) => {
      this.data = res;
    });
  }
}
```

Q. How do you create HttpInterceptor?



❖ **Steps to create HttpInterceptor:**

1. Generate interceptor using ng g interceptor auth
2. Inside interceptor get token from localStorage
3. Clone request using req.clone()
4. Add Authorization: Bearer token in headers
5. Forward request using next(authReq)

```
export const authInterceptor: HttpInterceptorFn = (req, next) => {

  const token = localStorage.getItem('token');

  if (token) {
    const authReq = req.clone({
      setHeaders: {
        Authorization: `Bearer ${token}`,
      },
    });

    return next(authReq);
  }
  return next(req);
};
```

Q. How do you intercept HTTP request before it is sent to the API in Angular?



- withInterceptors is a function in Angular used to **register one or more HTTP interceptors** so that they can intercept and modify all HTTP requests and responses globally.

```
// app.config.ts
export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideRouter(routes),

    provideHttpClient(withInterceptors([authInterceptor])),
  ],
};
```

Signals

- 👉 Signals Introduction
- 👉 Change Detection
- 👉 Signals Implementation

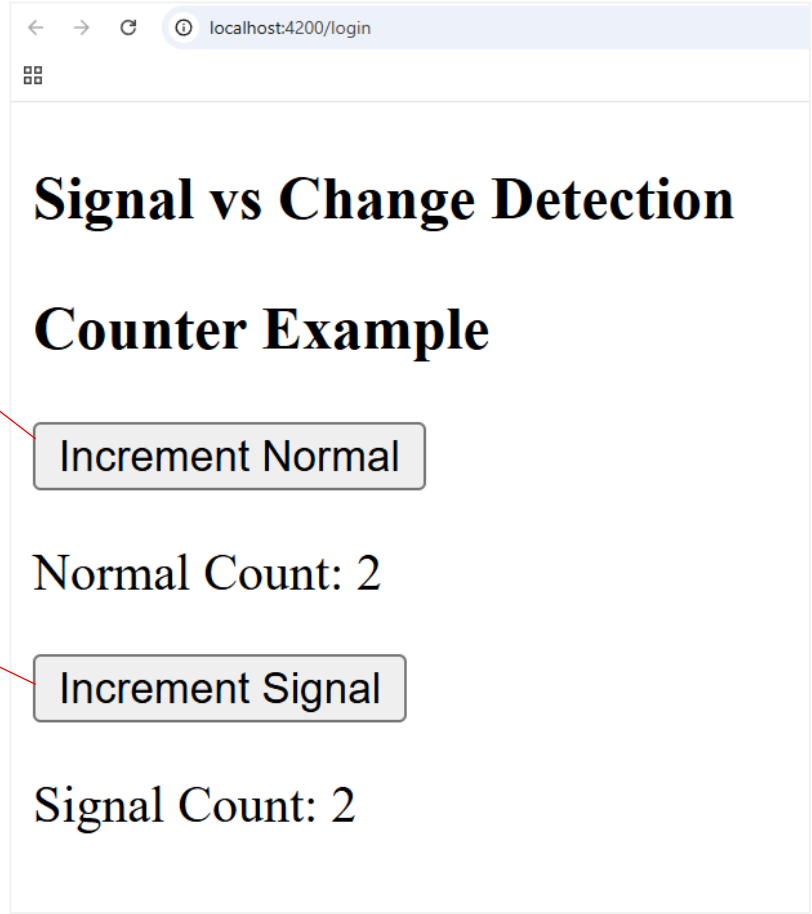


Q. What are Signals? V. IMP.



- Change Detection is the process by which Angular checks the application to detect changes in data and updates the UI accordingly.
- Change detection checks many bindings, even those which are not changed.

- Signal is a reactive value that updates only the required part of the UI automatically.



Q. How to implement using change detection without Signals?



- Change Detection is the process by which Angular checks the application to detect changes in data and updates the UI accordingly.
- Change detection checks many bindings, even those which are not changed.

```html
// withoutsignal.html
<h3>Change Detection Example</h3>
<h3>Counter Example</h3>

<button (click)="increment()">Increment</button>
<p>{{ count }}</p>
```

```ts
// withoutsignal.ts
export class MyComponent {
  count = 0;

  // Normal increment
  increment() {
    this.count++;
  }
}
```

Q. How to implement Signals?



- **signal()** is a function in Angular used to create a reactive value (Signal).It takes an initial value and returns a Signal object.
- **Signal** is a reactive value that updates only the required part of the UI automatically.
- **set()** is a method of a Signal, used to update its value.

```
// withsignal.html
<h3>Signal Example</h3>
<h3>Counter Example</h3>
<button (click)="increment()">Increment</button>
// Reading signal
<p>{{ count() }}</p>
```

```
// withsignal.ts
export class MyComponent {
  // Creating Signal
  count = signal(0);

  // Signal increment
  increment() {
    // Updating signal
    this.count.set(this.count() + 1);
  }
}
```

Angular 20+ Versions

- 👉 Before vs After Angular 20
- 👉 Project Files After Angular 20

Before Angular Version 20

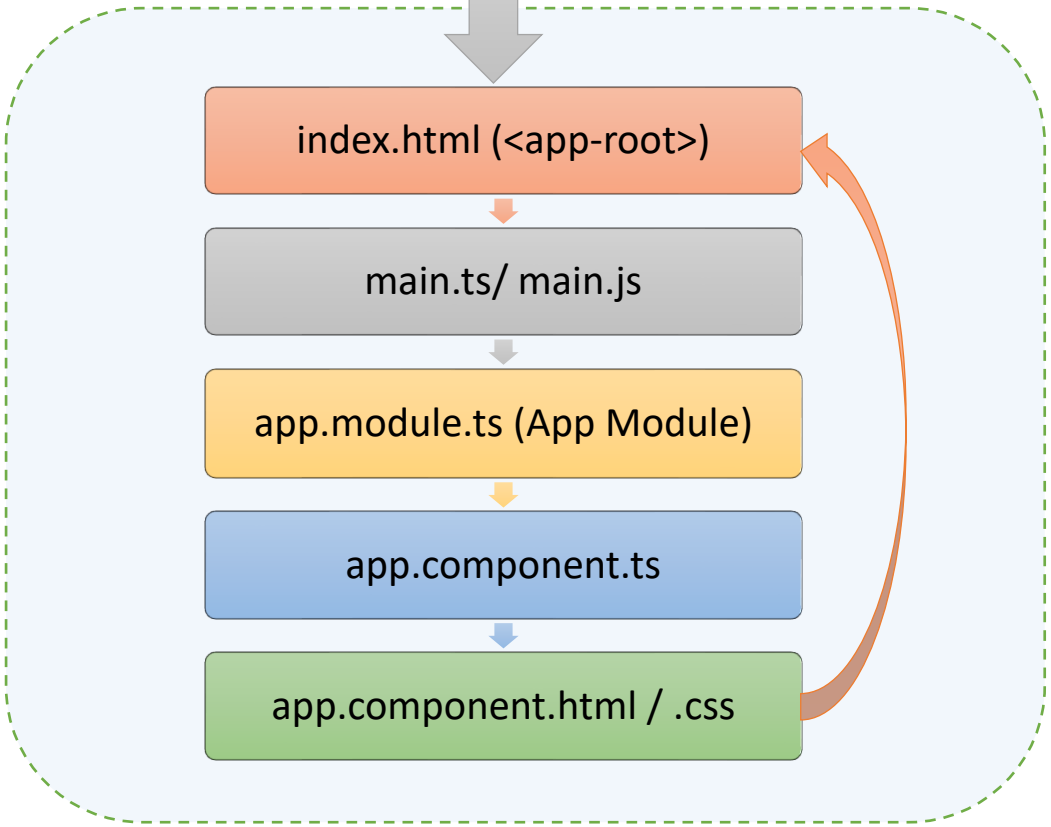
```
my-app/
├── node_modules/          All dependencies
├── package.json           Dependency & script list
├── angular.json           Angular project config
├── tsconfig.json          TypeScript compiler config
├── tsconfig.app.json      App TypeScript config
├── tsconfig.spec.json
├── src/
│   ├── index.html         Main HTML entry point
│   ├── main.ts            Bootstraps AppModule
│   ├── styles.css         Global styles
│   ├── assets/            Static images/assets
│   └── app/
│       ├── app.module.ts          Root module (NgModule)
│       ├── app.component.ts       Root component logic
│       ├── app.component.html     Root component template
│       ├── app.component.css      Root component styles
│       └── app-routing.module.ts  Routing module
```

Q. How does an Angular application get loaded before and after Angular 20?



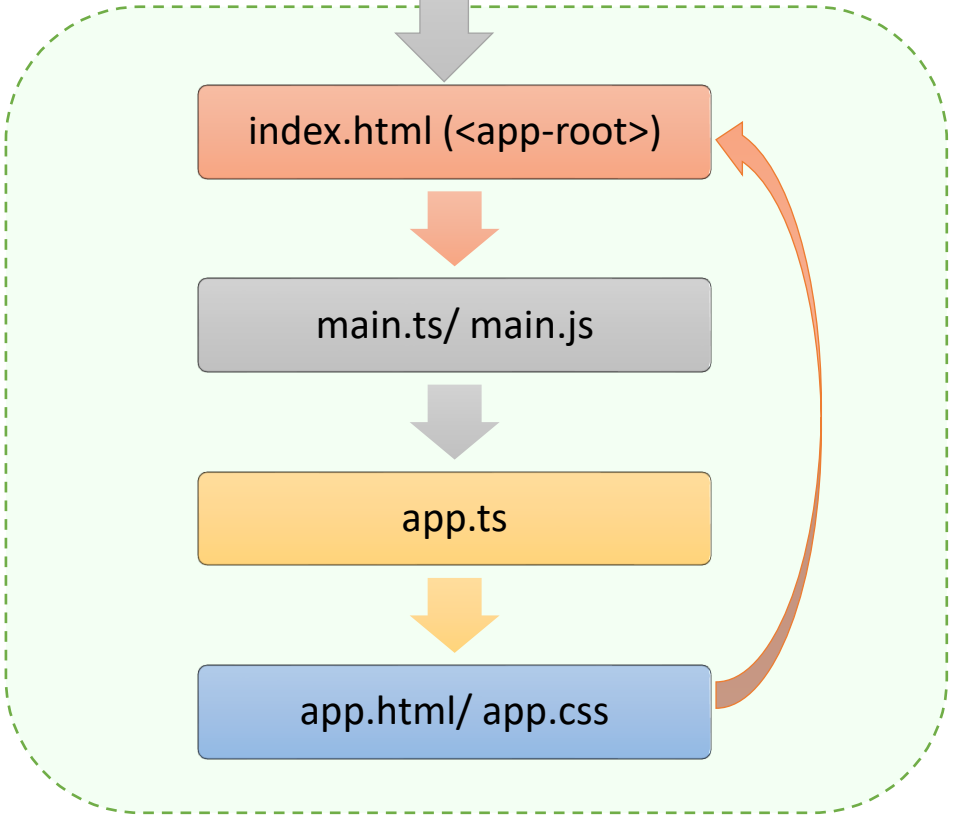
Before Angular Version 20





Angular Version 20 Onwards





Q. What changes were made in project files and folder structure after Angular 20?



Before Angular Version 20

```
my-app/
├── node_modules/          All dependencies
├── package.json           Dependency & script list
├── angular.json           Angular project config
├── tsconfig.json          TypeScript compiler config
├── tsconfig.app.json      App TypeScript config
├── tsconfig.spec.json
├── src/
│   ├── index.html         Main HTML entry point
│   ├── main.ts            Bootstraps AppModule
│   ├── styles.css         Global styles
│   ├── assets/            Static images/assets
│   └── app/
│       ├── app.module.ts          Root module (NgModule) ❌
│       ├── app.component.ts       Root component logic
│       ├── app.component.html     Root component template
│       ├── app.component.css      Root component styles
│       └── app-routing.module.ts  Routing module ❌
```

Angular Version 20 Onwards

```
my-app/
├── node_modules/
├── package.json
├── angular.json
├── tsconfig.json
├── public/                New place for static assets
├── src/
│   ├── index.html         Main entry HTML (same)
│   ├── main.ts            Uses bootstrapApplication()
│   ├── styles.css         Global styles
│   └── app/
│       ├── app.ts         Root standalone component
│       ├── app.html       Template of root component
│       ├── app.css        Styles of root component
│       ├── app.config.ts  Application configuration
                           (router, providers)
```

Q. What changed in the role of the main.ts file after Angular 20?



- Before Angular 20 → main.ts bootstrapped AppModule (module-based).

```
// main.ts before Angular 20
import { platformBrowserDynamic } from
'@angular/platform-browser-dynamic';
import { AppModule } from './app/app.module';

platformBrowserDynamic()
  .bootstrapModule(AppModule)
  .catch(err => console.error(err));
```

- After Angular 20 → main.ts directly bootstraps AppComponent (standalone approach).

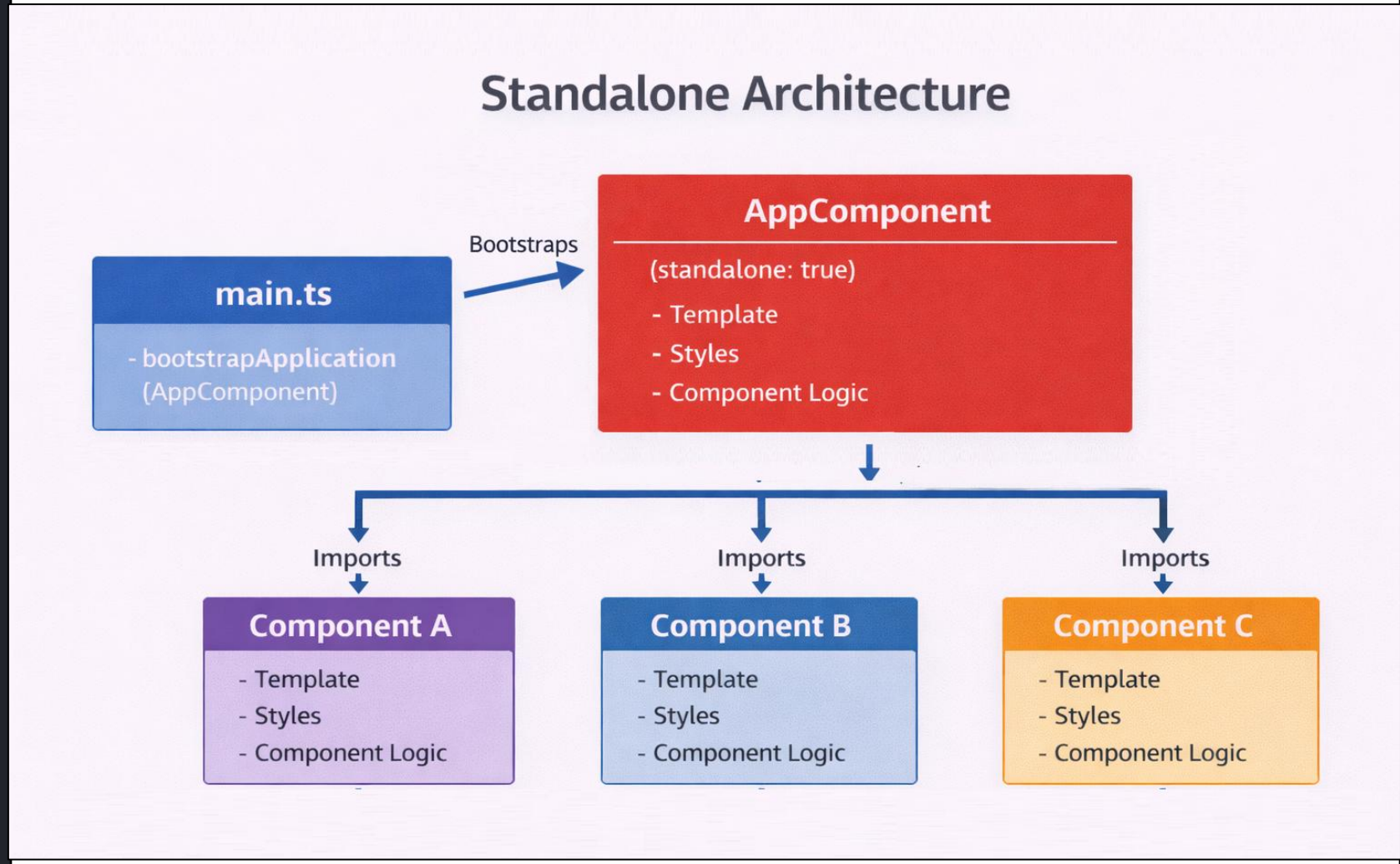
```
// main.ts Angular 20 Onwards

import { bootstrapApplication } from
'@angular/platform-browser';

bootstrapApplication(App,
appConfig).catch((err) => console.error(err));
```

Standalone Architecture

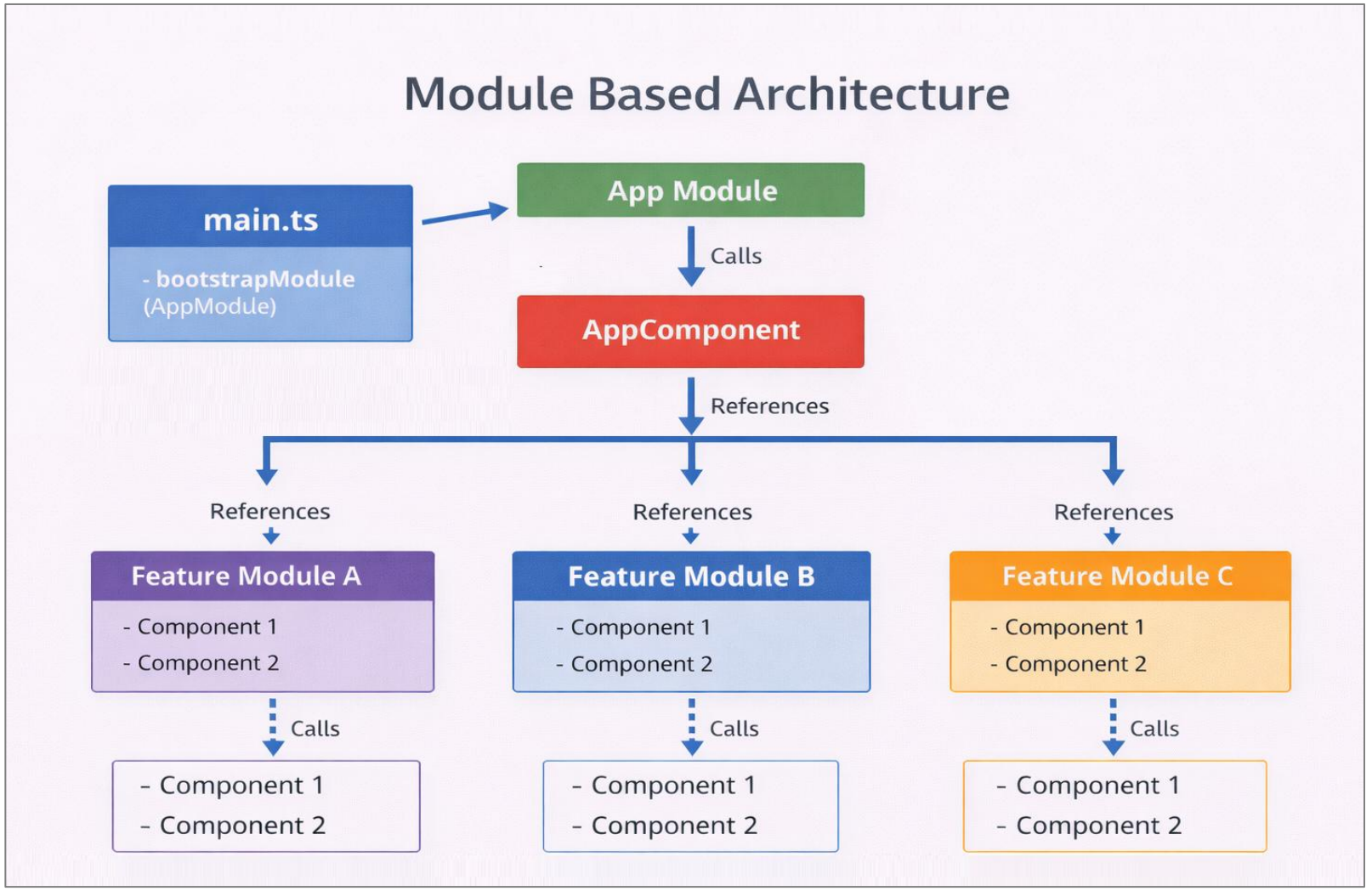
- 👉 Standalone Architecture
- 👉 Standalone APIs



Q. What is Standalone Architecture? Difference btw modular and standalone architecture?



- Module-Based Architecture means the application is **structured around modules**, and components can only be used through modules - not directly.



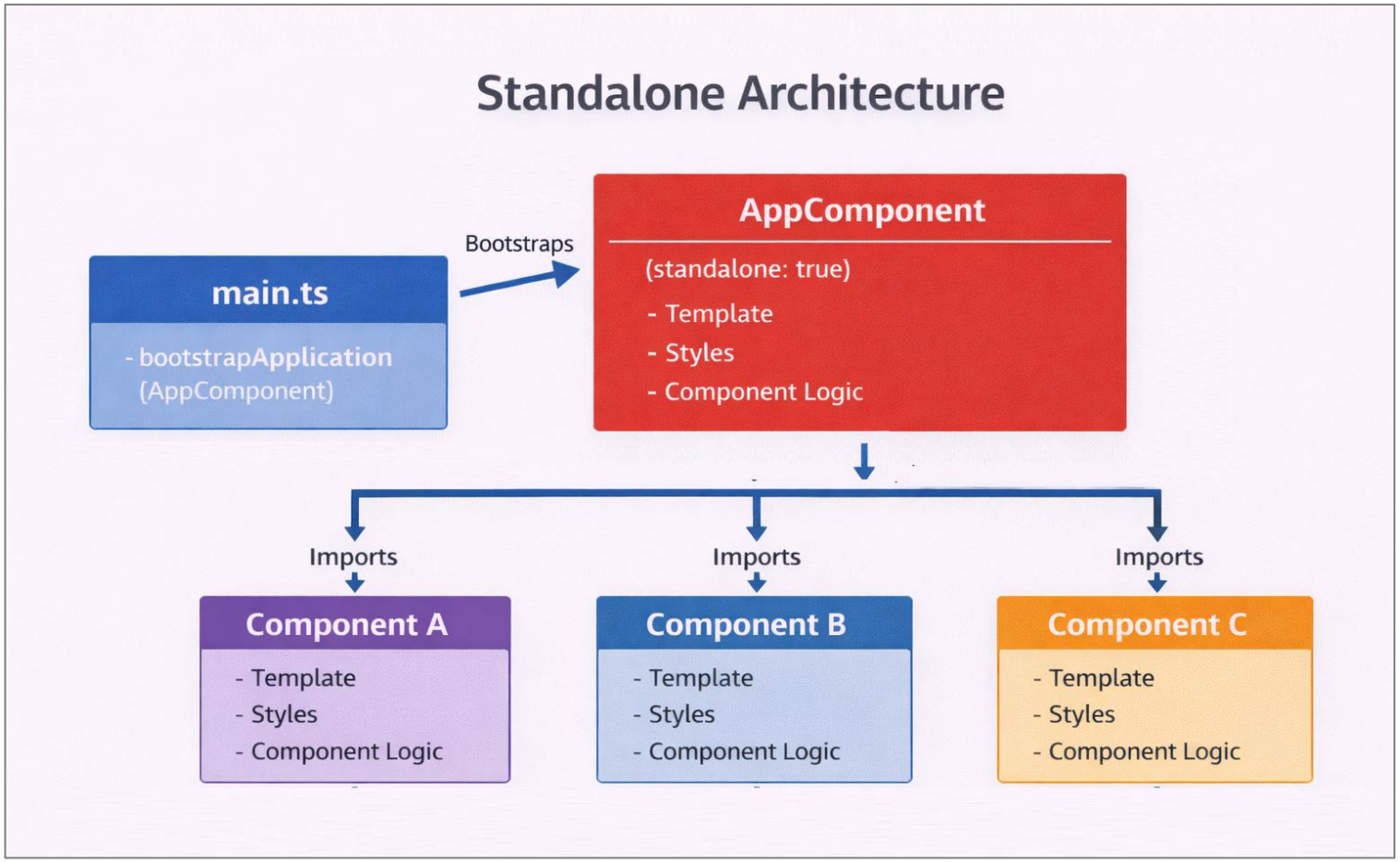
Q. What is Standalone Architecture? Difference btw modular and standalone architecture?



- Standalone Architecture is an Angular architecture where **components can function independently** without modules

- Advantages of Standalone:

1. Less boilerplate (no module code)
2. Simpler structure (no extra files)
3. Faster development (less code)



Q. What are Standalone APIs in Angular?



- Standalone APIs are Angular APIs (features) that help build standalone applications (without modules). For example, standalone: true, bootstrapApplication(), and provideRouter() etc.

```ts
// main.ts
import { bootstrapApplication } from '@angular/pl
import { appConfig } from './app/app.config';
import { App } from './app/app';

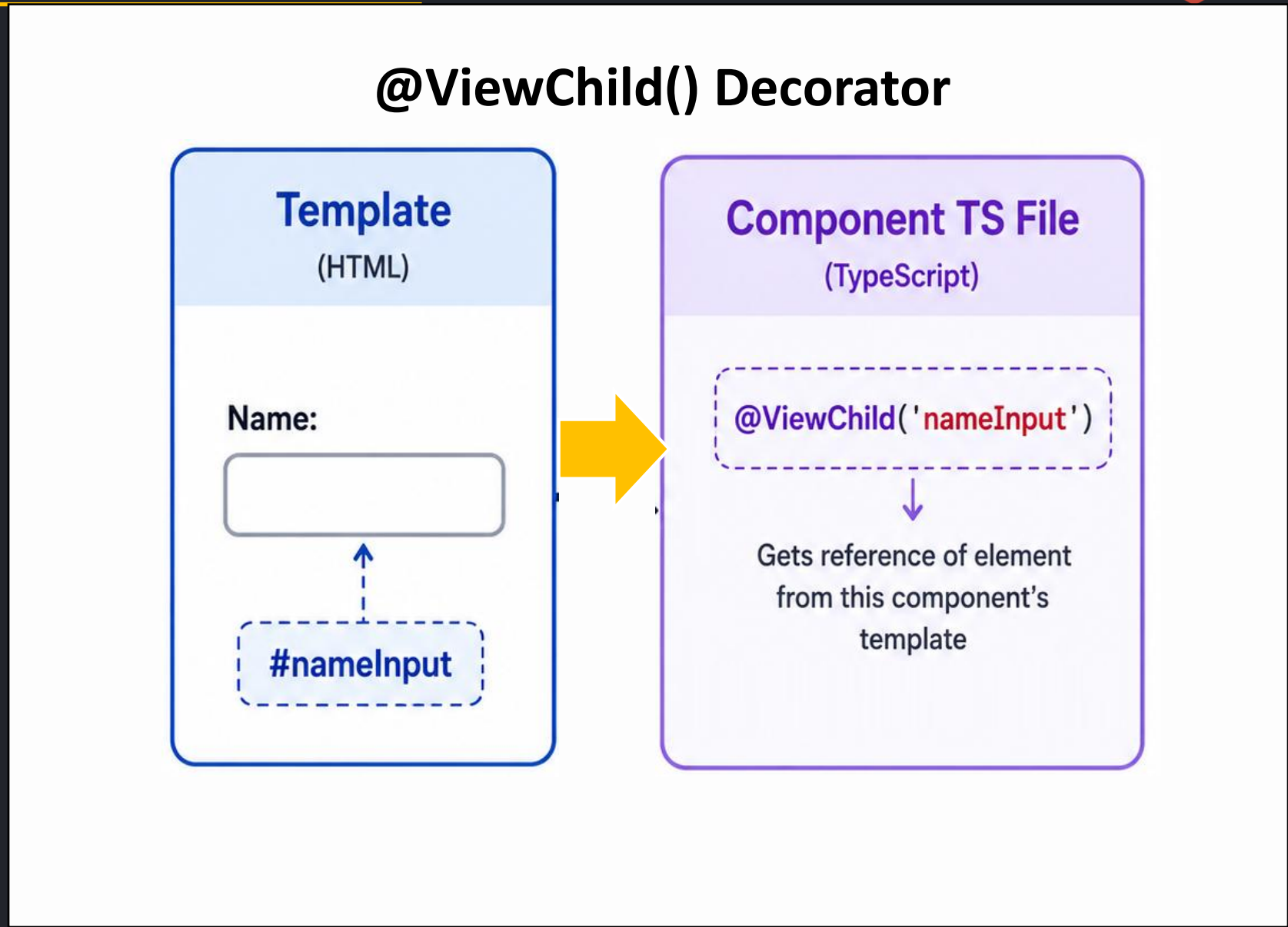
bootstrapApplication(App, appConfig)
  .catch((err) => console.error(err));
```

```ts
//app.ts
import { Component, signal } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet],
  templateUrl: './app.html',
  styleUrl: './app.css'
})
export class App {
  protected readonly title = signal('my-first-app');
}
```

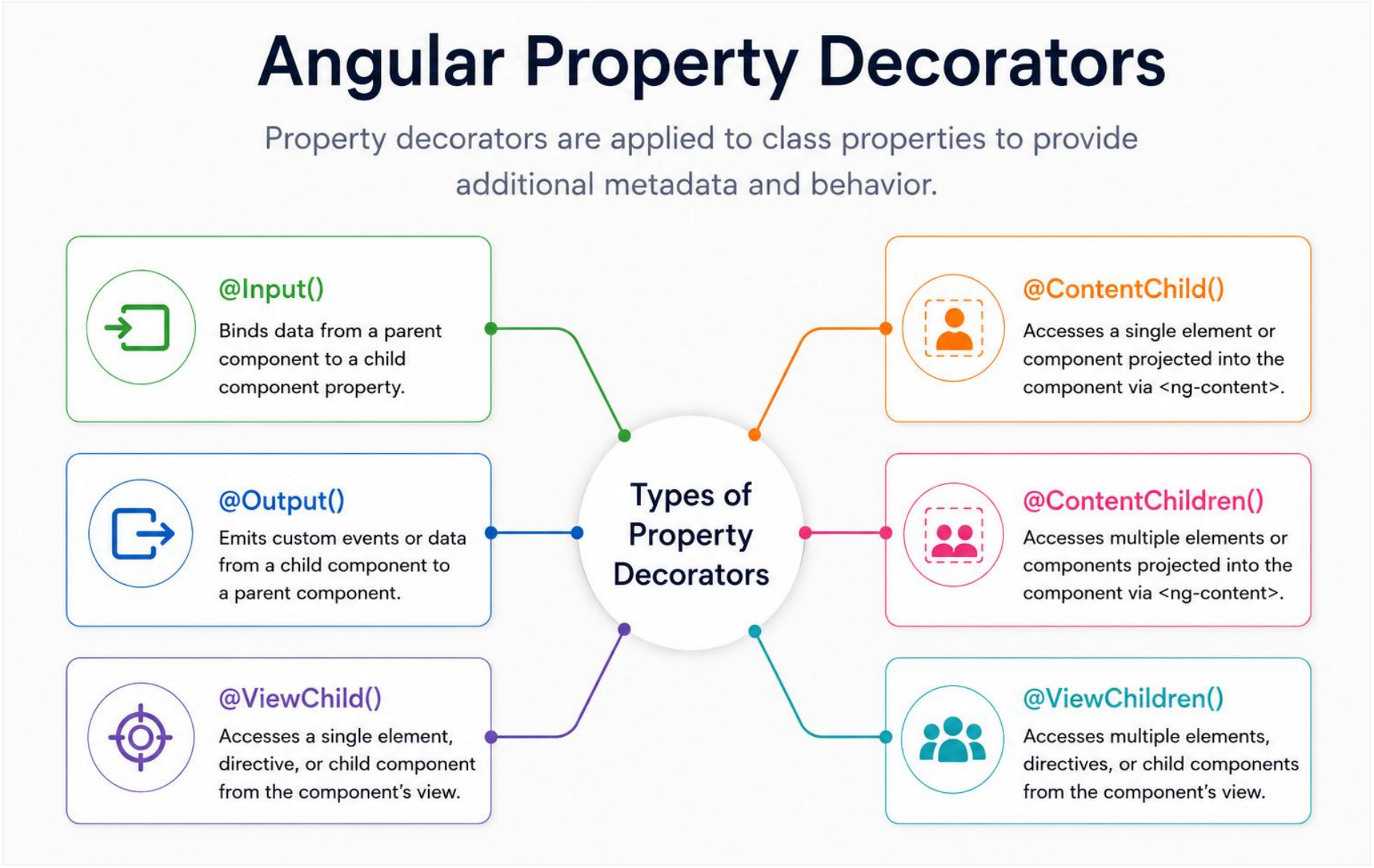
Property Decorators - ViewChild

- 👉 @ViewChild()
- 👉 Definite Assignment Assertion operator(!)
- 👉 ElementRef



Q. Explain the types of Property Decorators?





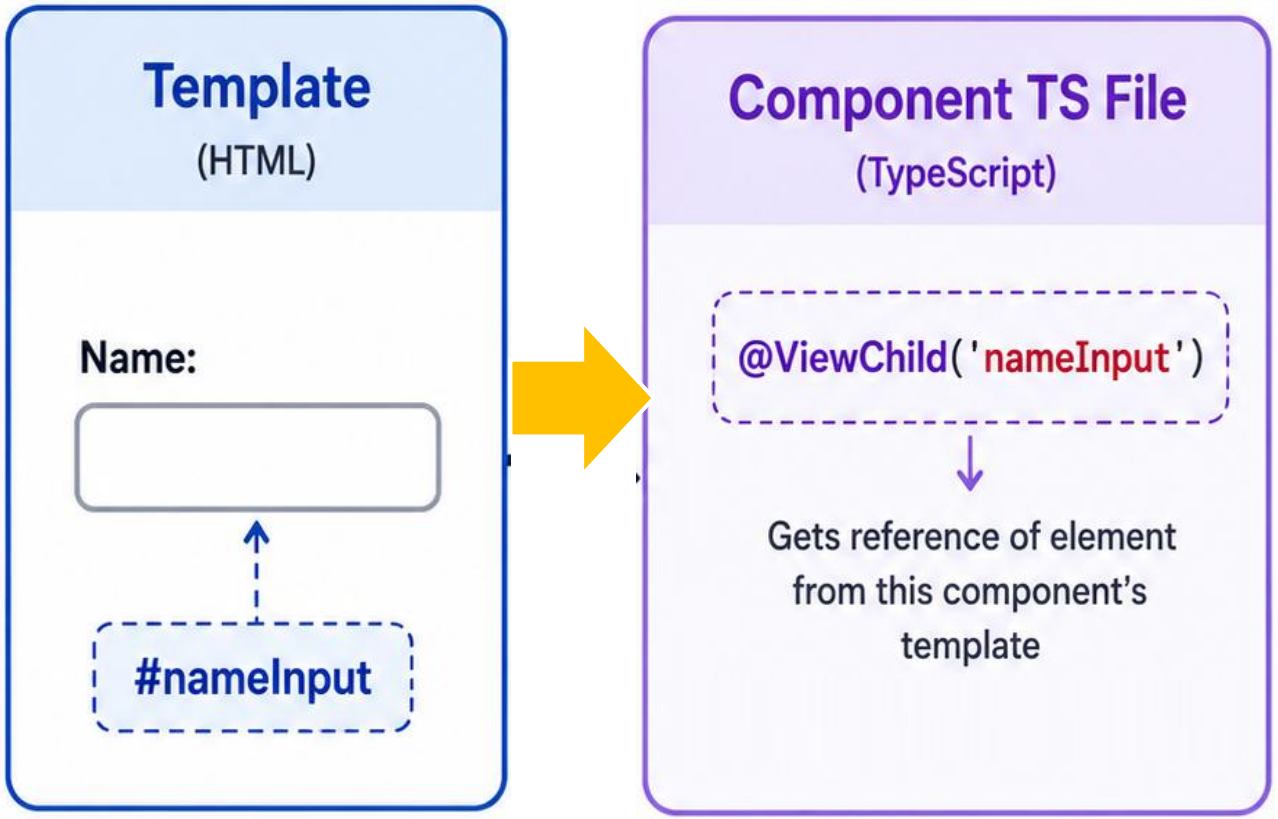
Q. What is the @ViewChild() decorator in Angular?



- **@ViewChild()** is an Angular property decorator used inside a component's TypeScript class **to access an element**, directive, or child component from the component's own HTML template.

- A **template reference variable gives a name** to an element in an Angular HTML template using the # symbol.

@ViewChild() Decorator



Q. How do you implement @ViewChild()? Give some example?



Steps to implement `@ViewChild()`:

1. Add a template reference variable to the HTML element.
2. Use `@ViewChild()` in the component TypeScript class.
3. Store the element reference in an `ElementRef` property.
4. Access the actual element using `nativeElement`.
5. Call the required method, such as `focus()`.

```html
<!-- .html -->
<input #nameInput type="text">
<button (click)="focusInput()">
  Focus Input
</button>
```



```ts
// .ts
export class ViewChildEx {

  @ViewChild('nameInput')
  nameInput!: ElementRef<HTMLInputElement>;

  focusInput(): void {
    console.log(this.nameInput);
    console.log(this.nameInput.nativeElement);
    this.nameInput.nativeElement.focus();
  }
}
```

Q. What is the role of Definite Assignment Assertion operator(!)?



- Definite assignment operator (!) tells TypeScript that a **property value will be assigned later**, so it should not show an initialization error.

- Example: `nameInput!: ElementRef;`

```
<!-- .html -->
<input #nameInput type="text">
<button (click)="focusInput()">
  Focus Input
</button>
```



```
// .ts
export class ViewChildEx {

  @ViewChild('nameInput')

  nameInput!: ElementRef<HTMLInputElement>;

  focusInput(): void {
    console.log(this.nameInput);
    console.log(this.nameInput.nativeElement);
    this.nameInput.nativeElement.focus();
  }
}
```

Q. What is the role of ElementRef?



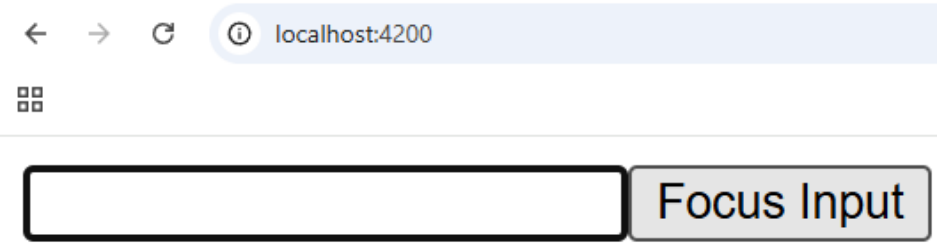
- **ElementRef** is an Angular class that holds a reference to an HTML element.
- **nativeElement** access to the actual browser HTML element inside ElementRef.

```html
<!-- .html -->
<input #nameInput type="text">
<button (click)="focusInput()">
  Focus Input
</button>
```

```ts
// .ts
export class ViewChildEx {
  @ViewChild('nameInput')

  nameInput!: ElementRef<HTMLInputElement>;

  focusInput(): void {
    console.log(this.nameInput);
    console.log(this.nameInput.nativeElement);
    this.nameInput.nativeElement.focus();
  }
}
```



Angular lifecycle hook - ngAfterViewInit()

👉 ngAfterViewInit()

👉 ngAfterViewInit() with @ViewChild()

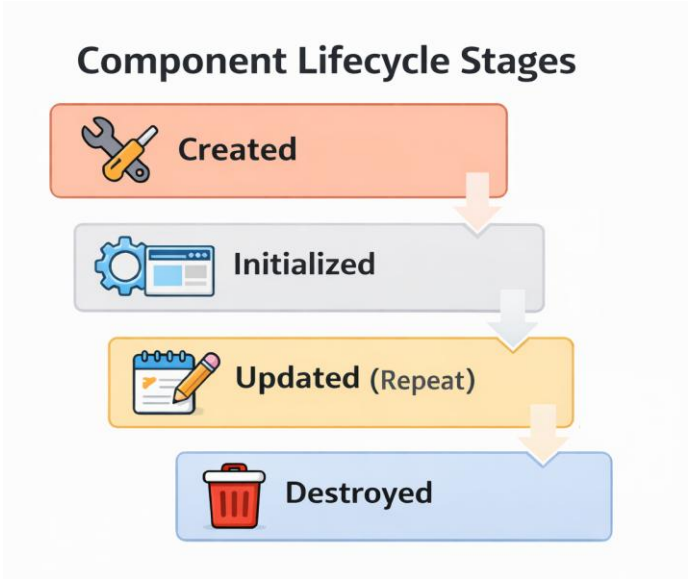


Q. What is ngAfterViewInit() Angular lifecycle hook? When to use it?



- ngAfterViewInit() is an Angular lifecycle hook that runs once **after Angular has fully initialized the component's view** and child elements.

- ngAfterViewInit lifecycle hook is commonly used to safely access **@ViewChild() or @ViewChildren() references**.





Q. How to implement ngAfterViewInit() with @ViewChild()?



Steps to implement `ngAfterViewInit()` with `@ViewChild()`:

1. Add a template reference variable to the HTML element.
2. Access it using `@ViewChild()` in the component class.
3. Implement the `AfterViewInit` interface.
4. Add the `ngAfterViewInit()` method.
5. Access the element inside `ngAfterViewInit()` because the component view is fully initialized at that point.

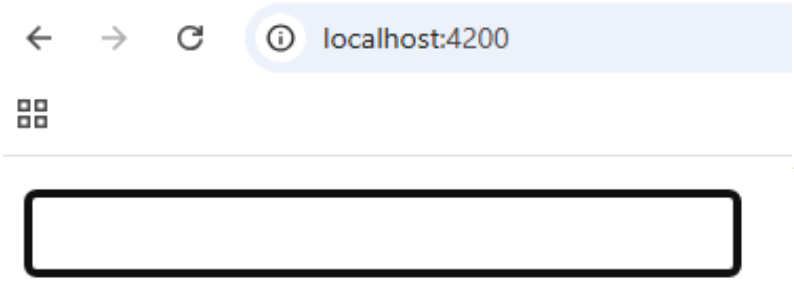
```
<!-- .html -->
<input #nameInput type="text">
```

```
export class App implements AfterViewInit {
  @ViewChild('nameInput')
  nameInput!: ElementRef<HTMLInputElement>;

  // Cannot read properties of
undefined('nativeElement')
  // constructor()
  // {   this.nameInput.nativeElement.focus(); }

  // ngOnInit(): void {
  //   this.nameInput.nativeElement.focus();
  // }

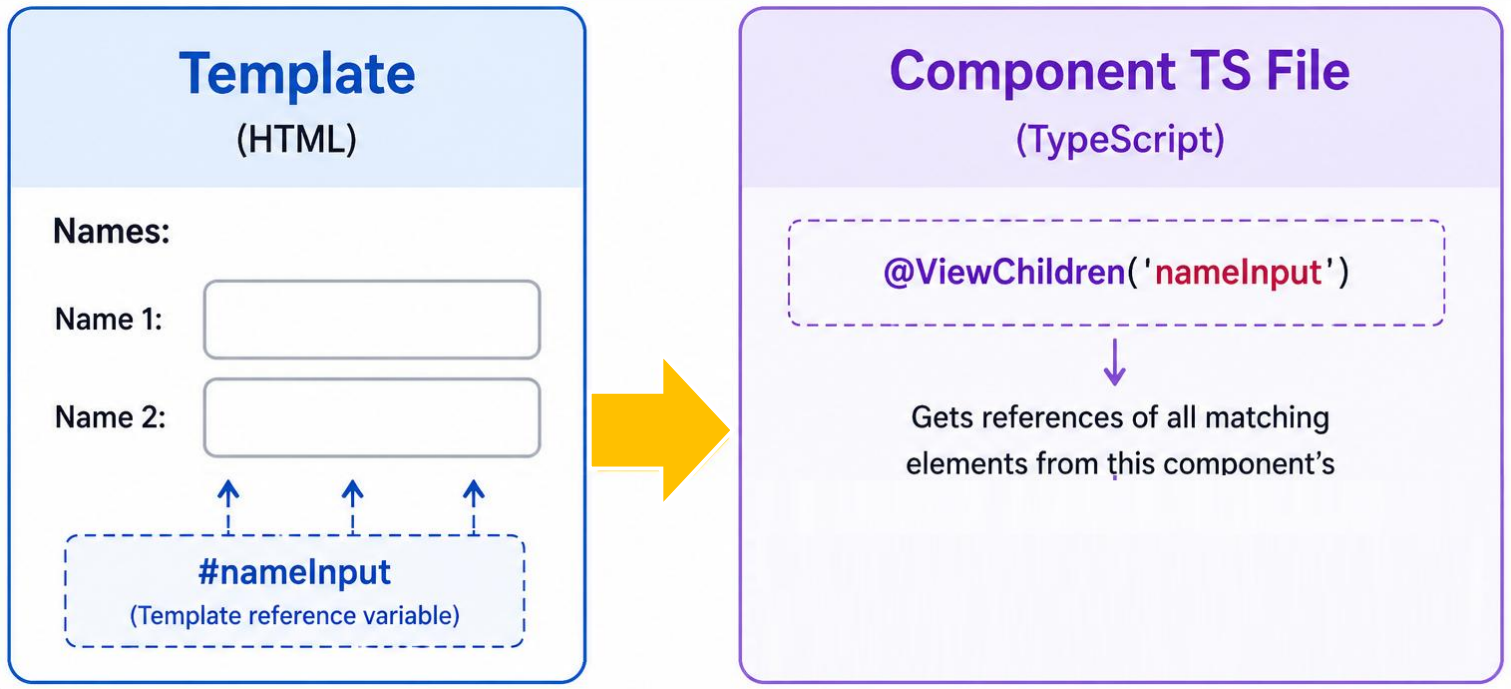
  // Able to read properties of 'nativeElement'
  ngAfterViewInit(): void {
    console.log(this.nameInput.nativeElement);
    this.nameInput.nativeElement.focus();
  }
}
```



Property Decorators - ViewChildren

👉 @ViewChildren()

👉 QueryList with @ViewChildren()

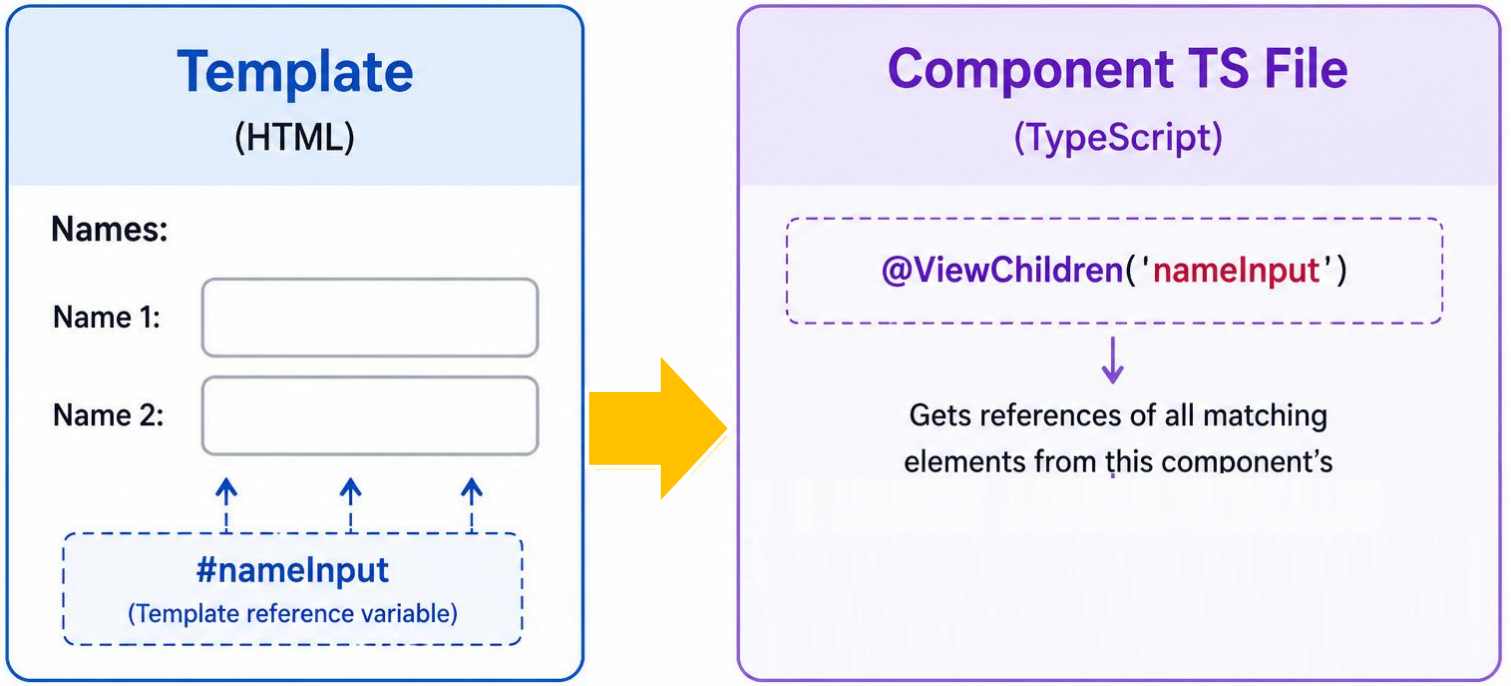


Q. What is the @ViewChildren() decorator in Angular?



- @ViewChildren() is an Angular property decorator used to **get references to all matching HTML elements**, directives, or child components from a component's own template.

@ViewChildren() Decorator



Q. Explain QueryList role in @ViewChildren()?



- QueryList is an Angular class that stores a **collection of multiple elements**, directives, or child components found by @ViewChildren() or @ContentChildren().

```html
<!-- .html -->
<input #nameInput type="text" />
<input #nameInput type="text" />
<button (click)="clearAllInputs()">Clear All</button>
```

```ts
// .ts
export class ViewChildrenEx {
  @ViewChildren('nameInput')

  inputs!: QueryList<ElementRef<HTMLInputElement>>;

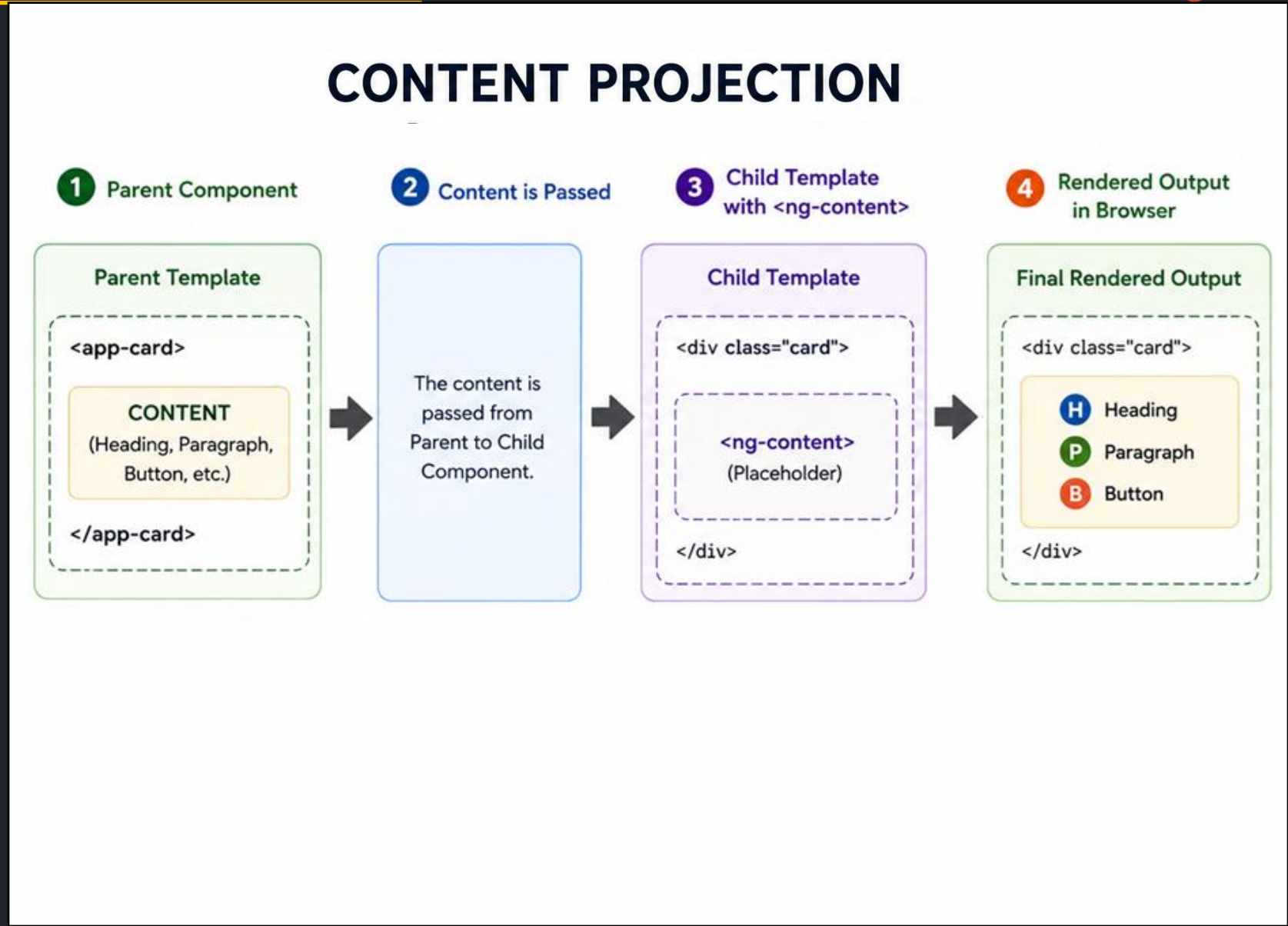
  clearAllInputs(): void {
    this.inputs.forEach((input) => {
      input.nativeElement.value = '';
    });
  }}
```



Content Projection

👉 Content Projection

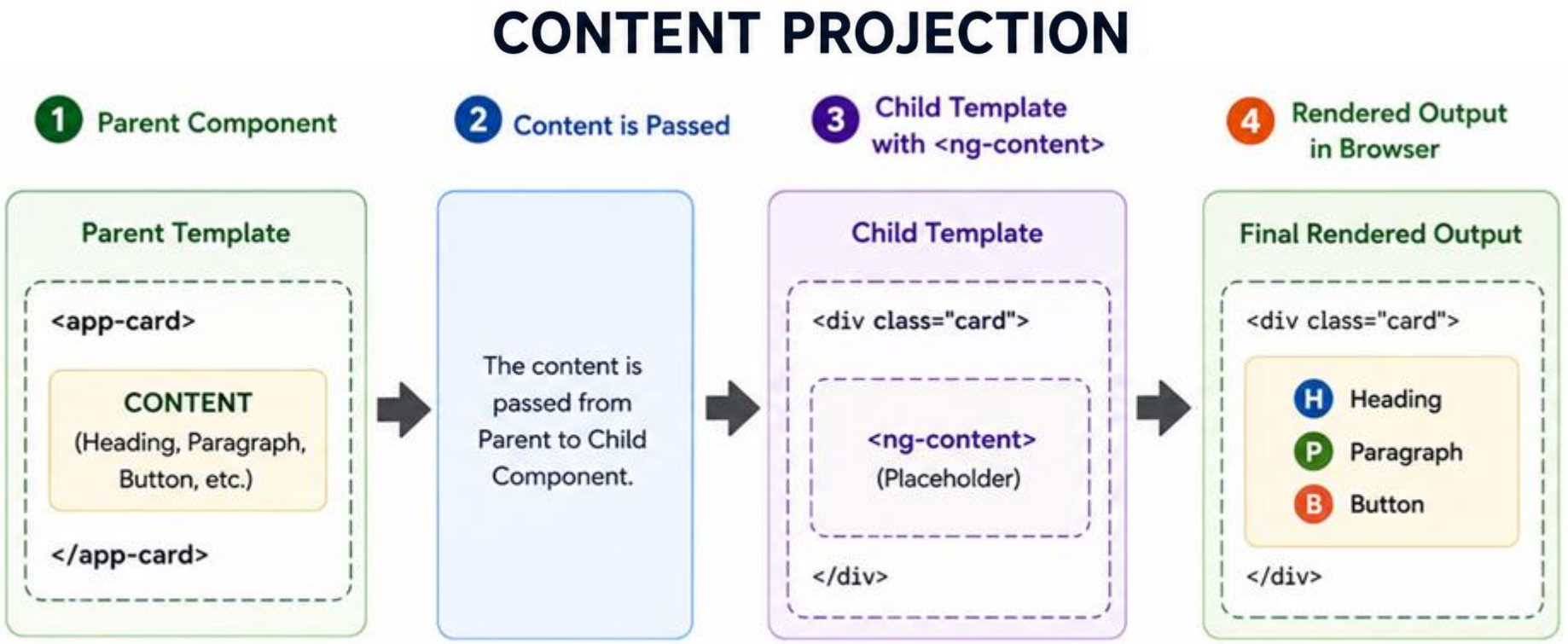
👉 ng-content role in content projection



Q. What is Content Projection?



- Content projection is an Angular allows a parent component to pass **HTML content** into a child component using ng-content.



Q. What is the role of ng-content in content projection?



- <ng-content> is an Angular element used in a child component's template to display HTML content passed from the parent component.
  OR
- In content projection, <ng-content> acts as a **placeholder** in the child component for content projected from the parent component.

```
<!-- parent component/ app.html --><div
class="parent-box">

  <h2>Parent Component</h2>

  <app-content-child-ex>

     <h3>Interview Happy</h3>
     <p>Learn Angular</p>

  </app-content-child-ex>

</div>
```

```
<!--content-child-ex.html -->
<div class="child-box">
  <h2>Child Component</h2>
  <ng-content></ng-content>
</div>
```

Parent Component

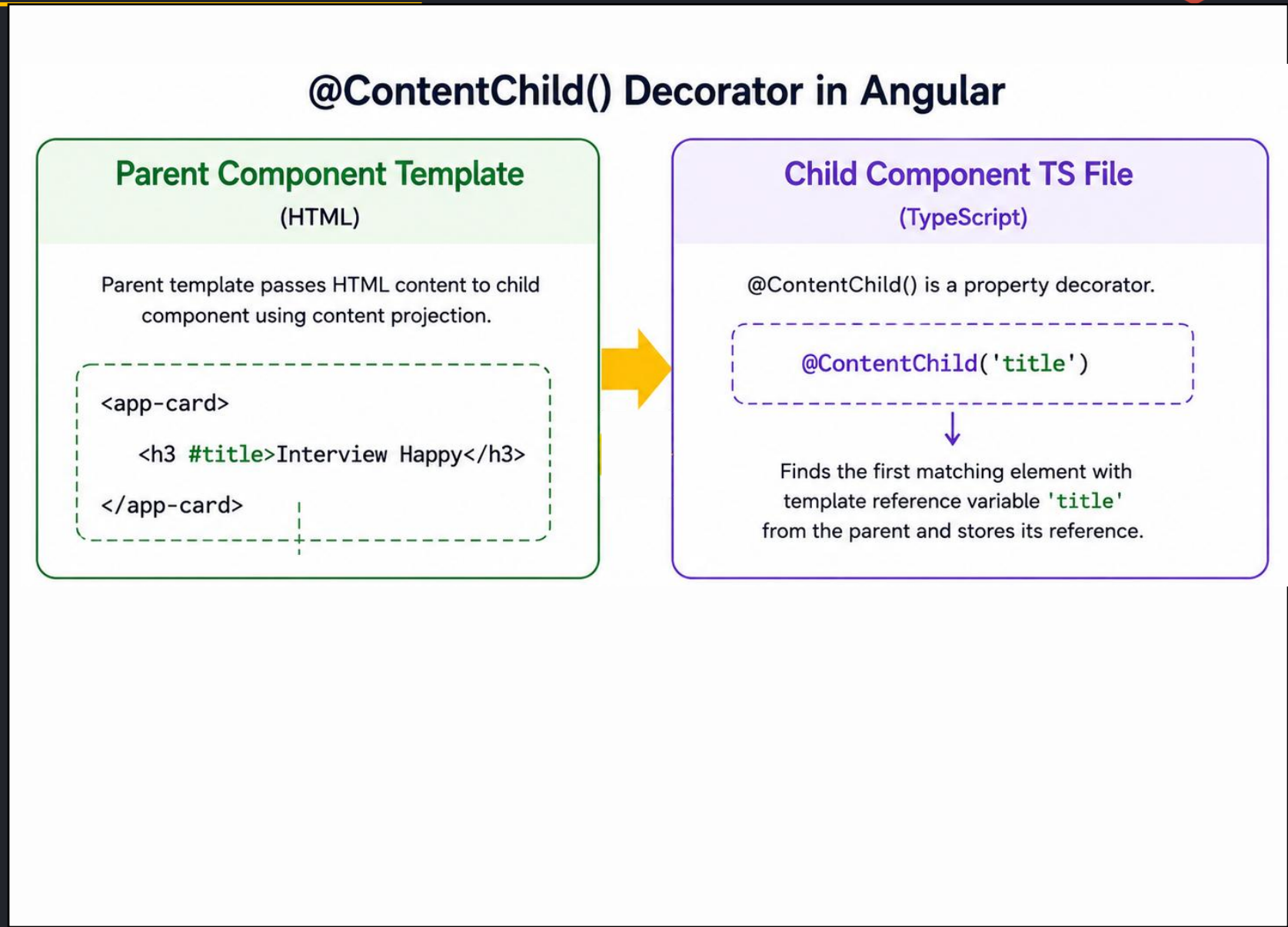
Child Component

Interview Happy

Learn Angular

ContentChild and ContentChildren

- 👉 @ContentChild()
- 👉 ngAfterContentInit()
- 👉 @ContentChildren()



Q. What is the @ContentChild Decorator?



- @ContentChild() is an Angular decorator **used in a child component's TypeScript class to access the first projected element**, directive, or component passed from the parent through <ng-content>.

@ContentChild() Decorator in Angular

Parent Component Template

(HTML)

Parent template passes HTML content to child component using content projection.

```
<app-card>
   <h3 #title>Interview Happy</h3>
</app-card>
```

Child Component TS File

(TypeScript)

@ContentChild() is a property decorator.

```
@ContentChild('title')
```

↓

Finds the first matching element with template reference variable 'title' from the parent and stores its reference.

Q. What is the role of ngAfterContentInit() lifecycle hook?



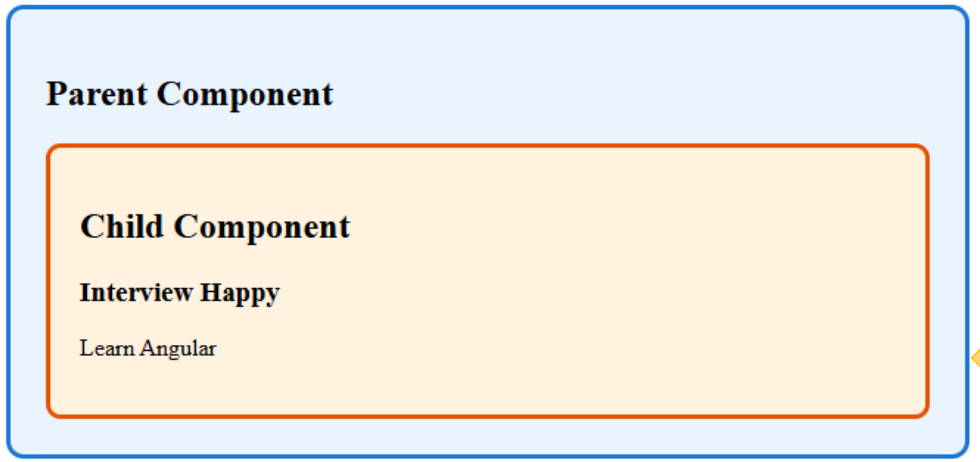
- ngAfterContentInit() is an Angular lifecycle hook that runs once after the content passed from a parent component is projected into the child component using <ng-content>.

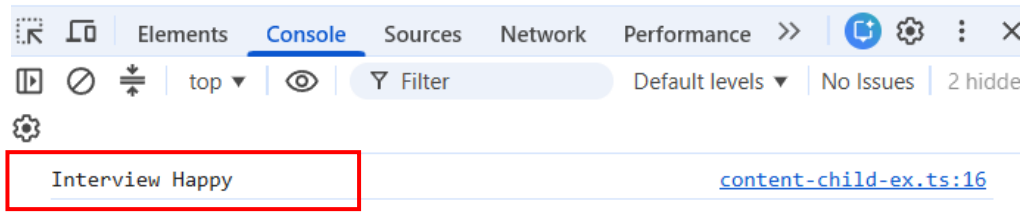


Q. What is the role of ngAfterContentInit() lifecycle hook?



- When to use ngAfterContentInit lifecycle hook?
- ngAfterContentInit() is mainly **used with @ContentChild() and @ContentChildren()** to safely access content projected through <ng-content>.






```
<!--app.html -->
<div class="parent-box">
  <h2>Parent Component</h2>
  <app-content-child-ex>
    <h3 #title>Interview Happy</h3>
    <p>Learn Angular</p>
  </app-content-child-ex>
</div>
```

```
<!--content-child-ex.html -->
<div class="child-box">
  <h2>Child Component</h2>
  <ng-content></ng-content>
</div>
```

```
// content-child-ex.ts
export class ContentChildEx implements AfterContentInit {
  @ContentChild('title')
  title!: ElementRef<HTMLHeadingElement>;

  ngAfterContentInit(): void {
      console.log(this.title.nativeElement.textContent);
  }}
```

Q. What is the @ContentChildren Decorator?



- @ContentChildren() is an Angular property decorator used in a child component to access **all matching elements**, directives, or components projected from the parent component.

@ContentChildren() Decorator in Angular

Parent Component Template

(HTML)

Parent template passes multiple HTML elements to the child component using content projection.

```
<app-list>
    <h3 #item>Angular</h3>
    <h3 #item>is</h3>
    <h3 #item>Awesome</h3>
</app-list>
```

Child Component TS File

(TypeScript)

@ContentChildren() is a property decorator.

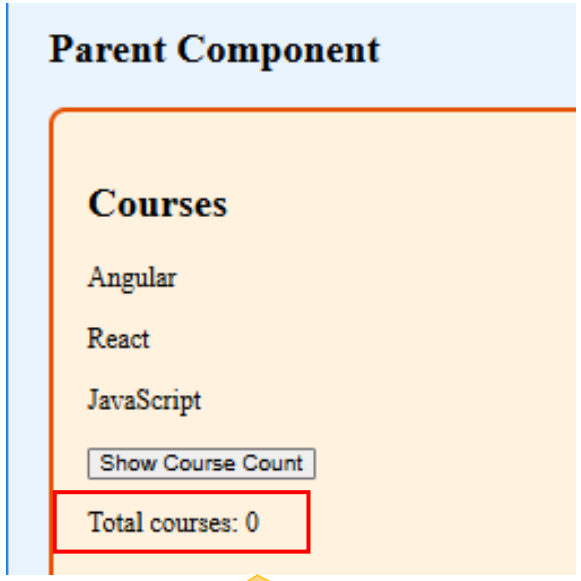
```
@ContentChildren('item') items!:
    QueryList<ElementRef<HTMLHeadingElement>>
```

↓

Finds all matching elements with template reference variable 'item' from the parent and stores them in a QueryList. QueryList is an iterable list that updates automatically if the projected content changes.

Q. How do you implement @ContentChildren Decorator?






```
<!--app.html -->
  <h2>Parent Component</h2>
  <app-content-children-ex>
    <p #course>Angular</p>
    <p #course>React</p>
    <p #course>JavaScript</p>
  </app-content-children-ex>
```

```
// content-children-ex.ts
export class ContentChildrenEx {
  @ContentChildren('course')

  courses!:
QueryList<ElementRef<HTMLParagraphElement>>;
  courseCount = 0;
  showCourseCount(): void {
    this.courseCount = this.courses.length;
  }}
```

```
<!--content-children-ex.html -->
<div class="child-box">
  <h2>Courses</h2>
  <ng-content></ng-content>
  <button type="button" (click)="showCourseCount()">Show
Course Count</button>
  <p>Total courses: {{ courseCount }}</p>
</div>
```

Final Quick Revision Sheet

1. Angular → Frontend framework for building SPA applications
2. TypeScript → Strongly typed language used in Angular
3. Components → Building blocks of Angular UI
4. Decorators → Provide metadata to Angular classes
5. Data Binding → Connect component data with UI
6. Directives → Modify DOM structure or behavior
7. Pipes → Transform data inside templates
8. Services → Share logic/data across components
9. DI (Dependency Injection) → Inject dependencies automatically
10. Routing → Navigate between Angular pages/components

Final Quick Revision Sheet

11. Observable → Handle asynchronous data streams
12. RxJS → Reactive programming library for Observables
13. HttpClient → Call backend REST APIs
14. Lifecycle Hooks → Execute logic during component lifecycle
15. Template-Driven Forms → Simple form handling using ngModel
16. Reactive Forms → Scalable and structured form handling
17. JWT Authentication → Secure user authentication using tokens
18. Route Guards → Control route/page access
19. Interceptor → Modify HTTP request/response globally
20. Signals → Reactive state management in Angular
21. Standalone Architecture → Build Angular apps without NgModules

MESSAGE FROM THE AUTHOR

MESSAGE FROM THE AUTHOR



Happy Rawat

Founder, Interview Happy

Mentoring software professionals for technical interviews

Remember, there is only one way to crack interviews:

Keep giving interviews and NEVER GIVE UP.

Every interview will teach you something new.
Keep learning and improving every day.

Every rejection will make you stronger.
Don't stop—growth happens outside the comfort zone.

And one day, all your hard work will finally pay off.
Stay consistent, stay positive, and believe in yourself.

I wish you all the best for your interviews and your software engineering career.

— *Happy Rawat*
Founder, Interview Happy

STUDENT SUCCESS STORIES

SUCCESS STORIES

From **Interview Happy** Students



Hello Sir. After watching all Your videos related to Dotnet I got placed for 34.5 LPA. This is my 2nd company and it helps me a lot after attending interviews after 10 years. Thank you!!!!!

@Yousi_Life



Thank you sir. I got a job with 8 LPA. I joined on 6th August as an Associate software engineer. I followed your video.

Najmur Hossain



Best and precise explanation of solid principles available on youtube... Thank you very much Sir, Your videos has helped me a lot in cracking 7 interviews. Finally I have got 200% hike. I have gone through your other videos like .net core interview questions, web api, c#, and sql server interview questions series.... and many more... You are gem for us..... Thank you so very much...

Aashna Vijay



Hi Sir, I got the offers from 4 companies. All thanks to you!

Raghav sharma

Keep Learning • Keep Practicing • Keep Succeeding